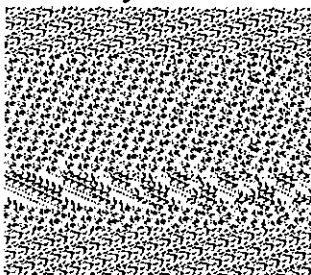


VECTOR

Mostly About ...



Including ...

- | | |
|------------------------------|-----|
| • Ziemann reviews J 2.05 | 53 |
| • Iverson teaches J (Part 1) | 76 |
| • Clough draws Stereograms | 110 |

*The Journal of the
British APL Association*



A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.11 No.4 April 1995

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production), and Winword-2.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1994-95

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	50	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 01439-788385 CompuServe: 100331,644

Contents

Editorial:	Anthony Camacho	Page 3
APL NEWS		
Quick Reference Diary		5
Correspondence		7
APL 95 Panels and Workshops		8
Abstracts		10
News from Sustaining Members	Gill Smith	14
The Education Vector	Ian Clark	17
REVIEWS SECTION		
APL Product Guide — Complete	Gill Smith	34
Locales in J	Chris Burke	49
J 2.05 for Windows	David Ziemann	53
RECENT MEETINGS		
APL in a Client-Server Environment		
BAA Workshop report	Jon Sandles	62
GENERAL ARTICLES: and J Feature		
Here be Dragons	Sylvia Camacho	69
Teaching with an Executable Notation	Ken Iverson	76
The Perils of Subtraction	Donald B. McIntyre	93
Primes, Spirals and Coffee Tables	Keith Smillie	104
Stereograms in J	E.A. Clough	110
TECHNICAL SECTION		
Puzzle Corner: The Age of the Vicar (solution)	Alan Sykes	118
Technical Correspondence		119
At Work and Play with J: Cribbage 15s	Gene McDonnell	135
Functional Programming in J for Operations Research	Richard Brown	139
J 2.05 for Windows (concluded)	David Ziemann	155
Index to Advertisers		159

DYALOG APL FOR WINDOWS



NOW AVAILABLE IN THE U.S.



FROM BEAUTIFUL SYSTEMS, INC.



Superb, state-of-the-art APL just a toll-free call away

Dyalog APL/W, the first true Windows APL, offers the rich capabilities and robust performance you'd expect from the industry leader. With its two year headstart, Dyalog is still light-years ahead of the competition. Only with Dyalog APL/W do you get the following as standard equipment:

BUILT-IN SPREADSHEET

Only Dyalog offers its own grid object which gives sophisticated, yet simple, spreadsheet functionality as an intrinsic part of its graphical user interface.

NAMESPACES

Only Dyalog offers object-oriented encapsulation through namespaces. The modular architecture afforded by this Dyalog innovation enables the tighter design of workspace-based APL applications.

FULLY CUSTOMIZABLE SESSION

Only Dyalog offers a fully customizable session allowing the personalization of your working environment for maximum productivity.

ODBC FOR EVERYONE

Only Dyalog includes ODBC support with every interpreter. Open Database Connectivity, the Microsoft standard, provides convenient access to a wide variety of databases from within APL. ODBC easily serves as the foundation for numerous client-server architectures.

These are just some of the advantages you get with Dyalog.

Fully-compatible UNIX versions also available.

30-DAY MONEY-BACK GUARANTEE

TO ORDER CALL 1-800-732-3735



Beautiful Systems, Inc.

VOICE: 215-732-3735 FAX: 215-772-1929 EMAIL: 73567.1016@Compuserve.com

308 Old York Road, Suite 4, Jenkintown, PA 19046 USA

Microsoft is a registered trademark, Windows and the Windows Logo are trademarks of Microsoft Corporation.

Dyalog APL is a trademark of Dyalog Systems Limited.



Editorial

by Anthony Camacho

The virtues of APL are that it is a complete replacement for conventional mathematical and logical notations (in their algorithmic roles) and that it is superior to those notations in the following respects:

Unambiguous	Concise	Consistent
Rich	Many uses	Easily written and read
Low threshold	Many learning paths	High ceiling

How does J compare? This issue of Vector is on the theme of J; that alone needs justification when some of our members wish Vector contained less J (see the report on the reader survey). There is a feast here for those who can see more to APL and J than earning their daily bread. The virtues of J are certainly worth a moment's thought.

J is as unambiguous as APL; it is more concise, more consistent, richer, has more uses, is arguably easier to read and write as it has no special symbols, also has a low beginners' threshold — arguably not quite so low — has many learning paths, has at least as high a ceiling. In J all the parts of speech are definable by the user, which makes it much more concise. Although many symbols of APL are rendered with two characters in J, this loss of brevity is more than made up by the extra power of each expression and the tacit form of programming. J has a more consistent syntax than any version of APL; it is amazingly simple and the only distinctions are between parts of speech. A part of speech you define ranks in every way the equal of a primitive. J is richer and has more uses. The richness is obvious to a reader of the Dictionary of J especially if he holds the Dictionary of APL in the other hand. It would be a surprise if the extra richness didn't lead to more uses, but on the other hand, APL could be used for everything and what other uses are there?

The problems with J are with the learning curve and the upward path. It is not that there is any obvious reason why J ought to be harder than APL but many of us have found it so. There is not as wide a choice with J learning materials as with APL, though Vector does its bit to increase the variety, and some of the material there is is tough. Professor Donald McIntyre deserved the SigAPL Outstanding Achievement Award just for demonstrating that even a complete APL diehard (which he was, you know) can learn J and get to love it. Donald has provided the best way available to learn J in his many tutorials and articles. There is a splendid example for you inside.

Not just a pretty interface...

J has the capabilities you need for today's software. If you want the benefit of advanced features other language vendors dream of, switch your development efforts to the new release of J.



Release 2 for Windows from Iverson Software combines an incredibly powerful programming language with the latest in Windows technology. J's clean design and lack of special requirements makes for a smooth fit with other Windows software, providing the most flexible environment ever for application development. Use VBX controls for graphics, spreadsheets and a host of custom controls, ODBC drivers for database access, DDE, OLE and DLL calls for general purpose inter-program communication. Why settle for less?

J is packaged in a sophisticated Windows integrated development environment, which features built-in script and forms editors and a session manager with customizable menus, support for multiple script and execution sessions, a simple yet effective interface to Windows, and access to virtually any database. Core language features include control structures and advanced support for locales and scripts.

The commercial license includes royalty-free runtime use. J Release 2 is also available for most other small systems, including DOS, OS/2, Linux, Mac, and UNIX workstations, and in low-cost personal and educational editions.

Discover how J can streamline your development efforts. Call today for more information!

Now available through Strand Software and authorized dealers. For more information, contact Anne Faust at Strand Software • 19235 Covington Court • Shorewood • Minnesota • USA 55331
Tel (612) 553-9382 • Fax (612) 559-2320 • Email amfaust@aol.com

Windows is a trademark of Microsoft Corporation. The J logo is a trademark of Iverson Software Inc.

Quick Reference Diary 1994-95

Date	Venue	Event
June 4th-8th 1995	San Antonio, Texas	APL95
July 7th	IEE, London	BAA Meeting (see below)

At the Committee meeting of 10 February 1995 it was decided that there was insufficient time and effort available to organize a Beginners Workshop for 25 March. Any such exercise cannot hope to succeed without considerable advance publicity as it must attract support from outside the established APL community. However, it was agreed that the Workshop held at the end of January 1995 should be repeated in October 1995. Please watch this space for details.

Please note that we have been unable to obtain the usual venue for the meeting of 19th May 1995 but speakers are being arranged for a replacement meeting on 7th July 1995. Details will be mailed as usual.

There will be an AGM on 19th May 1995 as announced but, on this occasion, without any following speakers. All members will have received a mailing of venue and Agenda and are welcome to attend.

British APL Association meetings are normally held in the IEE, Savoy Place. Nearest tube outlets: Temple or Embankment.

Dates for Future Issues of VECTOR

	Vol.12 No.1	Vol.12 No.2	Vol.12 No.3
Copy date	2nd June 95	1st Sept 95	1st Dec 95
Ad booking	9th June 95	8th Sept 95	8th Dec 95
Ad Copy	16th June 95	15th Sept 95	15th Dec 95
Distribution	July 95	October 95	January 96



SOLITON ASSOCIATES

SHARP APL... Your Competitive Edge

SHARP APL

is superior
in the rapid
development of
mission-critical
applications
which meet the
ever-changing
demands of our
customers.



Soliton provides SHARP APL for MVS and UNIX with:

- Superior productivity
- Ease in managing shared-file multi-user applications
- Powerful cooperative processing
- High performance DB2 interface
- Applications for end-users and programmers
- Responsive support services

visit our booth at APL95

For more information, telephone or FAX:

Soliton Associates Limited

44 Victoria Street, Suite 2100
Toronto, Ontario, Canada M5C 1Y2
Tel: (416) 364-9355 FAX: (416) 364-6159

In Europe

Groot Blankenberg 53
1082 AC Amsterdam
The Netherlands
Tel: + 31-20-646-4475
FAX: + 31-20-644-1206

In the U.S.

951 President Street, #3F
Brooklyn
New York, USA 11215
Tel: (718) 783-9330
FAX: (718) 783-1612

1100 University Avenue, Suite 111
Rochester
New York, USA 14607
Tel: (716) 256-6466
FAX: (716) 256-6469

or via Internet to: sales@soliton.com

Correspondence

From: John Sullivan

8 March 1995

I really must complain about the heading on the top of page 50 of Vol.11 No.2.

```
Croesoe a Cymru
SYNTAX ERROR
Croesoe a Cymru
^
```

APL is very pernickety about its syntax. So should APLers be when dealing with other languages. Welsh has some peculiarities which I shall not go into here, but suffice it to say that you cannot have a C on the start of a word following the word *a*, no matter which version of the word *a* you are using (it will begin with G or Ch). Assuming that we can mangle our syntax (or make a user-defined function called *a*) we now get the following:

```
Croesoe a Cymru
VALUE ERROR
Croesoe a Cymru
^
```

because Welsh does not have a symbol called *Croesoe* in its symbol table (vocabulary).

Enough of this idiocy: you can see what I'm getting at. Let's just say that I wish to complain that everybody tries to get their English (of whatever variant) correct, they try to get their APL (of whatever variant) correct, they even try to get their Russian correct. But nobody cares about getting their Welsh correct! This kind of thing is typical of the non-Welsh contempt of all things Welsh, and for a journal with delusions of academic excellence it just isn't good enough.

The correct heading should, of course, be

Croeso i Gymru.

P.S. Maybe Vector ought to contact the Welsh Office in Cardiff and ask about the availability of their Welsh language spell-checker. It won't find all of your errors, but it should find the ones I am complaining about here. Tel: 01222-488745.

APL 95

Dates: June 4-8 1995

Venue: St. Anthony Hotel, San Antonio, Texas

Costs and Administration:

	until 1 April	thereafter
ACM or SIGAPL member	\$335	\$435
Join-and-Register plan	390	490
Non-member	435	535
Accompanying adult	100	125
Accompanying child	75	75
Full-time student	100	100
Single day	175	200
Extra banquet ticket	65	65
Software Exchange, 3.5" disks	10	10

Payable by cheque in \$US on a US bank, money order in \$US, Visa, MasterCard, or American Express Card. Cheques and money orders should be made payable to: ACM/SIGAPL95 Conference.

Mail (including cheques/M.O.s) to:

APL95 Conference
c/o CM Planners
3700 Forums, Suite 201
Flower Mound, TX 75028 USA

Tel: +1 (214) 539 8298
Fax: +1 (214) 539 2850
e-mail: cmplan@unicomp.net



ACM SIGAPL is pleased to announce that the 1995 Kenneth E. Iverson Award for outstanding contribution to APL will be fully funded by an unconditional grant from Soliton Associates. The grant consists of US\$1,000 for the cash award to this year's recipient(s) plus up to \$500 for other expenses directly connected with the granting of the award. The SIGAPL Executive Committee thanks Clarke Bruce, Nancy Lamb, and all the employees of Soliton Associates for their support of SIGAPL and of our program for recognition of exceptional achievement by members of the APL community. Michael Kent, Secretary-Treasurer, ACM SIGAPL

Panels and Workshops:

Evolution of APL (90 mins)	Moderator: Kimmo Kekäläinen
Compiling array languages (90 mins)	Moderator: not finalised Panellists: Bob Bernecky, Wai-Mee Ching, others
Objects and APL (90 mins)	Moderator: Mike Kent Panellists: Bob Brown, David Liebttag, Bill Rutiser, John Scholes
APL Alternatives (3 hrs)	Moderator: Martin Neitzel Panellists: Bob Bernecky, John Feo, Richard Gaylord, John Howland, Clint Jeffery
Improving J (90 mins)	Moderator: Harvey Davies
ASCII transliteration	Moderator: Leroy J. Dickey

APL95 offers far more "professional development" opportunities to build directly and immediately applicable skill than ever before.

Workshops:

Business Graphics (3 hrs)	Leader: George MacLeod
Programming for Portability (90 mins)	Leader: Eke van Batenburg
Elementary Calculus (3 hrs)	Leaders: Walter Spunde, Richard Neidinger

Professional Development Seminars:

Programming in J (6 hrs)	Leader: Donald McIntyre
APL in Engineering (6 hrs)	Leader: Scott Kimbrough
Sharing and Associating (3 hrs)	Leader: Ray Polivka
Off the Couch, Onto the Highway (3 hrs)	Leader: Jim Brown
Nonlinear Constrained Optimization (6 hrs)	Leader: Bill Koko
Genetic Machine Learning (3 hrs)	Leader: Andreas Geyer-Schulz
Neural Networks in APL and J (3 hrs)	Leader: Ben Best
Concrete Math (3 hrs)	Leader: Ken Iverson
What to do 'til the Millenium Comes (3 hrs)	Leader: Mike Kent
APL at Both Ends (3 hrs)	Leader: Morten Kromberg
Making Nested Arrays Work for You (3 hrs)	Leader: Phil Benkard
Application Development in J (90 mins)	Leader: Chris Burke
Structured Programming in APL and J	Leaders: Bill Rutiser, Eric Iverson

APL 95 Abstracts

Accepted Papers

Is APL2 a Good Programming Language? Nicholas Beaumont. This paper evaluates APL2 in the context of systems development, noting that a language is not an end in itself, but one of several tools used to create an information system. Despite its many good features APL2, when evaluated in that context, has some disadvantages, especially the lack of control structures and data typing. These omissions make finding errors in programs and translating system requirements into programs harder than necessary. These and other disadvantages of APL2 (such as its lack of portability) make its relative unpopularity explicable. Some changes to APL2 are suggested.

The Role of Dynamic Programming & Control Structures in Performance. Robert Bernecky. Dynamic programming is used infrequently by APL programmers, in spite of its ability to reduce the computational effort required to solve many problems. This paper uses the *String Shuffle* problem as the basis for a comparison of brute force, recursive, and dynamic programming algorithms. Simple algorithms for each of these approaches are designed and evaluated, then individually optimized and re-evaluated, to show the benefits of dynamic programming.

The dynamic programming algorithm is then recast to use flow control structures recently introduced into ISI J and APL*PLUS III. Use of control structures in conjunction with dynamic programming results in orders of magnitude performance improvement over brute force and naive recursive algorithms. APL*PLUS III control structures are shown to provide a performance improvement of up to 30% over GOTO-based loops.

Recycling APL Code into Client/Server Applications. Richard J. Busman, Walter G. Fil and Andrei V. Kondrashev. This paper analyzes the current state of modern APL and deals with the authors' experiences in recycling and downsizing old APL applications and transforming them into more open information systems using client/server technology on different hardware and software platforms.

Infinity Arithmetic, Comparisons and J. Harvey Davies. The principles of infinity arithmetic are explained. There is a need to bring J arithmetic and comparisons into line with these principles. Many of the results produced by J are mathematically anomalous and do not comply with the IEEE 754 Standard for floating-point arithmetic.

Some proposals to generalise the concept of comparison tolerance are presented. There are also some tentative proposals regarding safety levels and precision.

The paper is oriented towards J but many of the concepts are relevant to all dialects of APL.

Deploying Complex Decision Technologies with APL to Solve the Daily Output Planning Problem: A Tale from Two of the Last of the Jedi Knights. Kenneth Fordyce, Gerald (Gary) Sullivan. A key decision faced each day, shift, or time period at a semiconductor manufacturing facility is how many wafers or lots should it commit to produce at each operation over a specified period of time to meet immediate demand and position the line to meet tomorrow's demand. This paper will describe an approach which integrates various decision technologies using APL to generate a "daily output plan" for low part number high volume wafer fabrication lines. A related approach for high part number logic lines using due dates is not described in this paper. The ability of the "APL Problem Solving Environment" to easily handle very different decision technologies was critical to the development of this approach.

Holland Classifier Systems. Andreas Geyer-Schulz. A Holland classifier system is an adaptive, general purpose machine learning system which is designed to operate in noisy environments with infrequent and often incomplete feedback. Examples of such environments are financial markets, stock management systems, or chemical processes. In financial markets, a Holland classifier system would develop trading strategies, in a stock management system order heuristics, and in a chemical plant it would perform process control. In this paper we describe a Holland classifier system and present the implementation of its components, namely the production system, the bucket brigade algorithm, the genetic algorithm, and the cover detector, cover effector and triggered chaining operator. Finally, we illustrate the working of a Holland classifier system by learning to find a path with a high payoff in a simple finite state world.

Bounded rationality and adaptive agents in economic modeling. T. Grimm, J. Mittöchner, W. Schönfeldinger. Traditional economic theory sees human economic decisions as rational choices of action to achieve maximum utility. However, economic reality shows that people often behave in ways different from that ideal: actions are not always determined rationally; often they are influenced by other factors, such as chance and tradition. In order to describe the behaviour of economic agents in the real world, we take irrationality and adaptation into account: we present work in progress on a market simulation that shows the effects of partly irrational and adaptive consumer buying decisions on the course of action of entrepreneurs.

Translation of Nested APL to C++: Problems and Approaches. Dmitri Gusev, Igor Pospelov. This paper discusses the task of developing a nested APL compiler and puts forward the project of such a compiler. The proposed approach consists of simulating the semantics of APL by using the object-oriented C++ programming language. Different kinds of problems

ranging from APL syntax ambiguity to the implementation of nested arrays and the robustness of compiled code are considered and different solutions are proposed. We conclude that it is feasible to develop a robust nested APL compiler, although the problem of performance demands further research.

Modelling the Effects of Acid Leaching on Concrete. Kevin Hartzfeld. This paper describes a partial differential equation model used to determine the service life of buried concrete vaults exposed to acidic ground water. A graphics package helped immeasurably in debugging and testing the algorithm. The collocation program developed in APL can easily be applied to solve other problems.

The Philosophy of Nothing and Everything. Neville Holmes, Colin Hallam. Perhaps inspired by the IEEE floating point arithmetic standard's introduction of a representation for aberrant numbers, such as the reciprocal of zero, the J interpreter overtly deals with infinities and indeterminacies.

This step got rid of some domain error messages, but brought in a few new difficulties, and exposed inconsistencies in the behaviour of elementary arithmetic functions presumably as delivered by the C compilers used for the J interpreters.

This paper proposes notation that would allow the J interpreter better to deal with failures of arithmetic. The discussion should assist development of arithmetic for other interpreters and compilers.

Rank and Uniformity. Roger K.W. Hui. Rank is elucidated through an executable model. Integrated rank support for specific functions can be vastly more efficient than the general algorithm in computing $v''r$, and uniform verbs provide a way to extend the more efficient implementation to a large class of verbs.

Representations of Recursion. Roger K.W. Hui, Kenneth E. Iverson. Recursive definitions are important tools in mathematics and in programming. They can be represented in a variety of ways, and in this paper we examine the relations and translations between them, using a range of examples for illustration. These examples begin with recursive definitions expressed in conventional mathematical notation, in SHARP APL, and in APL2; most are drawn from published sources.

APL, Dynamic Programming, and the Optimal Control of Electromagnetic Brake Retarders. Scott Kimbrough. The basis of Dynamic Programming is described. It is a method for finding the best way to control a system when that system can be controlled by its inputs. As an example, the Optimal (or best) Control of an electromagnetic brake retarder is derived. In this example the current to the brake retarder is the input being controlled. APL is shown to be a convenient language for writing a Dynamic Programming algorithm. A set of APL subroutines is provided for solving two-dimensional problems with one input. Adopt APL programmers may enjoy the challenge of generalizing this code to problems of arbitrary dimensions.

Regression Analysis with Fuzzy Knowledge Representation. Inna N. Louneva, Sergei M. Obraztsov, Alexander L. Shimkevich. An algorithm for fuzzy non-linear estimation and an APL implementation of this algorithm are discussed. The algorithm is based on using probability distributions for experimental data. The parameters of the distributions are given as linguistic variables by an expert.

The Role of Composition in Computer Programming. Donald B. McIntyre. Iverson has greatly enlarged the mathematical notion of function composition and made it available to computer programmers. This paper explains the concept, and uses practical examples to show how concise, powerful programs can be written and read. Alternative forms are given, and ways of overcoming initial difficulties are explained.

The systematic use of composition extends the well-known advantages of APL: namely conciseness, consistency, and generality.

Computing Multivariable Taylor Series to Arbitrary Order. Richard D. Neidinger. Automatic differentiation, manipulating numerical vectors of coefficients, is the efficient way to compute multivariable Taylor series. This does not require symbolic differentiation or numerical approximation but uses exact formulas applied to numerical arrays. Arrays of Taylor series coefficients of any elementary function can be built-up, as the array for each component (combination or function) is a combination of its argument arrays. The functions TIMES and EXP display the algorithmic ideas that enable all of the other standard functions. We study the interesting recursive formulas for these combinations, the resulting algorithms, and the implementation in APL. To handle all coefficients in n variables up to order m , the arrays are hyper-pyramid data structures, considered conceptually as n -dimensional but implemented as one-dimensional arrays. Unlike previous work, this implementation does not require huge arrays for binomial coefficients and indirect referencing. This APL*PLUS III implementation loops through one nested reference array and takes sub-arrays from another for a practical solution to this problem that can make tremendous demands on time and space.

Untying the Gordian Knot: Agreement in J. Martin Neitzel. The "Dictionary of J" is not written for the faint of heart. Its numerous self-references make it very difficult to tackle. The author thinks that section II.B, "Verbs", is one of the keys for getting a confident grip of the language. This paper basically just repeats this section using a tutorial style, some historical background, and several pictures. Hopefully, it will enable people with analytical minds to find their own way through the Dictionary. It closes with a discussion of possible language changes in the context of II.B.

AVCS: The APL Version Control System. Nikolai I. Puntikov, Maxim A. Volodin, Alexei A. Kolesnikov. This paper describes AVCS, which is an APL-oriented version control system devised as a tool to track the history of software projects and to control concurrent access to project components. The basics of version

control systems are explained, and specific aspects of applying a version control system methodology to project development in APL environments are considered. Particular attention is given to features which differentiate the approach accepted in AVCS from that of available version control and project management systems. Specification of AVCS's data maintenance, programming and user interface is presented to the extent required in order to explain how the system works. In conclusion, the possible application of AVCS to solving problems of porting APL projects across different environments is outlined.

Infix, Cut and Finite Automata. Clifford A. Reiter. The behaviour of one and two dimensional automata are displayed in two and three dimensions and via animations. Implementations of finite automata in J using "infix" and "cut" to distribute local definitions of finite automata are compared. Abstract automata are considered along with applications to image processing and surface plotting.

Early Bankruptcy Detection Using Neural Networks. Gottfried Rudorfer. In 1993, Austria had the highest number of bankruptcies since 1945. The total liabilities came to approximately US\$3 billion.

Powerful tools for early detection of company risks are very important to avoid high economic losses. Artificial neural networks (ANN) are suitable for many tasks in pattern recognition and machine learning. In this paper we present an ANN for early detection of company failures using balance sheet ratios. The network has been successfully tested with real data of Austrian private limited companies. The research activities included the design of an APL application with a graphical user interface to find out the relevant input data and tune the ANN.

The developed APL workspace takes advantage of modern windowing features running on IBM-compatible computers.

Fuzzy Control of Technological Processes in APL2. Alexander O. Skomorokhov, K.-H. Reinhardt, G. Roche, M. Tielemann. A fuzzy control system has been developed to solve problems which are difficult or impossible to control with a proportional integral differential approach. According to system constraints, the fuzzy controller changes the importance of the rules and offers suitable variable values. The fuzzy controller tested consists of simulator code to simulate the process dynamics of a production and distribution system and the fuzzy controller itself. The results of our tests confirm that this approach successfully reflects the experience gained from skilled manual operations. The simulation and control software was developed in APL2/2 running under OS/2. The benefits of this product, especially multi-tasking and the ability to run AP124 and AP207 windows concurrently, are also discussed.

Concrete multidimensional statistics in APL2. Bernhard Strohmeler. This paper deals with problems which occur in discrete multi-dimensional statistics and shows how they are solved using APL2. Especially the APL2 feature *primitive function with axis* proves to be very helpful and allows smart solutions to the problems of computing conditional probability tables and performing the iterative proportional fitting procedure. The implementation is done using the *iterate* APL2-style.

Namespaces: APL/W vs. APL2. Rexford H. Swain. This paper describes and contrasts the implementation of namespaces in two popular combinations of APL dialects and platforms: IBM's APL2 (version 2 release 2) running under VM/CMS, and Dyadic's Dyalog APL/W (version 7.0) running under Microsoft Windows.

In a traditional APL workspace, localization is the only mechanism available to isolate identifier names and values, and it is extremely potent. While a function that localizes a given name is executing, it is impossible to reference any other definition of the same name.

Namespaces are a significant addition to an APL system. They provide a way to "package" a collection of objects (variables, defined functions, etc.), and insulate them from the traditional workspace. A namespace co-exists with the workspace, yet its object names and definitions are completely independent of the workspace. Namespaces can help to organize and hide complexity in workspaces, avoid name conflicts, and share code among applications.

IBM has offered namespaces in APL2 since 1987, whereas Dyadic just recently introduced their facility. Both implementations provide encapsulation and name isolation within a workspace. But they use quite different methods to access objects in a namespace and store their initial definitions. The merits of each approach are discussed. An example is provided to illustrate how namespaces may be used to simplify the implementation and improve the functionality of workspace and function analysis tools.

Doing ANOVA Calculations on the Computer. Alan M. Sykes. This paper presents an approach to providing educational software for students in Management Science and Statistics which aids the calculations and understanding required for the analysis of experimental design data.

An APL Implementation of the Club of Rome's World Model. Clare Townsend. This paper discusses an application which models world population dynamics using initial value differential equations. The program provides the user with a simple interface that anyone with any level of experience with APL can use to generate different scenarios. In addition, the discoveries and difficulties encountered by a student using APL for the first time are explored.

Invited Talks

APL is Important at USAA. Rhonda K. Aikens. United Services Automobile Association (USAA), is a leading provider of insurance and financial services. Automobile insurance represents the highest percentage of USAA's multi-billion dollar operations. Rate-making applications, written in APL, constitute the core set of tools used by the Automobile Actuary Pricing Unit. This presentation provides an overview of the APL rate-making tools, how they interact, and why they are crucial in establishing accurate, equitable, and competitive rates.

A New Approach for Developing Applications using Asynchronous AP144 and Synchronous AP207.

Y. Akiyama, T. Itoh. This paper describes an approach to designing and implementing an application that uses asynchronously executed auxiliary processors such as AP144 (or AP145) and synchronously executed auxiliary processors such as AP207, for the user interface in the AIX Motif (or OS/2 PM) style.

AP207 offers very good and useful graphical functions. The window API (AP144 or AP145) commands emphasize client/server asynchronous execution of the API commands to display and update the screen, while the AP207 API commands are executed synchronously.

A difficulty comes in designing an application system when we want to implement an X Windows System or Presentation Manager (PM) style single-user interface using both types of auxiliary processor APIs.

The APL95 Software Exchange

from Dick Holt

I want three things from you:

- I want new or recently updated material for the APL95 Software Exchange. Any flavor of APL, J, or related software, in English or other languages, is welcome. Send your disk, by 5 May 1995, to 3802 N. Richmond St. Arlington VA 22207. Include a brief ASCII file describing what the software does.
- What we do: I want your suggestions about how SIGAPL can improve the quality, timeliness, and relevance of the Software Exchange. We've never asked APLers what they want or need. We dump 20 megs of stuff on the table once a year and a few people pick it up.
- How we do it: I want your suggestions about how SIGAPL can use this asset better. Here's the problem: once a year we collect a huge amount of software (APL93 was 9 disks). It's readily available one week per year to conference attendees (they constitute 2% of the APL community), and not so readily available to the other 98% for the rest of the year. It's like a python eating a pig -- one huge gulp followed by a long sleep. Electronic distribution helps, but the BBS\APL and FTP sites can't always get Software Exchange material easily or on time.

Help us out. Send your suggestions to Dick Holt at the above mail address or to sigapl_bd@acm.org.

News from Sustaining Members

Compiled by Gill Smith

Causeway Graphical Systems Ltd

Adrian Smith and Duncan Pearson have set up Causeway to enable them to work full time on Causeway application development. The company is based in Malton, North Yorkshire, and has two clear aims:

- to use Causeway (under Dyalog/W or APL*PLUS III as appropriate) to build high-quality decision-support tools with a true Windows 'look and feel'.
- to offer a professional level of support and training for the Causeway platform to APL sites needing an effective, reliable and future-proof environment for in-house development under Windows.

Over the first two or three years, we shall spend at least 60% of our time developing configuration and decision-support tools for the German software company SAP. The first of these will be included with release 3.0 of their R/3 product later this year. At APL95 we will provide Causeway for APL*PLUS III, and also for Dyalog APL/W version 7.1 (fully exploiting namespaces and other 7.1 features).

The Causeway architecture will remain in the public domain, and the new version of Causeway for Dyalog/W will be bundled free of charge with the interpreter.

MicroAPL Limited

Our last major upgrade to the Macintosh versions of APL.68000 Level II was released last summer when we announced support for the new Power Macintosh computers and also added a significant number of new features to the product.

We are now testing a further upgrade to our Macintosh versions of APL.68000 Level II and customers will shortly be receiving full details of the upgrade. This upgrade combines some features found in our RISC/System 6000 version of APL.68000 together with some additional enhancements to the `⌈WI` system function.

We have added the family of native file systems functions that are already available on our APL.68000/X product for the RISC/System 6000. These functions include `⌈NCREATE`, `⌈NERASE`, `⌈NLOCK`, `⌈NNAMES`, `⌈NNUMS`, `⌈NREAD`, `⌈NREPLACE`, `⌈NTIE`, `⌈NUNTIE`, `⌈NWRITE`.

Our APL component files on the Mac are now automatically multi-user, with users able to change their user ID via a preference dialog. We have also allowed users to specify the name for the APL component file and HELP file.

The $\square WI$ interface has been enhanced in a number of ways - we have added a new read-only property - class (the class of an object). The delete method is tolerated for non-existent objects (for better APL*Plus compatibility and ease of programming). A new Printer object allows control of the printer using $\square WI$. This include 'methods' for printing text, putting up a page dialog and a job dialog, plus properties for font, page size etc. We have added multi-select list boxes and multi-column list boxes. The new 'System' object will allow $\square WI$ access information such as screen size.

Finally we have added official support for Auxiliary Processors written in C. This last feature has been available for some time on an 'as required' basis but is now included with the interpreter.

In addition, all bugs reported to us have now been fixed.

Manugistics Inc

One of the comments that Manugistics hears a lot from its customers is "How can we promote the use of APL more?" It seems that while some of us have found the value in this language, many others are still using less powerful tools. In talking with many of the APL special interest groups, we believe that with the support of these groups and a good shareware version of APL we could all spread the word to more people. In order to support this effort, Manugistics plans on making available a shareware version of its APL*PLUS PC product. We intend to begin the distribution of this product through the APL special interest groups. We feel that with the support of a local group of APLers, new APLers can be found and trained.

This shareware version is a fully functional APL*PLUS interpreter with no artificial limitations. It will run under DOS, Windows or OS/2 in the 640K of RAM that DOS provides. There are 6 chapters of basic APL language documentation included online; this documentation was written specifically to address the needs of brand new users.

If you are interested in helping us to make this available to the community, please stop by the Manugistics booth at APL95 in San Antonio, TX or contact us directly at +1-301-984-5468. We hope to see you at APL95!

Insight Systems

By the time this is in print, we expect to be shipping the first copies of version 2.0 of our SQAPL Server for Unix. Version 2.0 will be based on the new SequeLink "Link Engine", which supports ODBC drivers under Microsoft Windows and on the Macintosh. By the middle of 1995, version 2.0 will allow you to turn APL databases or applications implemented using almost any APL interpreter running under Unix, Windows NT or OS/2 into "black boxes", which will appear to other applications as industry-standard ODBC data sources, as if they were Oracle database. Instead of moving APL data into a relational structure, losing flexibility and performance, we can now make data in APL files available to other applications *on their terms*. We believe that this opens up a lot of new opportunities for APL-based applications, which can be presented as open, "mainstream" products.

For "pure" APL-to-APL distributed applications, we are introducing a new product, which is already being tested by a few customers under the project name "APL Pipes". The APL Pipes product does not use SequeLink for transport, but is based directly upon TCP/IP "sockets". As a result, it is available not only under the server operating systems supported by SequeLink, but also under plain Microsoft Windows. APL Pipes allows a single APL task or workspace under Windows, Windows NT, OS/2 or Unix (if you need VM or MVS support, please call) to support several clients simultaneously. APL Pipes supports transfer of APL objects of any type, rank, shape (and almost any size) between clients and servers which may be based on different APL interpreters. For example, you could use the product to call an APL*PLUS server to read (or write) component files from a Dyalog APL client application (and vice versa). For developers wishing to use C or other compiled languages, we will also supply a library which allows APL servers to be called from such environments, effectively making any APL interpreter available as a C library (or DLL under Windows).

Dyadic Systems Limited

... please turn to page 158

THE EDUCATION VECTOR

April 1995

Editor Ian Clark

This Education Vector has been reprinted from VECTOR Vol.11 No.4. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Road, Redland, BRISTOL, BS6 6LS. Tel: 0117-973 0036.

Contents

Editorial	Ian Clark	18
Jot-Dot-Min		21
J-ottings 5	Norman Thomson	27
Fractnomials?	J R Sutton	30

Ian Clark
IAC/Human Interfaces,
9, Hill End, Frosterley,
Bishop Auckland,
Co. Durham DL13 2SX.

Tel: 01388-527190
Email: 100021.3073@compuserve.com

Editorial

This Nonsense Has Got To Stop!

Hands up who never heard that from their old headmaster. A young acquaintance of mine recently described morning assembly as "two hymns and a telling-off", so things haven't changed much since my schooldays.

So it's a novelty to hear the good old message in the mouths of school governors, but this time directed at central government. Now school governors, like local authorities (who are inclined to forget it) are creatures of Parliament, with no constitutionally independent right to exist. It's not as if the UK is a federation of counties — it isn't. Like Rutland, they can be made to disappear and reappear at will. If school governors embark on a campaign of mass disobedience (a distinct likelihood at the time of writing), the government might understandably feel aggrieved at having its hand bitten by the ones it has nurtured as a power independent of the local education authority.

But it's one thing to be a creature of Parliament, and quite another thing to have a school to run. Alarm and despondency is the inevitable result of having inadequate resources (and nobody has actually attempted to defend their adequacy) further squeezed by a national teachers' pay settlement which central government refuses to fund. School governors must be feeling that their localised power is nothing more than the power to take responsibility for a crisis not of their making. They must be wondering whether their sole reason-to-be, indeed the sole reason-to-be of any institution set up in the last twenty years by the Masters of the UK (till God wills) has been — not the better achievement of their nominated goals, but the fragmentation of non-ministerial power for no greater purpose than to sap the power of local authorities, indeed of any public body capable of effectively opposing the ministerial will. Meanwhile the ministry tells all-and-sundry that there's plenty of dead wood to chop in local government administration before one needs to go sacking teachers.

Now let us view this lake of blood from the vantage point of the very people this government has claimed to champion — the little guys of industry. Let us further restrict the discussion to the industry of producing educational materials for schools. Of all the victims of this sorry state-of-affairs, maybe they will be the least lamented, but like the canary down the coalpit their demise may amount to more than one little bundle of feathers. It might point to something unfriendly in the atmosphere. It's no part of this publication to pamphleteer on the wider aspects of social engineering, but to encourage diversity in educational

computing, for much the same reasons as we'd all like to encourage biodiversity in the environment. Monocultures mean mega-species mean mega-diseases.

To begin with, the innovation of letting schools manage their own budgets seemed a godsend to suppliers. The bigger they were, the more of a godsend it seemed, since promotion of their goods was henceforth going to rely on marketing muscle rather than on pleasing a small number of discriminating gatekeepers. Hitherto, like the NHS for drugs, there was effectively a single customer — the network of LEA centres which did a good job of buying new computer products for schools, encouraging the exploration of these products by working teachers and publishing catalogues in which the little product got as much space as the heavyweights. Then all of a sudden a vast amount of money was put into the hands of schools governing bodies, money which we are invited to imagine was being squandered on layers of administrators in County Hall, or perhaps on painting buildings that were already the right colour, mending roofs which it would have been cheaper to put a bucket under, or paying senior staff over the odds, most of whose work could be done by a cheaper recruit from today's flexible labour market. New suppliers flocked to the marketplace, big as well as small.

A stroll around the educational shows of the last two years showed a huge investment in promoting basically pedestrian products. Pedagogically pedestrian, but graphically gargantuan — but at the end of the day glitzy graphics are only part of the packaging. We all ought to play more poker, because then we'd understand what is meant by "upping the ante" — making it more expensive for new players to buy into the game. Now, with this throttling squeeze on school budgets we shall see educational products (whether books or software) going the way of cornflakes. More and more attractive packaging and less and less nourishment in the contents, because nourishment comes from a varied diet. Ultimately the innovatory suppliers will die out and a few big players will spend huge sums on competing with each other for a shrinking school budget with a dwindling library of rapidly obsolescent multimedia products. On the home front, Sega and Nintendo will place fantastically powerful hardware in the hands of our youngsters, capable of delivering all the educational principles past ages have had no end of trouble imparting. All to be used for — what? Making the blood and guts fly on even more convincing trajectories? And still mathematics teachers appeal to me for a good straightforward product to teach pupils how to balance brackets and multiply out expressions.

It's not as if the money isn't there. Why do school supplies have to compete with a proper remuneration for the practitioners? Our public utilities have shown

how it is possible to keep gas, water and electricity flowing into our homes, yet still be flush with profits and able to pay their top men what they're worth. How about paying teachers what they're worth? Is it beyond the wit of our lords and masters at the same time to keep the educational supplies flowing into our schools? Although we'd feel the pinch sooner, it would do us, the people of Britain, far less harm in the long run to have our gas, water and electricity turned off for a week than to screw down the supply of educational resources to our children. Yet the former could pay for the latter many times over.

This nonsense has got to stop. Otherwise creative British sources of educational materials will dry up and we'll be left with wall-to-wall "Arthur's Teacher Trouble".

Errata

We apologise for the following errors: In "*More about Fractional Series*" by John Sutton (Education VECTOR, October 1994, pp30-31), on the right hand side of equation 1 the 'n' should be in upper case. Mr Sutton's address should also have been shown as "16A Overland Road, Mumbles, Swansea, SA3 4LP". In "*The Common Mean and APL*" (Jan 95 pp21-23) the right-hand side of the 8th equation should read $(x_i + y_i)/2$, and at the bottom of page 22 it should read $1/\sqrt{2}$.

How to Get Hold of J

Vector, with its offspring Education Vector, has endeavoured to promote the J language with ample coverage in this and past issues, since we feel it is an important development of APL. However, some educational readers might not be aware that they can get hold of the language free of charge:

J Release 2 is now a fully-fledged product and can be obtained running on a number of platforms (IBM PC Windows and DOS) from I-APL Ltd., 56, The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU. It costs £55 for this "Personal Edition", plus postage, quite a reasonable price for the professional development system it has become.

However J Version 7 is still available as shareware, and we recommend this to educational newcomers to the language. Disks can be obtained from: I-APL Ltd. for £5. Both PC and Macintosh versions are available. However, for copyright reasons, manuals are no longer available for J Version 7. Instead we recommend the articles in this and back issues of Education Vector to give an introduction to the language. "*Programming in J*" by K.E. Iverson is still available from I-APL Ltd. for £10.00 plus postage and packing.

Jot-Dot-Min

an outer-product of rock-bottom APL matters

Robomaniacs of the Seventies used to delight in computerising out the Man-in-the-Loop (meaning the Wiener feedback loop) as the most unreliable component. This led to a proliferation of soul-destroying jobs where the Man was only there because the programmer couldn't figure out how to do without him. Many weapons systems had a man left in them solely for the purpose of having someone to court-martial if it went off by mistake. The celebrated cybernetician Stafford Beer uttered a warning that the progress of computer systems, far from increasing wealth, would rob vast numbers of the possibility of gainful employment. Perhaps even a soul-destroying job is better than no job at all.

Then the Human-Centred movement arose and turned the computing profession on its head. Far from being a weak link in a feedback loop, a sort of liveware fuse, the human in the system (the campaign for inclusive language was underway too) was its only reason for existence. Indeed all computers were only person-to-person information pumps and a computer system nothing but a communications medium or a personal amplifier. The US Federal Communications Commission (FCC) permitted Ma Bell to compete with Big Blue, thus wiping out the legally enforced distinction between data processing and electronic communication, ARPA fostered what became the Internet, and Apple promoted truly interactive computers for the rest-of-us.

What happened? A generation of users grew up oblivious of job control languages, except in the twilight world of DOS batch language (the language never even got a proper name and was deliberately crippled by a lack of numerical variables or any sort of arithmetic). The soul-destroying job re-emerged, as personal computer users struggled to earn their crust by endlessly repeating the same few menu selections and keystrokes again and again. People rediscovered the first principle of computer use: if you ever succeed in doing anything useful with a computer, you're going to want to do it again and again, but with variations here and there, which you hope you can anticipate. Isn't that the whole of programming?

After trying to bolt-on interactive event recorders like MacroMaker in a vain attempt to record and replay useful sequences, Apple has just reinvented JCL (Job Control language) in the form of AppleScript, and appeals for all new applications to be made 'scriptable', i.e. programmable by a series of lines of text in a 'script' (read: batch file) as well as executable by screen interactions.

Windows, the great me-too, will go the same way in due course. Ah, but scripts can be generated by user interaction, they say. Yes, but nobody writes them that way. They're all writing programs again. And in a language that isn't even as advanced as BASIC.

In spite of Adrian Smith's article on control structures in VECTOR 11.1, which pondered the ins and outs of loops very nicely, perhaps it's time for a few musings on the subject of Branch (→). Since we're all beginners together, we won't ask whether you should be using Branch at all, but consider some of the funny things people do with it. And one of these funny things is to write loops.

Fifty years of program language development (far, far longer if you go back to Babbage and Countess Lovelace) and humans are still writing loops! It's like the Buddhist Wheel of Becoming. Santayana — he say: one who forgets the past is condemned to relive it.

One of the joys of APL programming is to replace loops with bit-strings, which you put together by Boolean operations. It's just like rubber-stamping a pattern instead of trying to knit it. Novices who don't grasp that, erstwhile Fortranners and Cobollers who make straight for the right-arrow and sigh — ah! intelligent life! — are stuck in a Santayanan loop. Here's how you write a loop if you're a C programmer:

```
for(i=0; i<10; i++) {  
    DoSomethingWith(i);  
}
```

and you write it again and again until you can write it in your sleep (and generally do). Because if there's one thing that C programs do to the exclusion of nearly all else, it's step through a stream of characters, 'filtering' them. C arose out of the efforts to implement UNIX, and UNIX reduces the whole world to streams of characters which get piped through filters. So C is a language with a lot of plumbing in it.

It loops ten times, of course, stepping i from 0 to 9. The widget: `for( ; ; )` has three slots separated by semicolons, these say what to do at the start of the first loop, the condition for another loop, and what to do at the end of each loop. If you simply must copy this loop in APL, then APL lets you (because it's very forgiving), and the right-arrow looks comforting. "Here, I'll show you the way", it seems to say. And the way is? — back where you've come from. If you're lucky.

Here's the for-loop of C done into APL:

```

i←0
OPENBRACE:+(10<i←i+1)/CLOSEBRACE
DoSomethingWith i
→OPENBRACE
CLOSEBRACE:

```

Looks uglier than it does in C, doesn't it? In both cases, the code inside the loop can be imagined to do something like: pass forward the i 'th character if it's not blank. That's how you filter out blanks the C way.

Instead of knitting the output, character by character, the APL way is to construct a bit-string with one bit for each character in the input string (let's call it S), a 0 bit if it's blank, a 1 bit otherwise. The 0 bits mark the characters we want to filter out.

$S \neq ' '$

Put it into B like this: $B \leftarrow (S \neq ' ')$

Then the following is S with all the blanks squeezed out: B/S

You can put it all together in one expression: $SS \leftarrow (S \neq ' ') / S$

which avoids introducing B as a receptacle for the bit-string, using it immediately it is formed.

Yes, but how far can you stretch that principle? You can do some pretty fancy tests inside a loop, to decide whether you want to pass forward the character or not. Surely the bit-string approach only works for simple things?

No. The only limit to what you can do with bit-strings is your imagination (or your command of Boolean algebra, but you need that in C too). Suppose we want to throw out, not just blanks, but periods too.

B marks the blanks with zero: $B \leftarrow (S \neq ' ')$

C marks the periods with zero: $C \leftarrow (S \neq ' .')$

so $B \vee C$ (read: B or C) is the bit-string we need. Or putting it all together as before: $SS \leftarrow ((S \neq ' ') \vee (S \neq ' .')) / S$

In practice you'd use $(\sim S \in ' .')$ instead of $(S \neq ' ') \vee (S \neq ' .')$ and you could put further characters you might want to expel between the quotes. But it serves to illustrate the principle. And if you'd like to throw out periods only if they're followed by blanks, then just shift B to the left and combine it with C :

$(1 \div B, 1) \vee C$ which flags only those periods for which there is a blank to the right of it. All together, now:

$$SS \leftarrow ((1 \div (S \neq ' '), 1) \vee (S \neq ' ')) / S$$

You can do all your filtering this way. It would make a nice car-sticker:

"APL-ers Do It With Bit-Strings".

To quote David Diamond:

*...alas for me:
My programs map to binary.
No other reason need be sought
Why half my work comes out to naught.*

As a beginner I used to feel edgy about composing the whole history, in advance, of what I expected a loop to do. Wasn't it inefficient? After all, a loop only creates the truth values it needs for the current cycle. However, APL bit-strings are efficient. Very efficient. A typical C compiler will reserve a whole integer (2 bytes, or 16 bits) for a Boolean value, even in an array, but APL literally stores bits in bits. So a 16-bit string would only occupy 2 bytes for its actual data. Of course there's the workspace overhead for a variable, or even a transient value, but this is the same for a long bit-string as a short one. Furthermore, when you combine two bit-strings as in $B \vee C$, the 'or' operation isn't done a bit at a time but a word at a time. In 32-bit computers, that would combine 32 bits in a single cycle, the net result of 32 loops the C way.

The more advanced the computer, the more you can expect APL to start competing with C in filtering tasks. APL is about to come into its own with the advent of parallel computing.

Meanwhile there are occasions when you simply must have a loop. The commonest is the outer loop or duty cycle in a program which waits for user input, then acts on it. What in BASIC they used to call the 'input-execute-output' cycle. Instead of the C-faithful example above, you're better off using a computed branch back to the start of the cycle.

```
LOOP:
  DoSomething
  →LOOP IF carryOn
```

Computed branches are the mischief. There's a hundred and one ways they can go wrong, and what every program needs like a hole in the head is a branch to

goodness knows where. For that reason alone, the right-arrow should be handled like high explosive.

In this example, *carryOn* is supposed to be a scalar Boolean (a 0 or a 1), or a function (or expression) returning a scalar Boolean. Notice how we've used a dyadic function *IF* to make the branch more readable and provide some reassurance it isn't going to blow up and go off on a wild branch. In I-APL you can define *IF* in either of two ways:

```

      ∇ Z←A IF B
[1]  Z←B/A
      ∇

```

or: *IF*: ω/α

The use of *IF* is effectively the same as the commoner: $\rightarrow(\text{carryOn})/LOOP$ because in the expression: *LOOP IF carryOn α* takes the value: *LOOP* and ω takes the value: *carryOn* when *IF* starts to execute.

How does it work anyway? Well, $1/xxx$ is equivalent to *xxx* whatever *xxx* happens to be, char or num, vector or scalar. On the other hand, $0/xxx$ is always the empty vector. And right-arrow followed by the empty vector is a branch to nowhere — it does just nothing.

There's more ways of collapsing a label to the empty vector than applying $//$ to it. You might come across: $\rightarrow(\text{carryOn})\uparrow LOOP$ which has much the same effect. You can also use a down-arrow to mean "branch if *carryOn* is false", i.e. $\rightarrow(\text{carryOn})\downarrow LOOP$ behaves like: $\rightarrow(\sim\text{carryOn})\uparrow LOOP$.

One of the dreams of programming is that whatever you write works first time. But take a loop like the first example we gave. Does it loop 9 times or 10? It could be a matter of life or death.

I heard about a US Army conscript who was put to work on programming. He inserted a line of code in a monthly run to print an analysis of his sergeant's parentage the day after his demob date. Unfortunately he got his $\leq$ mixed up with his $<$ and spent his last month in uniform behind bars. The moral is: if you want something to branch reliably, always use tried and tested code.

The following never lets me down:

```
10 repeatsOf 'HALLO'      where:
    ∇n repeatsOf s
[1]  ±s
[2]  →×n←n-1
    ∇
```

but that's because I never try to write down *repeatsOf* from memory, but always copy it from a workspace of my pet utilities. That's the whole idea of reusable code. If it's wrong, somebody'd know by now. Eat your heart out, Formal Methodology!

Speed Test: how does this work? Quickly now, quick-quick-quick...

```
∇LoopyLoop
[1]  →carryOn,DoingSomething
    ∇
```

That's right. The Boolean expression *carryOn* is supposed to return a 1 or a 0. *DoingSomething* must return a numeric result, or else you have to split *LoopyLoop* into two lines. The catenation is a red herring, since Branch is only going to look at *carryOn*. So we have a Branch either to [1] (start over) or to [0] (quit). And now you know how *repeatsOf* works too.

That's all just aversion therapy to make you love bitmaps. But using bitmaps for filtering really becomes interesting at the classroom level when you apply them to 2-dimensional arrays. How do you flag the neighbours of a random assortment of elements in a 2D array? How can you use the result to build a population model where individuals infect their neighbours at random with given probability? How can you spot a Dover sole underwater? (Both the sandy bottom and the sole are random bit-arrays.) All this in the next issue, plus the rudest (uncommented) APL program ever published — Conway's Game of Life as a single-liner!

Of course, if anyone wants to submit solutions to these problems to the Editor, EV, in time for the next issue, feel free! I shall publish acknowledgements of the best solutions along with my own. Or instead of my own. And remember this — *Binary Data is Fun Data*, but for real laughs use a Branch!

J-ottings 5

by Norman Thomson

The way in which data is structured constitutes a major difference between APL and J. At one time these differences were referred to as "floating" and "grounded" systems of nested arrays. Performing the activities of joining objects in the two languages is one of the best ways of highlighting these differences.

For definitiveness, suppose that a general instruction is given by a client to "join" two objects, one of which is to be represented by a vector and the other by a matrix. In J, there are eighteen potentially different ways in which a vector-matrix join can be interpreted, which doubles if the order vector/matrix is reversed. In other words there are no fewer than 35 ways in which the original loose specification can be misinterpreted!

To go back a step, one of the fundamental ways in which APL thinking must be adjusted for J concerns the manner in which arrays are perceived. In APL, an array is an *arrangement* of $x / p p a$ scalar items, whereas in J it is a *sequence* of items each of rank one less than the rank of a . The individual dimensions of an APL array are of equal importance in the sense that they can all be used as axis qualifiers. The dimensions of a J array however possess an inherent order of importance. Specifically, an APL matrix is a two-way-ordered collection of scalar items, a J matrix is a sequence of rows, a three-dimensional J array is a sequence of matrices and so on. The scalar items in an array are called 0-cells. k -cells ($k > 0$) are counted from the inside out, so for a three-dimensional array, 1-cells are rows and 2-cells are matrices, whereas if $k < 0$, (-1)-cells are matrices and (-2)-cells are rows.

These distinctions explain why J has two separate joining operations, namely append items ($,$), and append ($,.$). The essence of "append items" is that it raises the objects to be joined to a minimum common rank, if necessary using itemize (monadic $,:$). Itemize involves adding shape value items of 1, an operation performed in APL by $, [1 0]$. For example if $v = .1.3$ and $m = .1.3\ 4$, v, m requires that v be made into a matrix of rank 1 3, whose single 1-cell is joined to the three 1-cells (rows) of m . If $a = .1.3\ 2\ 4$, v, a is obtained by first making v into a 1 1 3 array, then padding this to a shape of 1 2 4 which is then joined to the three 2-cells (matrices) of a , to give a result of shape 3 2 4.

"Append" on the other hand means join at the (-1)-cell level of the right argument, so with the values of the previous paragraph, $v, .m$ means join each

item of v to the corresponding row of m to obtain a matrix of shape 4×4 . Similarly $v, .a$ requires each item of v to be matched with the corresponding (-1)-cell item (that is matrix) of a to give a result of shape $3 \times 3 \times 4$.

J provides a third joining primitive, namely laminate $(, :)$. This works in a similar, but less restrictive, manner than laminate in APL. As with the latter, the effect on the shape vector is, in general, to add a new item to the shape vector of value 2.

Understanding how items can be joined in J thus amounts to appreciation of the three primitive operations, append items $(,)$, append $(, .)$, and laminate $(, :)$. The rank conjunction $(\wedge)$ provides separate possibilities for the left and right ranks up to the maximum ranks of the objects concerned. Suppose that the requirement referred to initially is to join a vector v and a matrix m in that order. There are six essentially different arguments for the rank conjunction, namely:

- 1 2 equivalent to omitting the rank conjunction altogether, which means apply maximum rank for both v and m . This is also equivalent to $_ _$ where $_$, standing for infinity, means "maximum rank";
- 1 or equivalently 1 1, meaning join the vector to the 1-cells, that is rows, of the matrix;
- 1 0 join the vector to the individual cells of the matrix;
- 0 2 join the individual cells of the vector to the entire matrix;
- 0 or equivalently 0 0, meaning join cell to cell. This is valid only if v is "row-compatible" with m , that is the number of items in v is equal to the number of rows in m ;
- 0 1 join the cells of v to the matching rows of m — this also requires row-compatibility.

To exhibit the results of all the various possibilities in execution would require an unjustifiably large amount of printed space. It is more useful and practicable to give a table of APL2-equivalent expressions, in which, for consistency with J , index origin 0 is assumed. As the table shows, *disclose* is used extensively to provide the padding with fill items which occurs naturally with J .

<i>rank</i>	<i>append items (,)</i>		<i>append (,.)</i>		<i>laminare (,:)</i>
<i>nil</i>	$\Rightarrow \langle v \rangle, \langle [1]m \rangle$	<i>A</i>	v, m	<i>R</i>	$\Rightarrow \langle \langle v \rangle \rangle \langle [1]m \rangle$
<i>1</i>	$\Rightarrow \langle \langle v \rangle \rangle, \langle [1]m \rangle$		$\Rightarrow [0]v, \langle [0]m \rangle$	<i>C</i>	$\Rightarrow \langle \langle \langle v \rangle \rangle \rangle, \langle [1]m \rangle$
<i>1 0</i>	$\Rightarrow \langle \langle v \rangle \rangle, [0]m$		$\Rightarrow \langle \langle v \rangle \rangle, [0.5]m$		$\Rightarrow \langle \langle v \rangle \rangle, [0.5]m$
<i>0 2</i>	$\Rightarrow v, [0]m$		$\Rightarrow v, m$		$\Rightarrow v, [0.5]m$
<i>0</i>	$\Rightarrow v, [0.5]m$	<i>R</i>	$\Rightarrow v, [0.5]m$	<i>R</i>	$\Rightarrow [10]v, [0.5]m$
<i>0 1</i>	v, m	<i>R</i>	$\Rightarrow v, [0.5]m$	<i>R</i>	$\Rightarrow v, [0.5]m$

Notes: *R* requires row compatibility

C requires column compatibility

A simplifies to $v, [0]m$ if APL row compatibility also present

It may be helpful to record in a matching 3 by 6 table the shape vectors which result from executing the above expressions for the case $v = .1.3$ and $m = .1.3.4$.

	<i>,</i>	<i>.,</i>	<i>,:</i>
<i>1 2</i>	4 4	3 5	2 3 4
<i>1</i>	3 7	n/a	3 2 4
<i>1 0</i>	3 4 4	3 4 3 2	3 4 2 3
<i>0 2</i>	3 4 4	3 3 5	3 2 3 4
<i>0</i>	3 4 2	3 4 2	3 4 2 1
<i>0 1</i>	3 5	3 4 2	3 2 4

For row-compatible v and m , the last two rows of the first table show that $v, "0 m$, $v, ".0 m$ and $v, ".0 1 m$ are identical. None of these pairs of expressions are equal in general for arguments of other ranks. Also "row compatibility" has to be generalised to "(-k)-cell compatibility" for objects of general rank where k is the smaller of the two object ranks. Thus to append a 3 by 4 matrix to a higher rank array (or vice versa) with rank conjunction 0, it is necessary that the shape vector of the array begins 3 4

In APL2, variations on join are provided by using join with axis qualifier, and enclose with and without axis qualifier. The next table turns things round, and shows the J equivalents of the various APL2 alternatives:

	<i>m not enclosed</i>		<i>m enclosed with axis</i>		<i>m enclosed</i>
<i>v not enclosed</i>	v, m	<i>C A0</i>	$\langle "0 v \rangle, \langle "1 :m$	<i>A0</i>	$\langle "0 v \rangle, \langle m$
	$:v, m$	<i>R A1</i>	$\langle "0 v \rangle, \langle "1 m$	<i>A1</i>	
<i>v enclosed</i>	$\langle \langle v \rangle \rangle, \langle "0 m$	<i>A0</i>	$\langle \langle v \rangle \rangle, \langle "1 :m$	<i>A0</i>	$\langle \langle v \rangle \rangle, \langle m$
	$: \langle \langle v \rangle \rangle, \langle "0 :m$	<i>A1</i>	$\langle \langle v \rangle \rangle, \langle "1 m$	<i>A1</i>	

Notes: R join along axis 0 _ requires APL row compatibility
 C join along axis 1 (default) _ requires APL column compatibility
 A0 enclose along axis 0
 A1 enclose along axis 1 (default)

Laminate is a less rich operation in APL2 than in J, since objects for APL lamination must be both rank and shape compatible. In the case of two matrices there are three possible laminations, for which the J equivalents are as follows:

$m0, [-0.5]m1$	$m0, [0.5]m1$	$m0, [1.5]m1$
$m0, :m1$	$m0, : "1 m1$	$m0, :&. :m1$

Although the above tables cover only a subset of the possible instances of join (namely those for vector left/matrix right argument), the principles involved apply equally to cases of other ranks, and in practice this way of working is preferable to using general formulae involving shapes and ranks of the arguments.

Fractnomials?

by J. R. Sutton

The polynomial

$$\sum_{n=0}^{N-1} c_n X^n$$

with coefficients

$$c_n = 1 \text{ for all } n$$

is the sum

$$\sum X^n = (X^N - 1)/(X - 1)$$

which was discussed previously (Clark, Education Vector 11.1 pp. 31-55) and Sutton, Education Vector 11.2 pp. 30-31) for both integral and fractional N . A polynomial of fractional degree should perhaps be called a *fractnomial*.

With the coefficients

$$c_n = \frac{(-)^n \Gamma(N)}{\Gamma(N-n)(n!)^2}$$

which satisfy

$$\frac{c_{n+1}}{c_n} = -\frac{(N-1-n)}{(n+1)^2} \quad (1)$$

with $c_0 = 1$, the polynomials are the Laguerre polynomials $L_N(X)$ which satisfy the recurrence relation:

$$L_{N+1}(X) = \frac{2N+1-X}{N+1} L_N(X) - \frac{N}{N+1} L_{N-1}(X)$$

and the differential equation:

$$XL_N''(X) - (X-1)L_N'(X) + NL_N(X) = 0$$

This differential equation can be integrated numerically for fractional (and negative) as well as integer N . So the Laguerre polynomials for fractional N can be defined as the solutions to the differential equation for fractional N .

But is this really a "fractnomial"? The polynomials for integer N can be regarded as infinite series in which almost all the coefficients are zero (because of the factor $(N-n)$ in equation (1) when $n = (N-1)$). In the case of non-integer N none of the coefficients are exactly zero.

J.R. Sutton
16A Overland Road
Mumbles
SWANSEA SA3 4LP

Renaissance Data Systems
P. O. Box 421 - V
Georgetown, CT 06829
(212) 864-3078

Renaissance Data Systems announces a change in its mailing address.
Please note that the telephone number remains the same.

If you would like a copy of our latest catalog of books on APL and J and other curiosities of merit, please send us a self-addressed legal sized envelope with one first class stamp (if in the U.S.).

Included are such titles as:

APL is EASY,
APL - An Interactive Approach,
APL2 at a Glance,
APL - the Language and its Actuarial Applications,
APL as a Tool of Thought proceedings,
I-APL publications and software,
Boolean Functions and Techniques,
The FinnAPL Idiom List,
The Toronto APL Toolkit,
Mathematical Experiments on the Computer,
Probability in APL,
APL - Stat: Do it yourself guide to computational statistics,
A Source Book in APL

Approximately 80 titles in all!

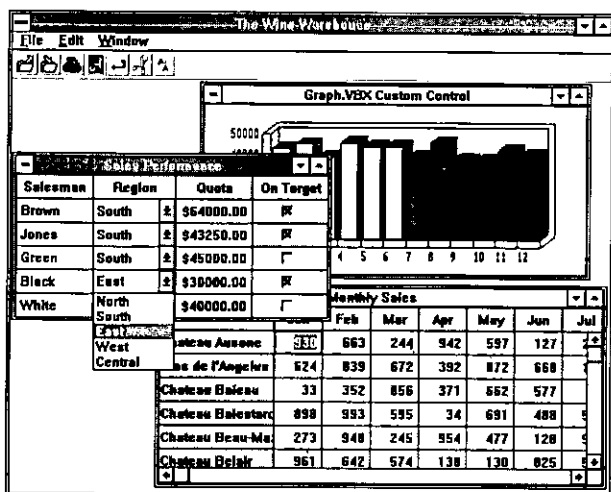
J publications:

Programming in J,
An Implementation in J (structure and source code),
Arithmetic, and Calculus, and
J2.

Shareware and commercial APL interpreters and J interpreters are available as well.

dyalog APL

The Definitive APL for Windows™



Dyalog APL/W Version 7.1 includes a sophisticated built-in Grid object, with numeric, currency, and date fields as well as combo boxes and button cells. The Grid also provides an undo feature, cut and paste facilities (which let you move data quickly and easily between APL and your favourite spreadsheet), resizable rows, columns and titles, and drag/drop editing. Dyalog APL/W supports Visual Basic Custom Controls, ToolBar, StatusBar and TabBar objects, automatic Hints and Tips, Metafiles, MDI, 3-D Forms and Controls, a fully customizable Session, an ODBC interface, namespaces for encapsulation, and a host of other features; all designed to make it easy to develop fast, responsive and attractive Windows applications.

That is why Dyalog APL/W remains the professional choice. For further details, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke, Hampshire, RG24 7AL, United Kingdom.

Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com.



APL Product Guide

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Pressure on space occasionally prevents us from printing the complete guide, however updates will always be listed. We do depend on the alacrity of vendors to keep us informed about their products. Anyone who is not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 01439-788385, Email: 100331.644@Compuserve.com

fr

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,666	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Interprocess Systems	APL2 Dev't Workstation	poa	Mainframe APL2 supported on a PS/2 via a co-processor card with 16Mb of memory running VM/ESA (370 mode). A complete system includes a PS/2, a P/370 co-processor card, and software licenses for VM/ESA, APL2, GDDM and the full line of Interprocess APL2 enhancements.
MicroAPL	IBM RS/6000	12,000+	POWER range of RISC systems running AIX. Dumb terminal or graphical interface.
	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
Optima	IBM Compatible	poa	Complete PC-based station, APL interpreters & all support eq't

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time	poa	
	Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.
Atlantis Software	Analytic Platform (K)	poa	K is an APL-like language
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL*PLUS PC Rel 11	250	STSC's full featured APL for IBMs and compatibles - Version 11 gives free runtime.

	APL*PLUS III for Windows	949	The new 32-bit native Windows APL*PLUS. Develop in Windows, and distribute APL applications with no runtime charges. Reasonable migration charges from APL*PLUS/PC and APL*PLUS II.
	APL*PLUS II for DOS	750	Now that APL*PLUS III for Windows is available, the facility for creating Windows applications in PLUS II has been removed, and the price reduced.
	APL*PLUS II for UNIX	poa	STSC's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altiq, Apollo, Bull, Dec, HP, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL
I-APL Ltd	I-APL/PC or clones	8 - 11	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	11	As above
	I-APL/Macintosh	13	As above
	Iverson Software Inc		I-APL is the UK agent for all ISI products, including APLIWIN and JWIN for PC and many other machines.
I-APL/ISI	APLIWIN	50	Windows APL including manual
I-APL/ISI	JWIN	55	"Personal Edition"; please call for details Please note the packing charge of £3 per order.
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APIs; TIME facility
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
	APL*PLUS/PC	poa	APL systems marketed and supported ...
Insight Systems	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc.	APL386	\$30	Sharp APL Release 20 for PC 386, 486 with graphics, and ability to operate under Windows.

	APLIPC	\$30	For PC under DOS
	APLIWIN	\$30	For 386/PC under Windows 3.1
	APL Reference Manual	\$30	Documentation for all the above.
	J System Kit	\$24	J 6.2 diskette with manual "J:Introduction and Dictionary"
	J Source Code	\$90	Full C source code plus 100-page book
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL.68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL.68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL.68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL.68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing Interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing Interface, hardware floating point.
	APL.68000 Level II ST	170	Second generation APL. Full windowing Interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing Interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing Interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Optima	APL*PLUS/PC	369	
	APL*PLUS II	950	
	APL*PLUS II PC Developers Kit	poa	
	Dyalog APL	999	
RE Time Tracker Oy	APL*PLUS/PC	poa	Complete APL*PLUS and Statgraphics product range and user support for Finland
	APL*PLUS II/DOS		
	APL*PLUS II/WIN		
	APL*PLUS/UNIX		
Soliton Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Unlware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
APL-385	APL-385 for APL*PLUS/PC FSM-385 DRAW-385	50	Including ... Screen development Screen design

	DB-385 GEN-385		Relational W.S. Miscellaneous Utilities
The APL Group	Qualedi	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler!
(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems; multi-user storage of APL2 arrays with efficient disk usage.
Causeway	Causeway for Dyalog/W	\$50	Manuals and Class-management utilities for the Causeway platform supplied free with Dyalog APL/W
	Causeway for APL*PLUS III	\$50	Software, class-management utilities and printed documentation for Causeway under APL*PLUS III
	Rain Graphics Workspace	\$250	Full on-line documentation (Windows Help) and handy-reference card for the Rain business and statistical graphics workspace supplied with Dyalog/W.
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	HELM	poa	Decision Support system for top management. Developed in Italy over 7 years. Requires APL mainframe or APL*PLUS/II. Optional modules: EIS, Excel Interface, DTP output via LATEX, output on map background.
CYBEX AB	APL Graf/PC	290	Presentation graphics for APL*PLUS/PC (CGI)
	APL Graf II/PC	390	Presentation graphics for APL*PLUS II/PC (CGI).
	Utility Functions APL2	1900	For APL mainframe; Incl. a very fast search.
	Utility Functions II/PC	130	Same package for APL*PLUS II/PC.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to Import IBM .ATF files to APL*PLUS.

	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces/AC/Graf		15	Graph plotting for I-APL/Mac
	IAC/Vox	15	Spoken APL characters for I-APL/Mac
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
Impetus Ltd	Impetus	poa	Corporate Modelling and Reporting System.
INFOSTROY	APL*PLUS/Xbase interface (i1/886 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(PC Version 2)	\$98	As above for APL*PLUS/PC.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet Interface. "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs
	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequelLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
Interprocess Systems	APL2 Development Workstation	poa	
	IEDIT	\$3000-5000	Full screen APL2 editor with immediate APL execution, and full-screen debugger
(mainframe)	AFM	\$15300	High performance component and keyed file system (VS APL and APL2)
	Enhanced Format	\$2575	A QuadFMT data formatter for VS APL and APL2
	PowerCode	\$2000	External functions for APL2
	WSORG	\$1500	Full-screen Workspace Organizer for APL2.
JAD Software	JAD SMS	150-500	Software management system for APL*PLUS II based on hierarchical databases; includes full-screen interface and stand-alone functions. Price depends on number of users.
Lingo Allegro	FRESCO Business Graphics	\$250	Fast and Easy Business Graphics DLL
	GDDME	\$1000	AP126 GDDM graphics emulation for Dyalog/W
	AP127	\$250	ODBC interface for Dyalog/W
	FACS	\$1000	EMMA-like functions for SQL tables
	TOPR	poa	APL Code and Application Management for Dyalog APL/W
	Rumba Connection	\$250	Connect Mainframe APL to Dyalog APL/W using Rumba
	IRMA Connection	\$250	Connect Mainframe to Dyalog APL/W using Irma for Windows

Mercia	LOGOL 92	poa	Logistics management system for 386/486 & RISC computers. Sales Forecasting, Inventory Management, Master Scheduling, Distribution Requirements Planning, Sales & Operations Planning.
	TWIGS	poa	A modular library of tools to teach and explore state-of-the-art materials management concepts. Developed by R.G. Brown.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
	STATGRAPHICS Rel 5	590	
RE Time Tracker Oy	UITJW	poa	TMT-Team Oy's User Interface Toolkit for APL*PLUS II and PLUS III under Windows. Comprehensive spreadsheets, replicated fields, special field types, etc.
	DB+	poa	TMT-Team Oy's database Interface for APL*PLUS II & PLUS III under Windows. Interfaces to almost twenty different databases.
Sollton Associates	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with Interfaces to DB2 and MVS data
UNIWARE (for mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL
	STSC's SHAREFILE	poa	component files for IBM VSAPL and for IBM APL2
	TOOLS & UTILITIES	poa	Including FILEPRINT, FILESORT, FILECONVERT FILEMANAGER(EMMA) STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL(VM or TSO)
(for APL*PLUS/PC)	APL Debugger 2.1	FF1950 FF9750	A visual APL debugger to help develop applications (site license)
	Menus 3.0	FF2450 FF12250	Complete set of hierarchical menu utilities (site license)
	ETATGEN 2.0	FF1950 FF9750	Page layout report generator (site license)
	UNITAB 2.0	FF4550 FF22750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNIASM 3.0 (site license)	FF4950	Assembler utilities to speed up APL*PLUS/PC applications
	UNISTAT 5.1	FF2900	Data analysis add-on module for Statgraphics
(for APL*PLUS II)	UNIWARE Toolkit II 4.1	FF39000	(site license only). Relational database system and complete set of utilities for APL*PLUS II development
	APL Debugger II 2.1	FF2950 FF14750	A visual APL debugger to help develop applications (site license)
	Menus II 4.0	FF3950 FF19750	Complete set of hierarchical mouse-driven menu utilities (site license)
	ETATGEN II 2.0	FF2950 FF14750	Page layout report generator (site license)
	UNITAB II 2.0	FF5950 FF34750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNISTAT Plus 5.2	FF4300	Data analysis add-on module for Statgraphics

Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9519067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	APL Programming	poa	Short or long-term consultant available.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, specialises in tree-processing algorithms.
APL People	Consultancy	poa	Consultants available at all levels. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	Management and upkeep of the Causeway development environment for individuals and large APL sites. On-site training for Causeway/Dyalog and Causeway/Plus III
Paul Chapman	Consultancy	poa	24-hr programmer: APL, C, Assembler, Graphics: PC, mini, mainframe and network.
David Crossley	Consultancy	poa	Broad experience in many APL environments
Peter Cyrilax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Dogon Research	Consultancy	poa	APL Systems consultancy, design, implementation, support, documentation and maintenance. All dialects with special emphasis on APL2 and Dyalog APL/W.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping; all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
Evestic AB	Consultancy	poa	Excellent track record from 10+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
General Software	Consultancy	from 120	
Greyman'te Assoc	Consulting	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.

Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
IAC/Human Interfaces	Consultancy	350	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using L-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	APL*PLUS & Windows consultancy. Porting of software written in C into APL*PLUS.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
Intelligent Programs	Consultancy	poa	Systems development, enhancements, support.
	Documentation	poa	Preparation of new manuals, rewriting of existing materials.
	Training	poa	Training for APL experts through to non-technical system users.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Kestrel	Consultancy	poa	All APLs, all environments. Design, analysis, coding, maintenance, documentation, training, interfacing.
Lingo Allegro USA	Consultancy	poa	General APL consultancy specializing in Prototyping, Migration, Mainframe to PC Downsizing, Performance Analysis, Troubleshooting, and Graphics.
MasterWork Software	Consultancy	poa	Consultancy
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Optima	Consultancy	poa	A range of consultants with 3-15 yrs APL PC and mf experience.
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	350	Specialising in Banking, Financial & Planning Systems.
RE Time Tracker Oy	Consultancy	poa	Specialised in comprehensive APL Windows user Interfaces, APL Multimedia, APL to API level interfacing for Windows, Windows applications, DLLs & databases. Also franchising consultancy
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Snake Island Research Inc	Consultancy	poa	23 years APL experience. Language design and implementation Performance consulting Database and other applications
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Unlware	Consultancy (Senior)	FF/day 5000	Consultancy from people with at least 8 years APL experience.
	Consultancy (Senior)	FF/day 7500	Advice and training in Windows programming with APL*PLUS II
	Training	FF10000	5-day class on Windows programming with PLUS II version 4.0
Wickliffe Computer	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Employees placed at all levels.
Bloomsbury Software	Training	poa	Contact the company for details.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — In English or French.
	The BBS\APL	\$24 p.a.	703-528-7617, 1200-14400b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 meg of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr). Selection available on disk for \$15 post-paid. Free on-disk catalogue.
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL In Social Studies	3	35pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	J Dictionary	16	by Ken Iverson
	Programming In J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	An Introduction to J	8	47pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	18	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
Iverson Software Inc.	Programming In J	\$15	76pp.
	Tangible Math	\$12	34pp.
	Arithmetic	\$18	123pp.
Kestrel	Employment	poa	Permanent and contract, home and abroad. From individual placement to supply of complete project teams.
	Software Library	poa	Low-cost software distribution service; call for details.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Solliton Associates	MVSJUNK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 Interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM/SIGAPL	International	Quote Quad		
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
APL Interest Group	South Africa	-	-	
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Chicago SIG	Chicago			
Hartford Group	Hartford, Connecticut, USA			

CPCUG APL SIG (Capital PCUG)	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland		Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc.	Tokyo, Japan			
Melbourne APLUG	Melbourne, Australia		Quarterly meetings	free
New York SIG	New York, USA			
Potomac SIG	Washington DC, USA		Free monthly meetings	
Rochester APL	Rochester, New York			
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter		
SOCAL	Southern California		Seminars	\$15 (\$5 students)
SovAPL	Obninsk, Russia			
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qly SI-Info		SF60 (SI) + SF20 (SAUG)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings	
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
ACM/SIGAPL	Donna Baglio	ACM, 1515 Broadway, New York, NY 10036 USA Tel: +1 (212) 626-0605 Email: baglio@acm.org
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX, UK. Tel: 01245-496647; Fax: 01245-496646.
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 589 5578 Fax: +61 3 589 3220
Adatum Systems		13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31-3474-2337, Fax: +31-3474-2342
Andrews	Dr Anne D Wilson	23, The Green, Acomb, YORK YO2 5LL, UK Tel: 01904-792670
APL-385	Adrian Smith	Brook House, Gilling East, York YO6 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
APL Bay Area Users Group		
APLBUG	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: +1 (415) 828-2058 Email: frp21a@prodigy.com
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattermann	Rheinstrasse 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1451
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108
APL Interest Group, South Africa	Mike Montgomery	Private Bag X11, Rivonia 2128, South Africa Tel: +27 (11) 803-7200 Fax: +27 (11) 803-9134 Email: mkmont@spl.co.za
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS UK. Tel: 01225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Grf sur Yvette, France. Fax: +33 1 69-08-79-63
Atlantis Software	Arthur Whitney	1105 Harker Avenue, Palo Alto, CA 94301 USA
BACUS	Joseph de Kerf	Rooienberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24

Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 4, Jenkintown, PA 19046, USA Tel: +1 (215) 732-3735; Fax: +1 (215) 772-1929
The Bloomsbury Software Co Ltd	Peter Day	3-6 Alfred Place, Bloomsbury, London WC1E 7EB UK. Tel: 0171-436 9481; Fax: 0171-436 0524
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9730036. email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS UK Tel: 01252-874897
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB UK. Tel: 0171-404 5401. Compuserve: 100343,3210
Causeway Graphical Systems Ltd	Adrian Smith, Duncan Pearson	5 The Maltings, Castlegate, MALTON, North Yorks YO17 0DP UK Tel: 01853-698760 Fax: 01853-697719 Compuserve: 100265,1564
Chicago SIG	Larry Mysz	836 Highland Drive, Chicago Heights, IL 60411 USA C'serve:73040,3032
Cinerea AB	Rolf Kornemark	Skyttegatan 25, S-193 00 Sigtuna, Sweden.
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652
CPCUG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: +1 (301) 762-9372.
David Crossley		187 Le Tour du Pont, Quartier Le Mourre, 84210 ST DIDIER, France Tel: +33 90-66-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: +46 31-45 37 40. Fax: +46 31-45 24 23.
Peter Cyrix Systems	Peter Cyrix	22 Hereford Road, London W2 4AA UK. Tel: 0171-229 5344
Danish User Group	Per Gjerløf	Email: gjerper@inet.uni-c.dk
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX UK. Tel: 01604-760241
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP UK Tel: 0181-520 6934 Email: bowman@apl.demon.co.uk
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 3474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orplington, Kent BR5 1JS UK. Tel: 01689-824741
Evestic AB	Olle Evero	Bertallusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410
FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS UK. Tel: 0181-864 9537
Greymantle Associates	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY UK Tel: 01442-878065 Email: 100412,1305@compuserve.com
Hartford CT Group	Bob Pomeroy	Mass Mutual Life, 1295 State St, Mallard F455, Springfield, MA 01111 USA Tel: +1 (413) 788-8411x2838
H.M.W.Trading Systems Ltd	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2832@compuserve.com
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Email: dick.holt@acm.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX UK Tel: 01388-527190. Compuserve: 100021,3073
I-APL Ltd	Anthony Camacho (for queries, order forms) J C Business Services (for pre-paid orders only)	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9760036 email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM 56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU UK
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (=2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com C'serve: GO IBMAPL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD UK Tel: 01732-885126

INFOSTROY	Alexei Mroshnikov	3 S. Tulen'In Lane, St. Petersburg 191188 Russia. Tel: +7 812-3111611 Fax: +7 812-3153321 Email: alm@infostroy.spb.ru
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: +45 42 10 70 22 Fax: +45 42 10 75 74 Email: Insight@inet.unl-c.dk
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 0171-265 1120
Interprocess Systems Inc.	Stella Chamberlain	11650 Alpharetta Highway, Suite 455, Roswell, Georgia 30076, USA Tel: +1 (404) 410-1700. Fax: +1 (404) 410-1773 Cserve: 70373,2676
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1895 Fax: +1 (905) 831-5172
Japan APL Association		23-2-302 Hiromichi, Adachi-ku, Tokyo 120, Japan
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessmer Drive, Stevenage, Herts SG1 2DX UK. Tel: 01438-310155 Fax: 01438-310131
Lingo Allegro USA Inc.	Walter G. Fil	113 McHenry Road, Suite 161, Buffalo Grove, IL 60089 USA Tel: +1 (312) 203-4926 Fax: +1 (708) 459-8501 Cserve: 71303,3224
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel and Fax: +64 (4) 232-4440 Email: 100242.2535@compuserve.com
Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3195, Australia Tel: +61 3 586 7574 Fax: +61 3 586 7600 Email: hld@dar.csiro.au
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Henegate Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8868 Fax: 0171-928 1006
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants UK. Tel: 01730-263843
New York SIG APL	Nestor Nelson	PO Box 138, NY 10185-0002, USA
Optima Systems Ltd	Paul Grosvenor	Airport House, Purley Way, Croydon, Surrey CR0 0XY UK. Tel: 0181-781 1812 Fax: 0181-781 1999
Potomac APL SIG	John Martin	Computer Sciences Corp, 1100 West St, Laurel, Maryland 20707-3587 US Tel: +1 (301) 497-2698 Fax: +1 (301) 498-8260 Email: jam@acm.org
QB On-Line Systems	Phillip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB UK. Tel: 01276-855890 Fax: 01276-855301
Renaissance Data Systems	Ed Shaw	PO Box 421, Georgetown, CT 06982, USA. Tel: +1 (212) 864-3078
RE Time Tracker Oy	Richard Eller	PO Box 363, FIN-00101 Helsinki, Finland. Tel: +358-0-400 2777
Rochester APL	Gary Dennis	Soliton Associates, 1100 University Avenue, Rochester, NY 14607 USA Email: gsd@lpsalab.tor.soliton.com
The Rochester Group Inc.	Robert Pullman	50 S. Union St., Rochester NY 14607-1828, USA. Tel: +1 (716) 454-4360. Fax: +1 (716) 454-5430
Rome/Italy SIG	Marlo Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	991 Creekdale Drive, Clarkston, GA30021 USA
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks HP8 4QH UK.
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@ecg.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Wilens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Laurie Howard	Soliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 645 4475 Fax: +31 20 644 1206 Email: ljh@soliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region, Russia Email: askom@apl2.obninsk.su
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (203) 868-0131 Fax: +1 (203) 868-9970 Email: rswain@ix.netcom.com Compuserve: 70062,2303
SWAPL	Stuart Yarus	PO Box 210367, Bedford, Texas 76095, USA Tel: +1 (817) 577-0165 Compuserve: 73700,2545
SwedAPL	Christer Ulthielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 Cserve: 100341,404

Swiss APL User Group

Swiss APL User Group, CH-3001, Bern 1, Switzerland
Email: sl@fi.unizh.ch

Sydney APLUG

Rob Hodgkinson

PO Box 1511, Macquarie Centre, NSW 2113, Australia Tel: +61 2 257 5313

Sykes Systems Inc

Roy Sykes Jr

4649 Willens Ave., Woodland Hills, CA 91364, USA

Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250

Toronto SIG

Marc Griffiths

PO Box 384, Adelaide St Post Office, Toronto Ontario M5C 2J5, Canada

Tel: +1 (416) 532-0943 Email: marcg@utoc.utoronto.ca Cserve: 76260,3314

Uniware

Eric Lescasse

Tour Neptune, Cedex 20, 92085 Paris la Defense 1, France.

Tel: +33 (1) 47-78-78-00. Fax: +33 (1) 40-80-04-11

Wickliffe Computer Ltd

Nick Telfer

76 Victoria Rd, Whitehaven, Cumbria, CA28 6JD UK. Tel: 01946-692588

Warwick University

Prof. Jeff Harrison

Dept of Statistics, University of Warwick, Coventry, CV4 7AL UK

Tel: 01203-523959

Zark Incorporated

Gary A. Bergquist

23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (203) 872-7806

Vector Back Numbers

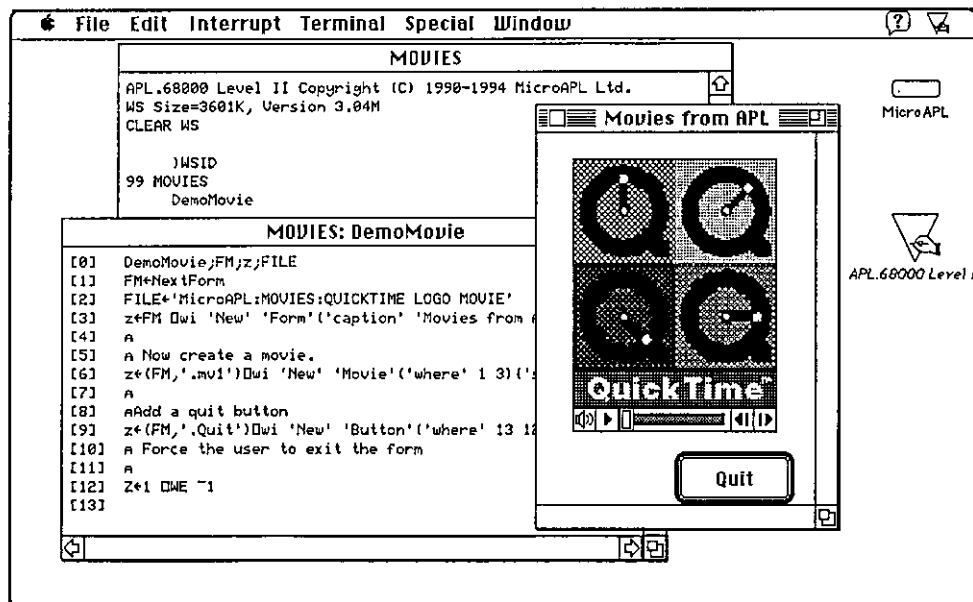
Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

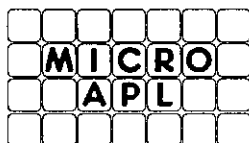
APL for the Apple Macintosh

Since 1985 MicroAPL has pioneered the use of APL in graphical environments. Our latest version of APL.68000 Level II for the Apple Macintosh is now available, offering dramatically enhanced GUI programming facilities.



APL.68000 Level II for the Macintosh includes the following features:

- Runs on all models of Apple Macintosh
- Native version for the Power Macintosh
- Conforms closely to the APL2 specification
- Uses the standard Mac interface
- Object-based GUI programming via DDI
- Full event handling via APL callbacks
- Free runtime version with application packager



MicroAPL Limited South Bank Technopark, 90 London Road,
London, SE1 6LN, UK
Voice: 0171 922 8866
Fax: 0171 928 1006
Applelink: microapl
Internet: microapl@microapl.demon.co.uk

Locales in J

by Chris Burke (Strand Software Systems Inc)

Introduction

This note describes the *locales* facility in J Release 2.

Locales allow you to partition the names in use. The main benefits are:

- name conflicts are avoided when loading two applications at the same time
- commonly-used utility functions can be made available without their names cluttering up listings of work in progress

Similar facilities are provided by Dyalog APL's namespaces, though the design is quite different.

Locales were introduced in J Version 7, and enhanced in Release 2 to allow for a parent/child structure — a small change that provided a dramatic improvement in functionality. The new definition is simple and very effective.

Locales are created and deleted automatically — you create a locale by defining an object in it, and delete a locale by deleting all objects in it.

You can create any number of locales. There may only be one parent locale, and any number of child locales, for example:

Child locales:	(base)	j	laserjet	stats	user
Parent locale:			z		

The only difference between the parent and child locales is in how J searches for a name. If a name is referenced in a child locale and not found, J then searches for the name in the parent locale. If then found, the definition in the parent locale is used, as if it had been found in the child locale; if not found, no further search is made.

The *base* and *z* locales are treated specially.

The Base Locale

The *base* locale is the default locale, i.e. if no specific locale reference is made, names are created and referenced in this locale, for example:

```
x=. 1 2 3
```

defines `x` in the base locale (except where noted, these examples assume there is nothing on the stack). Typically, development work is done entirely in the base locale. The base locale's name is actually empty. In the diagram above, it is shown as `(base)`.

When you invoke a function in another locale, this temporarily switches the default locale from base to this locale; this is the only way of changing the default locale. *A key feature of the design is that the default locale is a child locale — and not the parent locale.*

The z Locale

The `z` locale is the single parent locale. It differs from child locales only in the way `J` searches for a name. For example, consider:

```
res=. format i.100
```

`J` will first search for `format` in the base locale, and if found, will execute it. Otherwise it searches for `format` in the `z` locale, and if found, executes it in the context of the base locale. In both cases, `res` will be defined in the base locale.

If `format` is only defined in the `z` locale, it will not appear in namelists of the base locale. Typically, the `z` locale is used to store utilities that will be available to programs in any child locale, but whose names would not appear in namelists of those locales.

Note that in the above examples, no locale reference was used. Clearly, this would normally be the case when dealing with the base and `z` locales, but it is also typical of the definition as a whole — in practice, you rarely need to make locale references.

Locale References

To specifically reference a name in a locale, you append to the name: underscore, locale name, underscore. For example `calc_def_` is the name `calc` in the locale `def` (read as: `calc in def`); `calc_z_` is therefore `calc` in `z`, and `calc__` is `calc` in base. You create a locale by creating objects in it. For example, assume the locale `temp` does not exist. Enter:

```
x_temp_=. 1 2 3
names_temp_ ''
x
```

The first sentence creates the locale `temp` and the name `x` in it. The second sentence executes the utility names in the context of the `temp` locale (names formats namelists, and is read into the `z` locale with J's standard profile).

Locales have nameclass 6:

```
nl 6
```

def	j	laserjet	stats	temp	user	z
-----	---	----------	-------	------	------	---

A function `calc_def_` is executed in the context of the locale `def`, i.e. it temporarily switches the default locale from `base` to `def` while the function is running. Therefore, names referenced by `calc` have locale `def`, unless their locale is specifically given. The following definition assigns global names `LEN` in locale `def`, and `NDX` in locale `util`:

```
calc_def_=. 3 : 0
LEN=: 10
NDX_util_=: 5
)
```

equivalent to:

```
calc_def_=. 3 : 0
LEN_def_=: 10
NDX_util_=: 5
)
```

Using Locales

Now let's look at a typical example of using locales, and examine why locale names are typically not referenced by the user.

One of the packages supplied with J is a set of functions to support printing with a LaserJet. To load this system, you enter:

```
load 'laserjet'
```

This populates a `laserjet` locale with about 50 utility programs, and reads 5 cover functions into the `z` locale. These cover functions call corresponding functions defined in the `laserjet` locale, for example:

```
ljprint=. ljprint_laserjet_
```

In actual use, only the cover functions are invoked, and the user need not be aware that the real work is being done in the `laserjet` locale, or that a large number of utility functions have been read in.

Now suppose some development work has to be done on the system. Would it not be inconvenient to have to keep referring to names in the `laserjet` locale with their full locale name, e.g. as `ljprint_laserjet_`? The answer is no, since

all development work in J is done on the source script files, which are defined independently of the locales into which they may be read.

Finally, suppose one of the laserjet functions crashes and you want to view local definitions in the suspended function. While the function is suspended, all local names can be referenced without specifying the locale name, since invoking a function in the laserjet locale temporarily switches context to that locale.

Locale Design

Two facilities that have been requested are a system function to change the default locale, and the ability to arbitrarily nest locales.

In fact there seems no compelling reason to provide either facility, and should one ever arise, the definition could be changed with a compatible extension. However, they both seem more trouble than they are worth.

Consider the question of changing the default locale. It may be thought that allowing this would make it easier to reference objects in the new default locale, since its locale name would not have to be used. However, as already mentioned, locale names are rarely used anyway, so there is really no need to be able to switch the default locale. In fact, an early prototype of locales in Release 2 did contain a system function to change the default locale, but this was removed as it became apparent that it was of little use. On the other hand, if we allowed the default locale to be changed, the user would have to be constantly aware of the current setting, and of course recover from errors that may occur because of the wrong setting.

Now let's look at the question of nested locales. If locales allow you to avoid name conflicts within the session as a whole, would not nested locales avoid name conflicts within a given locale? Yes, but the argument is much weaker. It seems reasonable to insist that the code in a given locale have no name conflicts, and in the event a locale were so large that name conflicts were likely, it would be better to partition the locale into two or more separate locales rather than create nested locales to resolve them. Again, nested locales seem likely to lead to more errors.

Summary

Locales have a simple definition which is nevertheless quite adequate. The facility works well in practice, and has provided a major improvement over the flat workspaces of earlier versions of J and APL.

J Release 2.05 for Windows

*Reviewed by Dave Ziemann
dziemann@easynet.co.uk*

Introduction

Release 2.05 of J is a significant new version of the product. This review will focus on the J environment, Windows programming, DDE and J runtime. It does not cover the J language itself, the Open Database Connectivity (ODBC) interface (which allows J to access data from a DBMS via SQL), or the J component filing system.

I received release 2.04 of the Professional Edition of J via s-mail (the "s" stands for snail) from Strand Software Systems, the Canadian distributor. The package consisted of two glue-bound manuals, and five diskettes. Needless to say, the moment the package arrived, it was out of date. This gave me the opportunity to try out my new fast modem, and I arranged with Eric Iverson to download the latest release, 2.05 from his World Wide Web page. This is a more straightforward but less usual software distribution method than FTP, but it works just fine provided the host isn't too heavily loaded, which it wasn't at 7:30am UK time.

The Documentation

There are two manuals. The *"Introduction and Dictionary"*, written by Ken Iverson, will be familiar to J users. This is fundamentally a reference work, although it also contains introductory tutorials. It has the J vocabulary printed right there on the outside back cover of the manual, where you need it.

The *"User Manual"* is written by Chris Burke of Strand Software, and is in sharp and welcome contrast to the Introduction and Dictionary. Although both weigh in at 200 pages, the User Manual is informal in style, and contains explanatory text and practical examples. It does not concern itself with the core language, but concentrates on the environment, utilities, windows programming and database access.

These topics are covered in chapters entitled Session Manager, Writing J Programs, Application Development Environment, Standard Utilities, Window Driver, DDE and OLE, VBX Controls, ODBC Data Driver, and Packages and Examples.

One appendix contains a complete listing of the commands that can be used in the Window Driver scripting language, the heart of windows programming in J. Another appendix contains, perhaps surprisingly, a summary of the base ODBC SQL language and function calls. As you can see, the emphasis here is on the practical side of application development. I wish they had also printed the J vocabulary on the back of the User Guide.

Installation

Installation is straightforward; the "J setup" disk is the main one, and the "VBX and SQL ODBC Setup" disk is good for many of the User Manual examples. The other two disks are for ODBC setup, and can be installed as and when needed. The Professional Edition also includes a J Runtime Setup disk.

Installation results in a single Program Group icon. No uninstall icon is provided. The idea of removing J from a machine may surprise some people, but I believe that all professional Windows products should nowadays come with a comprehensive uninstall facility.

J Application Development Environment

The J development environment is a tidy admixture of utilities, scripts and locales, sitting underneath the J MDI session manager. The session manager has been described in a previous product review, so I won't dwell on it here, other than to suggest that it could be improved by enabling right mouse-button clicks, in particular for Windows *cut*, *copy* and *paste* operations.

Although the workspace appears to be initially empty, objects defined in utility script files inhabit the j and z locales, and are available for use. Locales are J's answer to the namespace problem so familiar to APL developers.

If a name cannot be resolved in the base local, where we normally work, the z locale is searched. In this way you can treat common objects as system global, even though they don't impact the namespace of the base locale. For example, if you go into J and immediately type `n1 3` you get an empty list. This is because although the verb `n1` is defined in the z locale, you are running it in the base locale, where there are no current definitions.

The z locale is populated with general purpose utilities and windows programming tools. For example, it contains the wonderfully useful utility `edit` which allows you to edit by name any definition that has been loaded in a J script file. You can type:

```
edit 'list'
```

and a window will open up on the script file that contains the definition of the verb `list`, with the definition scrolled to the top of the screen. This works because J now keeps a record of all the script files that have been loaded, and also remembers the names of the objects they defined.

In addition to useful utilities, there are a number of common nouns in the `z` locale, such as `CR`, `LF` and `TAB`, as well as a host of convenient nouns for use in Windows programming.

Windows Programming

If the authors of J were once language mavens, then the balance has been redressed. The important application areas appear to be solidly covered, particularly the Windows programming environment. So just how easy is it to do Windows programming? You can start doing Windows programming as soon as you load J. The core utility is the Window Driver verb `wd` which is already loaded into the `z` locale.

To query the name of the active session window, you can use:

```
wd 'qsmact;'
```

*type	10 result
*parent	sysresult
*data	c:\j205\temp\2.jx

This is not particularly interesting in itself; the purpose of the example is to show that the result is a 2-column matrix. Column 1 contains identifiers, and column 2 the corresponding data. Typically there are several rows. The first indicates the type of the response, in this case a Windows result. Often the most useful item is the data corresponding to the `*data` identifier.

The verb `wddata` sucks this data out of the box, e.g.:

```
wddata 'qsmact;'  
c:\j205\temp\2.jx
```

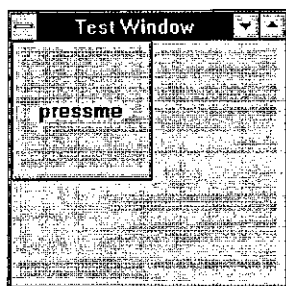
The universe of `*type` responses from `wd` (e.g. `10 result`) can be shown by using the `qt` command in conjunction with the `list` utility:

```

60 list wddata 'qt;'
0 ?          1 error          2 nowait        3 fkey
4 menu       5 button         6 close         7 enter
8 cancel     9 break          10 result       11 ddepoke
12 ddeex     13 ddereq        14 ddeadvice    15 ddestart
16 smbbreak  17 timer         18 vbxn

```

What you type inside the quotes must conform to the rules of the Windows scripting language developed by Eric Iverson. If the last command is "wait" then a result is only returned after the user has taken some action. Since several windows may be active at any time, the response may indicate an action in any of these windows. For example:



```

t='.pc mywin;pn "Test Window";'
t=.t,'cc pressme button;pshow;wait;'
wd t

```

creates a window mywin (pc=parent create) with title "Test Window" (pn=parent name), and then adds to it a push button called pressme (cc=child create). The window is then displayed (pshow) and a wait actioned.

In the absence of sizing and positioning information appropriate assumptions are made. The window remains active until explicitly closed by a wd'pclose;' command. When you press the push button, J gets the result:

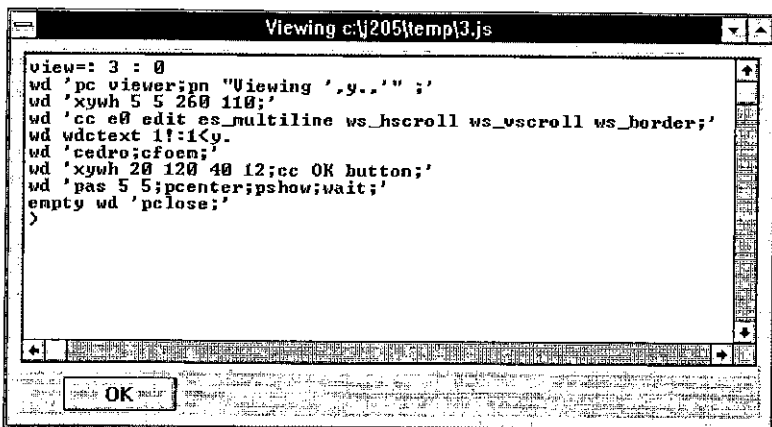
*type	5 button
*parent	mywin
*id	pressme
*pinfo	1 0 0 200 200 192 173 800 600

Window Driver commands can be used with the wd verb to build a simple file viewer in a few lines of code. I saved the definition of a verb named view in a J script file called c:\j205\temp\3.js and then ran the script and the viewer as follows:

```

0!:0<'c:\j205\temp\3.js' NB. run the script
view 'c:\j205\temp\3.js' NB. run the viewer

```

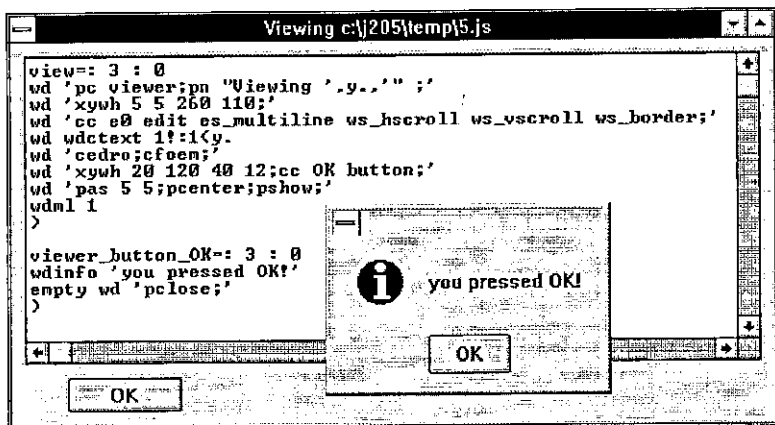


The argument *y.* is the file name. The *1!:1* verb reads the file as a character vector, and the utility *wdctext* constructs a *ctext* command from its argument. The *ctext* command writes the text into the listbox child. The *pas* command adjusts the size of the parent window, *pshow* shows the window and *wait* causes the wait for user input. The *empty* verb merely discards the explicit result of the *pclose* when the OK button is pressed.

In general we want to set up a "message loop" where we wait for the user to do something, respond to the resulting Windows message, take an appropriate action and then re-issue the wait command. This is handled in J by using the standard verb *wdml*.

How do we catch and respond to Windows events? You need to know three things to do this: the **parent*, **type* and **id* of the Windows result. The *wddl* verb looks for a handler function whose name is formed from these three parts, i.e. a function of the form *parent_type_id*. The argument to this function is the result from Windows. So if we wanted to handle the event resulting from pressing the OK button in the above example, we must define a handler verb *viewer_button_OK*.

I did this in the script that defined the view verb, and this is what happened when I clicked the OK button:



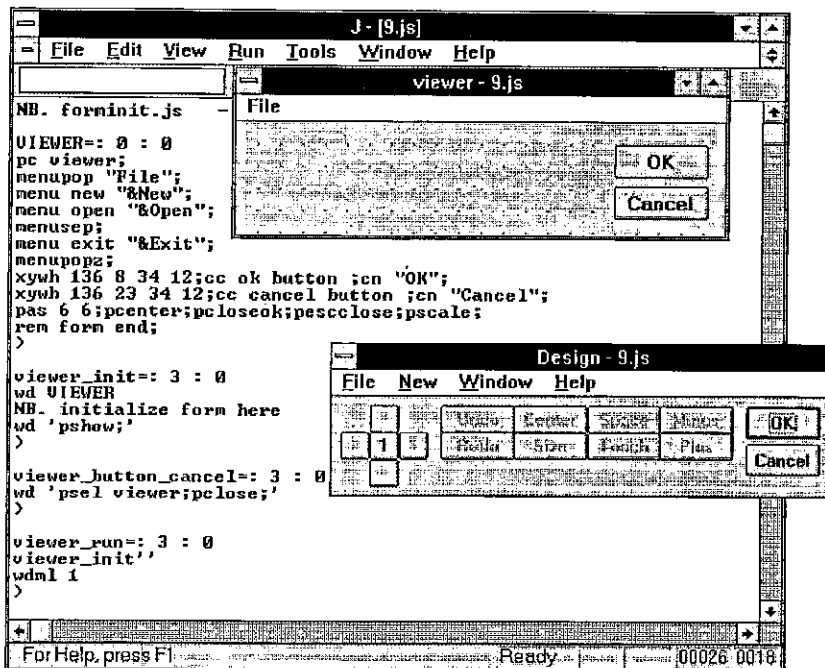
I removed the wait command and put in a call to wtml. The button handler uses the winfo utility to display a simple dialog box to test the approach. This is a major improvement over earlier versions of J, where a single handler verb was used for each parent window, leaving you the work of identifying and actioning each control event. Now the program code for a single control event is expressed within a single verb, the handler verb for the particular control event.

The Form Editor

No Windows-based product is nowadays complete without a window builder or form editor to do the work of building windows for you. J's answer to this is the new "form editor".

As a simple experiment, I thought I'd recreate the hand coded file viewer using the form editor. I opened a new script window and selected the "Form Edit" option which is installed as standard on J's "Edit" menu. I was then prompted to name the form in my mini-application. As soon as I entered "viewer" and pressed enter, the form editor immediately starting writing code, and put it into my new script window. This code creates standard OK and Cancel buttons, and builds a standard "File" menu option with "New", "Open" and "Exit" options already present. This is code that you would otherwise have to write by hand. It also builds a stub verb which you can use to initialise the controls prior to the window being opened.

The system then runs this code, which displays a skeleton form, and presents you with the form editor Design window, from which the form construction process is driven. This is what I got:



You can then use the Design window to select from a list of Windows controls and position them on the form. The grid of square buttons in the design window is used for accurate positioning of controls in the form. The number in the middle is the number of pixels the control moves with each click. It can be toggled between 1 and 4.

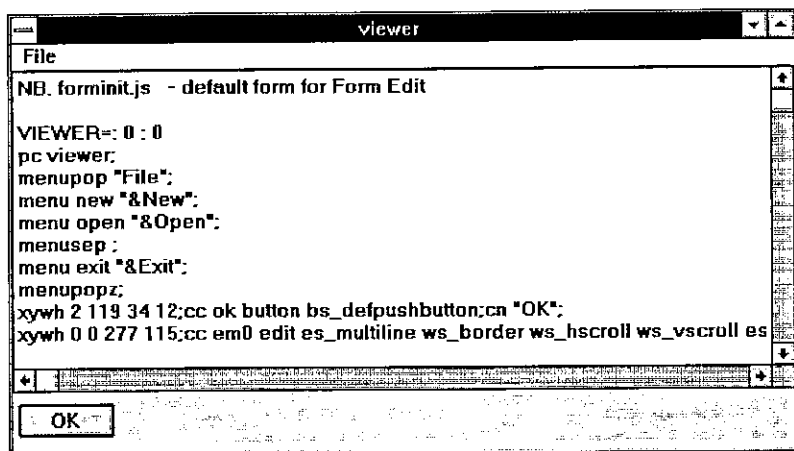
By double-clicking a control in the form, you open up a window that lets you edit its attributes. For example, I selected a multi-line edit control, and specified that it should be read-only and permit horizontal and vertical scrolling. I also deleted the Cancel button, because it's not required for this simple viewer.

As you design the form, the underlying script is modified, building the code which defines the layout you want. At any time you can test the form to see what it looks like. Particularly useful is the ability to select a control in your form, and

then go directly to the script code which handles that control. So you can write the code that handles a control event as soon as you put the control on the screen.

The only coding I had to do myself was to pass the file name argument from the calling verb down into the initialisation verb, and to initialise the edit control with the file contents. This is what I got when I ran it:

```
viewer_run'c:\j205\temp\10.js'
```



The form editor is simple to use and makes the process of window construction a lot easier. This is not a sophisticated editor, for example it won't allow you to define resizing rules for individual form controls, and it doesn't let you drag-drop controls. On the other hand, it does give you a lot of good strong functionality, such as a VBX control, which you just define in your form like any other control, double-click it, and hook into your favourite VBX.

... please turn to page 155

RECENT MEETINGS

This section of Vector documents all British APL Association meetings, and any other events of interest to the APL world. If you have recently attended any gathering which you feel would be interesting to Vector readers, please let the Editor have a brief note, and we will include it here.

APL in a Client-Server Environment

BAA Workshop notes by Jon Sandles

Background

This year the BAA decided to try to do something a bit different with its program of events. The Friday afternoon talks were under-attended and often uninspiring events – the highlight being the tea-break when you could meet old friends and chat about current trends! The message coming through was that people were desperate to find out how to build Windows applications quickly and how to take advantage of the various nuts and bolts that the Windows operating system provides.

The committee decided, after good results/feedback from the BAA-GUI workshop a year and a half ago and the workshops at Swansea, that our time/money was better spent running a series of one-day workshops through 1995. The first workshop was held at City University in January. Unfortunately Duncan Pearson had to drop out due to work commitments, but thanks to Morten Kromberg for coming over from Denmark and Adrian Smith for covering for Duncan. Also thanks to Duncan for lending me his notes – the majority of which are reported below.

DLLs

DLLs are Dynamic Link Libraries. These are libraries of utility C-code on which most of the Windows Operating System is dependent – but are made available for any program to call. To a C programmer calling these functions is simple and analogous to using any other library of C functions. APL has had to come up with new low-level quad functions to interface with these libraries. The two main commercial PC/Windows APLs have come up with differing methods of calling these DLLs.

One of the most difficult things about DLLs even before you have worked out how to call one from your APL is finding out whether a DLL exists for what you want to do and then finding out the arguments required to call that DLL. The best place to start is the Microsoft *Software Development Kit* (SDK) and the associated help file which comes bundled with lots of Windows development tools. DLLs are documented using C Syntax and their arguments are therefore C datatypes (e.g. `int` `char` `struct` etc.). Many of these datatypes are defined in the header

file of the library you are using or `windows.h` (e.g. `LPCSTR` `UINT` etc.) Hence, before you can even begin to call a DLL you need a reasonable knowledge of C and the Windows environment.

Once you've got to grips with some of this you'll want to call one of these from APL. I'll explain how to do this in Dyalog first - because it teaches you more about how the interface works. PLUS III avoids a lot of the tricky stuff - but as a result hides what's really going on.

In Dyalog you must define the function (via `⌈na`) you are about to call by declaring the C datatypes you are about to pass to it. Dyalog is then able to translate your APL arrays into C datatypes and successfully call the function

```
{aplfnname} ⌈na 'returntype dllname.P16|cfname argtype1 argtype2 ...'
```

P16 is the calling structure - Pascal 16 bit - most Windows 3.1 DLLs execute in this mode.

Once defined you can call it like a normal function with APL variable arguments (`r+aplfnname arg1 arg2 ...`). Dyalog converts your arguments into the C data types that you defined for the function and then calls the C function with these arguments. If you define an argument incorrectly you will almost certainly find the function generates a *General Protection Fault* (GPF) when it is run or it returns junk information. This can be very frustrating!

Dyalog uses a special syntax to map APL variables onto C datatypes - the following table gives some common examples:

Argtype	Description	C typedef	Arg
<OT	ASCII string used by the function	LPCSTR	text vector
U	unsigned integer	UINT	scalar integer
>OT	ASCII string returned by function	LPSTR	number of bytes for buffer
=OT	ASCII string both used and returned by the fn	LPSTR	text vector
I	signed integer	INT	scalar integer
U4	long (4 bytes unsigned)	LONG	scalar integer
=I[n]	array of n signed integers (in and out)	INT i[n]	n element vector of integers
>I[]	array of ? signed integers (out only)	INT *	no of integers to be returned
>U1[]	array of ? unsigned integers (out only)	HUGE *	no of unsigned ints to be returned

Examples

Opening, Closing and Reading files – Windows style! Although APL provides its own native file handling we can also use the Windows file-tie and read utilities.

`_lopen` is the Windows equivalent to `FTIE`. In the SDK documentation `_lopen` takes two arguments – a filename (pointer to a null terminated string – thus we declare it as `<0T`) and a mode in which to open it (int – `I`) – it also returns a file handle (int – `I`). To run the function we must use `NA` to associate the DLL function/library with an APL function name. Thus:

```
NA'I kernel.P16|_lopen <0T I'
```

creates a function called `_lopen` (it defaults to the C function name if no left argument is supplied to `NA`). The SDK documentation tells us that the open modes are `READ`, `READWRITE` and `WRITE` – these are predefined C constants – unfortunately we must use the real integer values of these constants and hence look in the `windows.h` to find out what these constants really are in this implementation of Windows (presumably its done this way for portability of code) The value of `READ` is 0. Hence:

```
fh+_lopen 'c:\windows\system.ini' 0
```

gets us a read-only file handle. To read the file we need to find out how big it is first because we can only read 64K bytes at a time and hence we are going to have to loop if it's bigger than that limit. The function `_llseek` moves a pointer through a file and we can use this to discover the size of the file. The SDK tells us it takes three arguments – a file handle (int-`I`) – a number of bytes to move (long-`U4`) and a position to move from (int-`I`) and it returns the new offset in bytes of the pointer from the beginning of the file (long-`U4`) – thus:

```
NA'U4 kernel.P16|_llseek I U4 I'
```

associates the function and the SDK also tells us the possible values for the position to move from are 0,1 and 2. A 2 means that the position to move the pointer from is the end of the file. A 0 means move the pointer from the beginning of the file. (A 1 is move it from wherever it currently is), so:

```
size+_llseek fh 0 2
sink+_llseek fh 0 0
```

moves the pointer 0 bytes from the end of the file (i.e. to the end of the file!) and hence returns the size of the file. We then stick the pointer back to the beginning of the file which is where we want to start reading from. As long as `size` is less

than 64K we can read the file in one glob - we will assume it was for simplicity!
We can now read the file in.

`_lread` is the function which the SDK tells us to use to read files and takes three arguments - a file handle (`int-I`) - an address of a buffer to put the data in (`huge*>U1[]`) - and the number of bytes to read (`uint-U2`). It returns the number of bytes successfully read in (`uint - U2`) ...

```
⊞NA'I kernel.P16|_lread I >U1[] U2'
ct bytes←_lread fh size size
```

... reads from the file `fh` and puts the result in the variable `bytes` - the number of bytes read is `ct` which in this case will be the same as `size` because we checked how big it was first. Notice how Dyalog handles arrays which are returned by the DLL. You simply pass the required size of the array (so Dyalog can create a buffer the correct size) and it then appears as an extra result of the function so that you can assign it into a variable. PLUS III handles this differently - I will show how later.

As in APL we have to free the file-handle once we've finished with it. `_lclose` does this and takes just a file-handle as its argument and returns 0 or an error code if unsuccessful:

```
⊞NA'I kernel.P16|_lclose I'
Done:sink←_lclose fh
```

That's all there is to it! (How many Windows SDK programmers does it take to change a light bulb? 1 to write `GetBulbHandle`, 1 to write `GetBulbContext`, 1 to write `GetBulbStatus`, 1 to write `FreeBulbHandle` etc.)

As I mentioned earlier the real problem with this stuff is not writing it but knowing/finding which functions to call or whether a function even exists for what you want. For example when faced with deleting a file I immediately looked in the SDK under `D*` but found nothing, then `_*`, still nothing - finally I looked for anything with `File` in it and found `OpenFile`. `OpenFile`'s 3rd argument is a mode, one of which is `OF_DELETE` (512) which deletes the file! Hence:

```
⊞NA'I kernel.P16|OpenFile <0T={U1 U1 U U1[4] T[128]} U'
file←OpenFile 'c:\autoexec.bak'(136 0 0(4p0)(128+' '))512
```

deletes the file and returns a file handle. The structure is for the file details and is filled with information about the file - we are probably not interested in this if we have just deleted it!

Structures

In the previous example the 2nd argument of `Openfile` was a pointer to C-structure which was both passed to the function and returned. This was represented by `={ * }`. Where the `*` is the individual elements of the structure. In Dyalog's current implementation of this you cannot nest the structures, i.e. if a structure contains a pointer to another structure, i.e. you can't do `{ * { * } * }`. This means that you cannot call some of the DLLs in Dyalog. (You actually can call them but you have to allocate your own memory and pass your own pointers as *I*4s. This is probably beyond most mortal APLers.). Once again the PLUS III implementation gets around this...

Calling DLLs from PLUS III

Manugistics have a much more advanced interface to DLLs than Dyalog. As well as supporting `□NA` they have an interface called `□wcall` which gives you immediate access to all the more common Windows DLLs. So instead of spending half your development time with your head buried in the SDK manual you just have to know the function exists! For example:

```
□wcall '_lread fh size size'
```

would read a file of size *size* (already opened with `_lopen` etc.). They achieve this by defining the association of a DLL call in the `aplw.adf`. All the common calls have been pre-defined in this file and also you can add any new ones (you have to map all the datatypes as in Dyalog and then recompile the `.adf` file) and also document all the calls – so if you forget the arguments you can query the call, and it will return the documented calling syntax. Also in the `.adf` you can define not only function calls but you can also define C structures and other derived data types. These mappings can then be used by the function mappings themselves and allow you to define functions which refer to nested structures etc., hence overcoming the problem that Dyalog `□NA` has.

`□wcall` also has the benefit of handling output arrays in a slightly more consistent manner. Rather than using Dyalog's approach of having to pass the size of the output array and then returning the result as an extra returned element, PLUS III requires you to create a variable of the right size and returns this variable updated.

Not having a PLUS III interpreter/documentation this is as much as I can tell you. I'm sure that *Vector* readers would be very interested in reading an article that describes the PLUS III `.adf` in detail and how to add new definitions to it and also how to define structures, if there's anyone out there who can write it – please do!

Common Problems Calling DLLs in Dyalog

The workshop highlighted some of the problems people have when trying to call DLLs from Dyalog. The exercise was to use `WritePrivateProfileString` to create an ini file called "dlltest.ini" in the windows directory with a section "[User]" and an entry "name" with the value of someone's name, for example:

```
[User]
name=jon sandles
```

The function just takes four pointers to strings as arguments:

```
'wpps' NA I kernel.P16 | WritePrivateProfileString <0T <0T <0T <0T'
```

Most people at the workshop managed this quite quickly. A few got the `<0T` confused with `>0T` which defines OK when `NA`'d but gives a domain error when called with the required arguments:

```
wpps 'User' 'name' 'jon sandles' 'dlltest'
DOMAIN ERROR
wpps 'User' 'name' 'jon sandles' 'dlltest'
```

Because it was defined (incorrectly) as a `>0T` the interpreter returns a `DOMAIN ERROR` because it is expecting a buffer size for the returned strings, hence:

```
wpps 0 0 0 0
0 K1ör
```

works fine – but returns 0 (for failure) and gobbledegook in the four null-terminated strings. This illustrates two possible areas for confusion.

1. If you define the data mapping incorrectly – the `NA` may well still work but when the correct arguments are passed it gives a `DOMAIN ERROR` (or maybe even GPFs).
2. If you define the data mapping incorrectly and pass the wrong arguments (that match the incorrect mapping) the function runs and returns data – but probably gobbledegook. This situation can be very tricky to debug!

A typical occurrence of (2) is when defining the mapping of integer datatypes – if you get the defined length of the integer wrong (double, long, unsigned?) then the results can be very confusing. As we have already seen the correct definition of `_lread` is:

```
NA 'U4 kernel.P16 | _lread I >U1[] U2'
```

Run correctly it returns (*fh* is a file handle tied to *c:\windows\win.ini*):

```
2>_lread fh 10 10
91 119 105 110 100 111 119 115 93 13
asc[1+91 119 105 110 100 111 119 115 93 13]
[windows]
```

But if defined erroneously as (*U2* rather than *U1*):

```
ⓘNA'U4 kernel.P16|_lread I >U2[] U2'
2>_lread fh 10 10
30555 28265 28516 29559 3421 30556 28265 26926 26990 0
```

you get data that is far too large to be ASCII! This problem can also be very tricky to debug. Of course to C programmers this probably all seems quite obvious – but to an APLer who typically only has to distinguish between integers and characters this can get most frustrating. A similar problem is when you define datatypes incorrectly so that the memory they take up is not as large as the function requires – the chances are the function will then write all over unreserved memory and the program will GPF.

Hopefully Dyalog can learn from PLUS III and provide a way of pre-defining the more common calls. One approach might be a utility workspace, allowing you to copy them in and learn from them if you need to add your own.

Overall

Apologies to people who found the overhead projection equipment inadequate – next time we will make sure the presentation facilities are better – also apologies to those who were expecting more on the DDE side – in the end we had very little prepared on the topic – but we hope to cover it very thoroughly in the next workshop. Well done to all those who completed all the exercises!

We learnt a lot from the day and the Committee will be laying down guidelines for events in the future. We need places with good PC facilities and presentation equipment; if anyone has any suggestions please let me know. We would be particularly interested in facilities in the Birmingham area.

We have also received details about the BCS's *IDPM Continuous Professional Development Programme*. For a small overhead we could register our workshops for you to gain points towards your own development. If anyone is interested in gaining IDPM credits by attending APL workshops, then please contact me (see inside back cover for address details).



by Sylvia Camacho

In *Chaotic Behaviour Revisited* (see Vector 11.2 p82) Gérard Langlet refers to the Verhulst or logistic equation which is commonly expressed:

$$x_{n+1} = kx_n(1-x_n) \text{ where } 0 \leq k \leq 4$$

Note that Langlet only quotes the equation with $k = 4$.

If $y = kx(1-x)$ is plotted for values of x between 0 and 1, the graph is a parabola of a steepness determined by the value of k . If, on the other hand, one creates a vector of results by iterating the calculation for a given value of k and an initial value for x_n between 0 and 1; the graph of the result vector will reduce to zero, will oscillate with varying amplitude around an average or will be apparently chaotic dependent upon the value of k . The force of Gérard's paper is that iterated results are unreliable because the precise value calculated when multiplying the two fractional numbers x and $1-x$ will depend upon the number of places to which the results are worked. Any x may result in a fraction which cannot be represented in a finite number of digits but, more importantly, a computer using IEEE standard floating point arithmetic will lose significance after about the fiftieth iteration. As a consequence an expression which mathematically always gives a result of 1, if iterated on a computer, may appear chaotic and Gérard provides an example for calculating a determinant to demonstrate his point.

Gérard does not make explicit the contribution that decimal to binary conversion may make to this problem but it is easy to demonstrate by setting Quad PP to 17 (normally the maximum precision) and then entering:

```
7×0.5×0.2
0.5×7×0.2
0.2×7×0.5
```

which with Sharp (APLIWIN) gives three different answers. This seems to suggest that, for Sharp, multiplication has ceased to be associative. With APL*PLUS v8, on the other hand, all three answers are the same although different from any given by APLIWIN. This, dare we call it *peccadillo*, of APLIWIN makes a very convenient laboratory to test the consequences of a difference in the 15th place after the point. I have set up functions to test this

using $k=4$ and a starting value of $x=0.51$. The difference between the two result vectors exceeds the default comparison tolerance after 6 iterations and has reached 5% at iteration 50. Within 100 iterations there are examples where function *logistic* gives $\text{iteration}[n] > \text{iteration}[n+1]$ but function *logistic2* gives $\text{iteration}[n] < \text{iteration}[n+1]$. These discrepancies are completely eliminated if the result of each iteration is rounded to 13 places after the point before calculating the next generation.

Are we seeing here the spoor of a real dragon or is Gérard just frightening us with speculative doodles in the less well explored parts of the computer-world map? Donald McIntyre has also crossed this territory and he reports feeling hot dragon-breath (see *The Perils of Subtraction* in this issue). Donald shows that iterations of $1-x$ will become unreliable earlier than Gérard suggests.

The Devil is in the Detail

Chaos Theory, and in particular the Mandelbrot set, by showing that the 'image' is enlargeable to an unlimited extent while remaining self-similar, shows the absurdity of the continuum. It is plainly not true that the real world contains objects with the complexity of, say, the Earth of a size $10^{-1,000,000}$ of the diameter of an atom. If the continuum is an inadequate model of the real world and our notation implies a continuum then our notation must be inadequate even though it is marvellously convenient for practical purposes. Mathematicians and physicists will have to re-appraise (and are already doing so) the limitations of the concept of a space-time continuum and the notation used to describe it. If new models are required of a granular space-time then, as Penrose suggests, integer arithmetic will be needed and with integer arithmetic it will be possible to program computers to calculate everything to full accuracy.

"The normal picture that one has is that between any two points one can find others and that one can go on subdividing ad infinitum.....When you think of it this is really an absurd idea.....So we have this complex continuum arising in quantum mechanics and we have the real continuum arising in the structure of space-time.....if we are going to get rid of it in the one place we have to get rid of it in the other. So I tried to develop a theory where one builds up the idea of space and of quantum mechanics simultaneously, starting from combinatorial ideas — pure counting ideas to begin with. I always have had the feeling that counting and other combinatorial concepts are more likely to lie at the root of physics than concepts which depend on the idea of the continuum."

Roger Penrose in Buckley & Peat, "*A Question of Physics*" pp42 & 43, Routledge & Kegan Paul 1979.

Langlet, of course, wishes to take this argument much further and to use Integer Modulo-2 Algebra (see *Binary Algebra Workshop Foils* in Vector 11.2). Gérard has been developing this approach for well over a decade and is producing very significant results, only a small part of which are seen by the APL community. Do not be deceived by his mischievous bilingual puns; he is out there in strange country trying to charm the dragons (see Vector 11.2 Editorial *Monsieur Langlet's Enterprise*).

The complacency with which mathematicians and physicists accept the limitations of number representation ought to be a concern to anyone using iterative algorithms. For instance, I have a textbook for final year and graduate students published in 1992 by Cambridge UP entitled *Nonlinear Systems* by P G Dazin. Not being a mathematician, I do not understand most of its 300 pages but I am disturbed by Chapter 3, *Difference Equations*, which has under the heading *Numerical and Computational Methods* the following:

"Most computers are digital rather than analogue, using discrete rather than continuous variables. So no fixed or floating point variable, however accurate the computer is, can represent a real variable exactly. This leads to round-off errors."

This is followed by a few lines of BASIC to calculate and plot, on a y axis of 1000 ticks, successive values of $x = a * x * (1 - x)$. There is no code to retain the values of x calculated and the first 65 values are not plotted, although they are converted for output to a loudspeaker (audible chaos?). Presumably the later values are felt to be more spectacularly chaotic. If Gérard and Donald McIntyre are right they are anyway well beyond the reliably computable range. The suggestion that an analogue might be preferable to a digital computer in order to avoid *truncation errors* is evidence that the author assumes that an analogue computer, constructed of non-infinitely-divisible materials, can emulate an infinitely divisible space or time; ignoring the fact that infinite, infinitesimal and irrational numbers are mathematical abstractions which cannot participate in any algorithm which is required to produce a numerical value expressible in a finite number of digits.

Teasingly, Langlet has chosen to present his paper as a paradox by introducing the logistic equation as *non-linear* and then recasting it in a form which he maintains is obviously *linear*. He often makes a play on words but this is a play on mathematics. Consider the following definitions taken from James & James *Mathematics Dictionary*, ed 5 1992, Van Nostrand Reinhold NY:

"**LINEAR**, *adj.* (1) In a straight line. (2) Along or pertaining to a curve. (3) Having only one dimension."

"linear equation or expression. An algebraic equation or expression which is of the first degree in its variable (or variables);"

"DIMENSION, *n.* Refers to those properties called length, area and volume...."

Well, well; this is a dragon of a different colour. None of the graphs or expressions we are talking about are linear by definition (1). The graph of the parabola is linear by definition (2) and, by definition (3) *et al*, the mathematical expressions used for the logistic equation can slither (remember dragons used to be called Worms) between *linear* and *non-linear* by a suitable application of equivalences. Interestingly the textbook from which I took the quotation about analogue computers also quotes the equivalences used by Langlet for the case of $k=4$ and refers to a paper by von Neumann way back in 1951 where he points out that the case $k=4$ is rather special because it enables us to 'solve' the difference equation (*Collected Works 5* (1961) ed A H Taub, pp768-70, Oxford: Pergamon). However, for a much more penetrating analysis of the issues surrounding this topic see the paper by Joseph Ford, *What is chaos, that we should be mindful of it?* in *The New Physics* ed Paul Davies, Cambridge University Press, 1989.

The logistic equation, when iterated, is an example of *feedback* and the way that this is affected by a constant multiplier. It is of interest because it demonstrates that, at certain values of the multiplier, no equilibrium is established: there is an *attractor* around which the values *hunt* but always in vain. To avoid the logical problems that arise from loss of precision the equation ought to be studied in integral form. $1-x$ must be expressible as an integer and so the result of $kx(1-x)$ *must be rounded*. The equation will only apply to non-continuous phenomena: any result requiring that a woman give birth to 3.4 children is not acceptable; maybe it was 3 maybe it was 4 but, as Gérard said at APL 94, "*Nature always decides*". The consequence is that the hunt for the attractor will be different for every different population size as well as being sensitively dependent upon the constant and the initial value of x .

I can remember as a child being impressed when I was told, "a difference in size may be a difference in kind". For instance a set of 22 people, if appropriately fit and skilled, is compatible with a game of hockey. Three people are not. Any large number of people is compatible with simultaneous games of hockey: providing the number is a multiple of 22. All numbers have a unique factorisation and for some purposes that may be a crucial *quality*.

Suppose we want to represent a no speed limit sign or a circle-bar with people. Take our 22 players and arrange them in a circle. What number of extra people do we need to draw the bar to the same 'point-density'? Well now, what a nice number 7 would be: for $22 \div 7$ is 3.142 and that is as near to π as the number in the

player population will allow. In this case the problem is easily solved with very little inaccuracy introduced by the integers. It is important to remember as we play with the logistic equation and others connected with Chaos Theory that, however seductive the patterns may be, some applications of them are limited to integers. For example predator-prey population studies. In Chaos Theory we are investigating the *qualities* of numbers and only numbers.

The trouble with people is that they want to work in decimal notation and the trouble with computers is that they are fundamentally binary. Both notations lose precision when handling fractions and conversion between the notations exacerbates the problems. That is why Gérard is drawn to a Boolean notation to implement an algebraic calculus. That is why Integer Modulo-2 Algebra shows such promise as a way to model, without loss of information, a world full of non-continuous phenomena.

```

□io←0
□pp←17

7×0.5×0.2
0.69999999999999994

0.5×7×0.2
0.69999999999999996

0.2×7×0.5
0.69999999999999998

Vlogistic[□]V
V r←k logistic vec
[1] A vec is initial value, number of iterations
[2] r←,vec[0]
[3] l1:→(vec[1]=pr)↑0
[4] r←r,k×(¬1↑r)×1-¬1↑r
[5] →l1
V

Vlogistic2[□]V
V r←k logistic2 vec
[1] A vec is initial value, number of iterations
[2] r←,vec[0]
[3] l1:→(vec[1]=pr)↑0
[4] r←r,(¬1↑r)×k×1-¬1↑r
[5] →l1
V
```

```

      rounded[[]]
v r+k rounded vec;s
[1]  A as logistic but
[2]  A rounds each generation assuming
[3]  A a maximum population of one million
[4]  r+,vec[0]
[5]  l1:=(vec[1]=pr)+0
[6]  s+k*(-1+r)*1-1+r
[7]  r+r, 9 7 s
[8]  +l1
v

```

```

      rounded2[[]]
v r+k rounded2 vec;s
[1]  A as logistic but
[2]  A rounds each generation assuming
[3]  A a maximum population of one million
[4]  r+,vec[0]
[5]  l1:=(vec[1]=pr)+0
[6]  s+(-1+r)*k*1-1+r
[7]  r+r, 9 7 s
[8]  +l1
v

```

```

      integer[[]]
v r+c integer vec;pop;k
[1]  A logistic but calculating in integer
[2]  A assuming maximum population of approximately
[3]  A one million or less
[4]  A c is constant for logistic equation
[5]  A vec is 0<seed population<1,
[6]  A 2*n which is maximum population size &
[7]  A number of iterations
[8]  r+,[vec[0]*2+vec[1]
[9]  k+ 7 3 c*128
[10] l1:=(vec[2]=pr)+end
[11] pop+k*vec[1] nextgen -1+r
[12] r+r,[pop÷128*2+vec[1]
[13] +l1
[14] end:
v

```

```

      nextgen[[]]
v r+n nextgen pop
[1]  A assumes maximum population of 2*n
[2]  A n+20 will give max pop of 1048576
[3]  A returns max pop less current pop
[4]  r+pop*(2*n)-pop
v

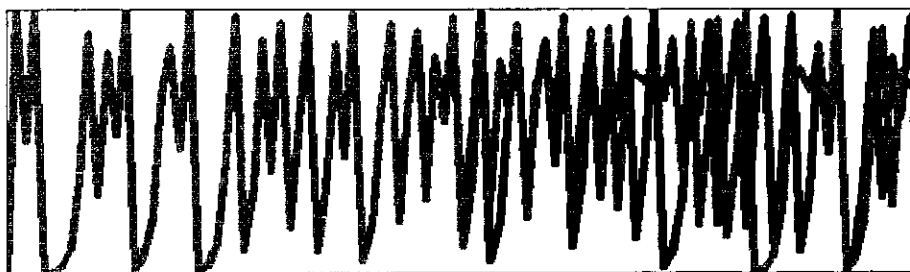
```

```

      v hicjacet [[]] v
    v r+c hicjacet vec
[1] r+c*((10vec)*2)*((20vec)*2
    v

```

logn



The above graph shows vector *logm* which was created using the function *logistic* overlaid by vector *logn* created using the function *logistic2* both with $k=4$ and starting value 0.51. Each vector is catenated to 0,1 in order to scale the vertical axis. The display function is APLWIN utility function *gfx* ' '. The first fifty or so values are sufficiently similar for the colour of the second vector to overlay the first completely, thereafter there is a substantial discrepancy.

Teaching with Executable Notation

Chapter 1: Introduction

by Kenneth E. Iverson

This paper concerns the use in teaching of a mathematical notation (or *language*) that is not only effective in analysis and exposition, but is also directly executable on small computers. Because the language is executable, the computer can be used to explore and elucidate topics with a clarity that can only be appreciated from direct experience of its use.

One mark of an effective teacher of mathematics (or of the use of mathematics in related fields) is a propensity to formulate any problem in mathematical language, and to conduct further analysis and exposition in that language. The ability to do this requires mastery of the language of an order different from that required for passive reading, or even from writing (or developing procedures or programs) in a specific topic.

Such mastery is gained from repeated attempts to use the language in this way in a variety of topics, and from digressions to explore the language itself under the stimulus of new problems. This suggests that anyone wishing to use a new language in exposition might approach learning it in a manner different from that appropriate to the more casual reader or writer.

This paper is an attempt to provide such an introduction to the executable mathematical language J. It proceeds by developing some topic in the language; then digresses to explore some new language entity encountered, using it in one or more topics, and thereby returning again to the thread of some topic.

It is organized by sections that begin by using the language to explore some thread, and includes series of exercises. The reader is urged to treat these exercises in four passes:

1. Study the given sentences and results.
2. Enter the sentences on the computer and experiment with related sentences that might throw light on the matter.
3. Study the J dictionary [1] or other references suggested in the questions.
4. Read the answers and discussions provided in Appendix A.

A. The Elements of J

The examples in this section introduce all major aspects of J, that is, all of the parts of speech:

```

10+1.5
11.5
  price=.10      NB. price is 10 (use of copula)
  markup=. 1.5
  price+markup   NB. Verb with pronouns
11.5
  prices=. 10      NB. Collective noun
  markups=. 1.5 1 3.75
  p=. prices [ m=. markups
  p+m           NB. Convenience of short names
11.5 7 28.75
+/m
6.25
  sum=. +/
  sum m
6.25
  p^3
1000 216 15625
  cube=. ^&3
  cube p
1000 216 15625
  (10^2)+(3^3)   NB. Use of parentheses
127
  x=. 0 1 2 3 4
  (1+(2*x)+(x^2)) = ((x+1)^2) NB. Identity
1 1 1 1 1
  pi=. 3.14159

```

A.1 Explain the use of the abbreviation NB.

A.2 The grammatical terms (such as *pronoun*, *verb*, and *part of speech*) used here are from English. State the corresponding terms used in maths.

A.3 State the effect of the slash (as in $+/m$). Also state the name given to such an entity in maths, and suggest a corresponding term suitable to J.

A.4 Repeat Ex. A.3 for $\&$.

A.5 Repeat Ex. A.3 for the parentheses.

A.6 State the purpose of $[]$.

B. Elementary Functions

The phrase *elementary functions* is commonly used to refer to a loosely-defined collection of functions that includes at least the trigonometric functions, the exponential, the square, and the square root. They are *elementary* in two senses:

They are rather easily defined (or approximated) in terms of addition, multiplication, and integer powers.

They serve as "building blocks" to define a wide variety of functions and computational procedures needed in applied mathematics.

These elementary functions are all both *monadic* (applying to a single argument) and *scalar* (applying to a single element). We will extend the notion in two ways. Firstly, because of the convenience provided by collectives (lists, tables, and higher-rank arrays), we will include verbs that generate and manipulate arrays:

```

a=. i. 4 [ b=. i. 3 4
square=. ^&2
sqrt=. ^&0.5
square a
0 1 4 9
sqrt b
0      1 1.41421 1.73205
2 2.23607 2.44949 2.64575
2.82843 3 3.16228 3.31662
exp=. 2.71828&^
exp a
1 2.71828 7.38905 20.0855
sine=. 1&o.
cos=. 2&o.
((square@sine) + (square@cos)) a
1 1 1 1
|. b
|: b

```

- B.1 Annotate the preceding sentences, stating clearly what each of the verbs do. Note that the results of the last two sentences are not shown, but may be obtained by entering all on a computer.
- B.2 Experiment with other members of the family of verbs obtained by bonding left arguments (such as 0 through 7) to the dyad o. and state the function performed by each. Also state the mnemonic scheme used in assigning the "codes" used in the left arguments.

Secondly, we include dyadic functions (such as addition, subtraction, multiplication, and division) that are also essential building blocks. Just as addition and multiplication tables prove useful in displaying interesting

properties of those functions in elementary arithmetic, so do corresponding verb tables illuminate more general dyadic functions. In what follows, some of the results are not shown, but can be obtained by using the computer:

```
a=. 2 3 5 [ b=. 0 1 2 3
a */ b
0 2 4 6
0 3 6 9
0 5 10 15
b %/ b
0 0 0 0
_ 1 0.5 0.333333 NB. The symbol _ denotes infinity
_ 2 1 0.666667
_ 3 1.5 1
b !/ b
1 1 1 1
0 1 2 3
0 0 1 3
0 0 0 1
over=. ({.;).)@":@,
by=. ' '&;@,@[,.]
```

a by b over a */ b

	0	1	2	3
2	0	2	4	6
3	0	3	6	9
5	0	5	10	15

tba=. 1 : '[by]over x. /'

a * tba b

	0	1	2	3
2	0	2	4	6
3	0	3	6	9
5	0	5	10	15

```
t=. tba
b ! b
(i.10) !t (i.10)
b -t b
b ^tb
```

B.3 Annotate the preceding sentences.

B.4 Enter `s=. ( i. 9) - 4` to obtain a list of integers symmetric about zero, and use it instead of `a` and `b` in the preceding sentences to explore the resulting verb tables.

B.5 Use the format function in expressions such as `8.3 " : b%/b` to produce a more readable division table.

A verb applied to two arguments is said to be *dyadic* or to have *valence 2*, and a verb applied to a single argument is *monadic* with valence 1. The minus sign used in mathematics is *ambivalent*, representing the dyad *subtraction* when supplied with two arguments, and the monad *negation* when supplied with one.

In J such ambivalence is extended to all verbs, the applied valence of any verb being determined by context. For example:

```

x=. 2 3 5 7
y=. 0 1 2 3
x-y
2 2 3 4
-y
0 _1 _2 _3
x%y
_3 2.5 2.33333
%y
_ 1 0.5 0.333333
x^y
1 3 25 343
^y
1 2.71828 7.38906 20.0855
+/y
6
x+/y
2 3 4 5
3 4 5 6
5 6 7 8
7 8 9 10
3|.y
3 0 1 2
|.y
3 2 1 0
x<y
0 0 0 0
<y

```

0	1	2	3
---	---	---	---

(< 1 0 2 |: i.2 3 4), (< |: i.2 3 4)

0	1	2	3	0	12
12	13	14	15	4	16
				8	20
4	5	6	7		
16	17	18	19	1	13
				5	17
8	9	10	11	9	21
20	21	22	23		
				2	14
				6	18
				10	22
				3	15
				7	19
				11	23

B.6 Annotate the preceding sentences.

B.7 Explore relations between other monads and dyads.

C. Tables and Rank

The verb tables of the preceding section are said to be of rank two, and to have two axes. Each row of such a table is a *list* (of rank 1), and each element of the list is an *atom* (of rank 0). More generally, an array of higher rank (such as the rank 4 array produced by i. 2 3 4 5) "contains" *cells* of lower rank.

The rank conjunction (denoted by ") constrains a verb to apply to each cell of specified rank. For example:

```

]a=. i. 2 3 4
0 1 2 3
4 5 6 7
8 9 10 11

12 14 16 18
20 22 24 26
28 30 32 34

12 13 14 15
16 17 18 19
20 21 22 23
+/"1 a
6 22 38
54 70 86
mean=. +/ % #
mean 2 3 4 5
3.5
+/"2 a
12 15 18 21
48 51 54 57

```

```
(mean a);(mean"2 a);(mean"1 a)
```

6	7	8	9	4	5	6	7	1.5	5.5	9.5
10	11	12	13	16	17	18	19	13.5	17.5	21.5
14	15	16	17							

Functions of ranks 0 and 1 are also called *scalar* and *vector* functions, respectively.

C.1 Annotate the preceding sentences.

C.2 The monad , ravel's its argument. Experiment with ravel's of various ranks.

C.3 Experiment with box (<) of various ranks.

The rank conjunction can be used to specify the ranks of each argument of a dyad. For example:

```
]b=. 0{a
0 1 2 3
4 5 6 7
8 9 10 11
  3 4 5 *"0 1 b
0 3 6 9
16 20 24 28
40 45 50 55
  2 3 5 7 *"1 1 b
0 3 10 21
8 15 30 49
16 27 50 77
]c=. 3 4 $('abcdefghijkl')
abcd
efgh
ijkl
  'ABC' , "0 1 c
Aabcd
Befgh
Cijkl
```

C.4 Annotate the preceding sentences.

C.5 Consult the definition of the rank conjunction in the dictionary, and experiment with further uses of it.

D. Writing

Anyone wishing to use a new language well should make early attempts at writing on some topic of interest. Because of limited knowledge of the language this may prove frustrating and seemingly fruitless. Nevertheless, it should be

attempted: even failed attempts will alert a student to facilities in the language whose significance would otherwise be missed.

For example, a student attempting to produce subtotals (partial sums) in a treatment of accounting might happen upon the prefix adverb:

```

receipts=. 3 1 4 2 7 6
sum=. +/
sum receipts
23
sum \ receipts
3 4 8 10 17 23
scan=. \
max=. >./
max scan receipts
3 3 4 4 7 7

< scan receipts

```

3	3	1	3	1	4	3	1	4	2	3	1	4	2	7	3	1	4	2	7	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

As a consequence the student would be alerted to, and perhaps explore, related partitions such as \. and /.. Later attempts at treating polynomials might lead to the table $c*/d$ for the product of all pairs of elements of the coefficient lists c and d as a first step in computing the coefficients of the product polynomial, and finally, to the need for diagonal sums of the table:

```

c=. 1 2 1 [ d=. 1 3 3 1
c*/d
1 3 3 1
2 6 6 2
1 3 3 1
sum/.c*/d
1 5 10 10 5 1

</.c*/d

```

1	3	2	3	6	1	1	6	3	2	3	1
---	---	---	---	---	---	---	---	---	---	---	---

Beginning attempts at writing should begin at the beginning of a topic, not *in medias res*. Moreover, it is well to choose topics which themselves enjoy wide application, For example:

Calculus
 Difference Calculus
 Linear Functions
 Polynomials

Classification and sets
 Directed Graphs
 Permutations

Introductory treatments of these topics are presented in the *Dictionary of J*.

D.1 Annotate the J sentences in this section.

E. Properties of Functions

The terms *distributivity*, *commutativity*, and *associativity* are used in maths to describe properties of scalar dyadic functions. We will use them in the same way, but will extend the notion of distributivity to the distribution of a *monadic* function over a dyad, and will introduce the terms *uniformity* and *symmetry* to cover important properties of functions of higher rank.

It is commonly said that the dyad $\star$ distributes over addition because:

$$\begin{aligned}(a \star (b+c)) &= ((a \star b) + (a \star c)) \\ \text{or, equivalently} \\ ((a+b) \star c) &= ((a \star c) + (b \star c))\end{aligned}$$

This relation might also be expressed by saying that the *monad* $a\&\star$ (or the monad $\star\&c$) distributes over addition:

$$\begin{aligned}(a\&\star (b+c)) &= ((a\&\star b) + (a\&\star c)) \\ (\star\&c (a+b)) &= ((\star\&c a) + (\star\&c b))\end{aligned}$$

In the case of a non-commutative dyad such as division ($\%$), the statements in terms of the monads $a\&\%$ and $\%&c$ are to be preferred over a statement about the dyad $\%$, since one of them distributes over addition and the other does not.

E.1 Enter

```
a=. 3 1 2 4 [ b=. 2 3 5 7 [ c=. 4 3 2 1
```

and experiment to determine the distributivity over both addition and subtraction of the following monads:

```
a&+ *&a a&% %&a
a&- -&a -: +: *: %:
```

E.2 Test the distributivity of the incrementation $a\&+$ over maximum ($>.$) and the distributivity of the minimum $a\&<.$ over maximum. The commute or cross adverb $\sim$ reverses the sense of its verb argument. For example:

```
3 % 5
0.6
5 %~ 3
0.6
into=.%~
5 into 3
0.6
```

Consequently, a test for the commutativity of a function f may be written as $f=f\sim$. For example:

```

      3 4 5 (+=+~) 5 4 3
1 1 1
      3 4 5 (into=into~) 5 4 3
0 1 0

```

E.3 State a characteristic common to the verb tables of all commutative functions.

E.4 Enter the following sentences, and comment on the differing behaviours of the function + (which is associative), and - (which is not):

```

b=. 1 0 0 1 0 1 0 0
x=. 1 2 3 4 5 6 7 8
b <; .1 x

```

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

```
(b +/; .1 x); (+/b+/; .1 x); (+/x)
```

6	9	21	36	36
---	---	----	----	----

```
(b -/; .1 x); (-/b-/; .1 x); (-/x)
```

2	_1	7	10	_4
---	----	---	----	----

A function f is *uniform* if the shape of its result agrees with the shape of its argument, i.e., if $g=. \$ - : \$@f$ is a tautology (true for any argument).

A monadic vector function is said to be symmetric if it yields the same result when applied to any permutation of its argument. For example:

```

]a=. 3 A. b=. 2 3 5 7 11
2 3 7 11 5
(+/a); (+/b)

```

28	28
----	----

```
(-/a); (-/b)
```

0	8
---	---

The final result illustrates the fact that -/ (the alternating sum) is not symmetric. A monad $f/$ is symmetric if and only if the dyad is both associative and commutative.

Chapter 2 of *Arithmetic* [2] provides discussion of further properties of verbs, including inverses and identity elements. Make and experiment with expressions to illustrate these notions.

E.5 Enter the following sentences, and comment on the results:

```
+: b=. 1 2 3 4
+: ^: 2 b
+: ^: 0 1 2 3 b
+: ^: _2 _1 0 1 2 b
```

E.6 Enter the following sentences and comment:

```
+: ^: _1
^^: _1
inv=. ^: _1
^inv
10&^. inv
^&4 inv
% . inv
```

E.7 Enter and comment upon:

```
+: b. _1
-:
%. b. _1
%.
+: b. 0
_ 0 0
%. b. 0
2 _ 2
```

F. Proofs

The results of:

```
] odds=. 1+2*i.n=. 10
1 3 5 7 9 11 13 15 17 19
+/odds
100
n^2
100
```

might suggest that the sum of the first n odd numbers equals the square of n , and the following use of the prefix scan adverb would tend to confirm the conjecture:

```

+/\odds
1 4 9 16 25 36 49 64 81 100
(1+i.n)^2
1 4 9 16 25 36 49 64 81 100

```

Insight into *why* the relation holds for any n can be provided by a sequence of equivalent sentences that differ in simple steps:

```

+/\odds
100
+/\|.odds
100
+/(odds+|.odds)%2
100
+/(n # 2*n)%2
100
+/n#n
100
n*n
100

```

Such a sequence constitutes an informal proof that the last sentence is equivalent to the first, a proof that can be made formal by annotations that state *why* each sentence is equivalent to the one that precedes it. For example:

+/\odds	
+/\ .odds	Summation is symmetric
+/(odds+ .odds)%2	Sum of equals divided by 2
+/(n # 2*n)%2	n copies of the sum $2*n$
+/\n#n	$2*n$ divided by 2 is n
n*n	Definition of times in terms of addition

Inductive proofs may be written similarly, by stating and using an inductive hypothesis. For example:

$(+/1+2*1.n)=(n^2)$ Inductive hypothesis

The assumption that the hypothesis is true for some value of n is first used to prove that it must then be true for $n+1$. Finally, it is shown to be true for some value of n ; in the present instance, when $n=.1$ the value of $+/1$ clearly equals 1^2 . Thus:

$+/1+2*i.n+1$	
$(+/1+2*1.n)+(1+2*n)$	Addition is associative
$(n^2)+(1+2*n)$	Induction hypothesis
$(n^2)+(2*n)+1$	Addition is commutative
$(n+1)^2$	Expansion of square

F.1 Enter the following sentences and prove the identity that they suggest:

```
a=. 2 3 5 [ b=. 3 1 4 1
p=. 4 3 2 [ q=. 2 7 1 8
(a*p) */ (b*q)
(a*/b) * (p*/q)
```

We conclude with a somewhat longer proof of the associativity of the matrix product. The matrix product is a collection of sums of products, and we begin with its definition in terms of the dot conjunction (which provides a range of useful "dot products" based on functions other than sum and product):

```
mp=. +/ . *
```

The dot conjunction is itself defined (in [1]) as follows:

```
u . v is defined by u@(v"(1+1v),_))
```

For the present case of ordinary matrix product on matrices, v is $*$, and (since its left rank $1v$ is zero) this simplifies to $u@(*"1 _)$, thus applying multiplication by each row vector of the left argument to the entire right argument. The effect is to weight each row of the right argument by the corresponding element of a given row of the left argument before applying summation ($u=. +/$) over the weighted result. For example:

```
t=. *"1 _
s=. +/"2
q=. 4+r=. i. 2 2
q;r:(q t r);(s q t r);(q mp r)
```

4	5	0	1	0	4	10	19	10	19
6	7	2	3	10	15	14	27	14	27
				0	6				
				14	21				

We will first illustrate the fact that mp is associative:

```
p=. 4+q
p;q;r:((p mp q) mp r); (p mp (q mp r))
```

8	9	4	5	0	1	206	395	206	395
10	11	6	7	2	3	254	487	254	487

and then show a sequence of simple transitions from $(p \text{ mp } q) \text{ mp } r$ to $p \text{ mp } (q \text{ mp } r)$, displaying the explicit results for only the first two sentences:

```

      (pmp q) mp r
206 395
254 487
      (s p t q) mp r
206 395
254 487
      s (s p t q) t r
      s s (p t q) t
      s s p t (q t r)
      s p t (s q t r)
      s p t (q mp r)
      p mp (q mp r)

```

F.2 Annotate the foregoing informal proof.

G. Plan of Study

This introduction should be supplemented by treatments of various topics in the manner illustrated for linear functions in the succeeding chapter.

Because the main purpose is to learn to use executable notation in exposition, the reader is urged to approach each topic in the following steps:

- Read the English language statement of the matter to be addressed.
- Before reading further, attempt to treat the matter in whatever language is most congenial; if this is not J, then translate your work to J and try it on the computer; if it is J, first experiment with the result and then translate to conventional maths to better appreciate notational questions.
- Read the further text and associated exercises and answers, and compare them with your own work.

[Ed: Chapter 2 to follow in Vector 12.1.]

Appendix A: Answers to Chapter 1

A.1 The abbreviation NB. (nota bene) precedes a comment, which is ignored in J.

A.2 In maths a pronoun such as x is called a *variable* to emphasize that in a sentence that expresses an identity or other relation the relation holds for *any* value assigned to x , that is, when x changes or *varies*.

This terminology is used even when it is clear that the name (such as $p1$ in the final sentence) is *not* expected to vary but to remain *constant*. The term *variable* is particularly inappropriate for a name such as *sum* used as a *proverb*.

In maths the terms *function* (from the past participle of the Latin *fungi*, to perform) and *operator* are used in the sense that *verb* is used in J.

Since formal grammatical rules are not imposed on sentences in maths, there appears to be no analogue to the phrase *part of speech*.

A.3 The slash applies to a verb (such as $+$ or $*$) to produce a related verb (such as *sum* or *product over*), and therefore acts like an *adverb* in English. In maths it would be called a *functional* or *operator* (a usage that conflicts with the use of *operator* as a synonym for *function*).

A.4 The $\&$ is an example of a *conjunction*. In the phrase $\wedge \& 3$ it “joins” the verb $\wedge$ (power) to the right argument 3 to produce the verb to be called *cube*.

In a phrase such as $\& |$ it acts like the copulative conjunction *and* in English to define a verb as a sequence of actions (as in *run and hide*), in this case *subtraction* of the *magnitudes* of the arguments. Enter the following sentences on the computer:

```
a=. 1 _2 3 _4
b=. _10 _20 30 40
(|a)-(|b)
_9 _18 _27 _36
a _&| b
_9 _18 _27 _36
```

A.5 Parentheses are used in J as they are in maths, to prescribe the order of execution of a sentence; they are analogous to *punctuation* in English. See the discussion of punctuation on page 5 of the dictionary of J. Note particularly the discussion of the fork, and experiment with its use.

A.6 The verb denoted by the left bracket is called *left*, and yields its left argument. In its use above it serves to unite two sentences in a single sentence and might be pronounced as *and*, as in “ p is prices and m is markups”.

- B.1 The verb `l .` produces lists, tables, and higher-rank arrays of successive integers. The verb `l .` reverses the order of the *items* or *major cells* of an array, and `l :` (called *transpose*) reverses the order of the axes of its argument. See Section II.A of the dictionary.
- B.2 Find the circle function (denoted by `o .`) in the vocabulary on the back cover of the dictionary, and use the indicated page number to find the table of circle functions. Note that the odd functions (sine, tangent, hyperbolic sine, and hyperbolic tangent) are encoded by odd integers, and the even functions by even integers. If you are not familiar with the notion of odd and even functions, find the conjunctions *even* and *odd*, and consult their definitions.
- B.3 The verbs `* /` and `% /` provide multiplication and division tables. Note that the entries for division by zero are all infinite (denoted by `_`).

The *outof* function `!` yields binomial coefficients, and `b ! / b` therefore yields Pascal's triangle, completed by meaningful zeros.

The functions `by` and `over` provide a more readable display of verb tables. Their definitions must first be found (Look up `by` in the index of the dictionary) and entered in the computer. They should be treated as *utility* functions, for use rather than for immediate study.

The utility adverb `tba` is defined in terms of the verbs `by` and `over`, as shown on page 35 of the dictionary.

- B.4 Note the symmetry (or skew-symmetry) of some of the tables resulting from `s`, as well as the interesting division into quadrants. Note particularly the results of `0 ^ 0` and `0 % 0` and `0 ! 0`.

Also note the useful extension of `!` to negative arguments, an extension based on the gamma function.

- B.5 Although the result of the `format` function appears like a numeric table, it is actually a table of characters. Confirm this by applying `$` to determine its shape, and by trying to apply an arithmetic function (such as `*`) to it. Also use `do = . "` to produce a corresponding numeric table.
- B.6 For mnemonic reasons, the monadic and dyadic cases of verbs are chosen so that they are related in some way, or so that their functions are suggested by the symbol used. Thus:

<code>(-y)</code>	<code>(0-y)</code>	
<code>(%y)</code>	<code>(1%y)</code>	
<code>(^y)</code>	<code>(e^y)</code>	NB. Where <code>e = . ^1</code> is Euler's number
<code>+ / y</code>		NB. plus over y
<code>x + / y</code>		NB. plus over each pair
<code> . y</code>		NB. Reversal

3 .y	NB. Rotation
<y	NB. < squeezes into box
x<y	NB. Narrow end to smaller

- C.1 The *items* or *major cells* of an array are the cells of rank one less, and +/ "inserts" the function + between the items of the argument. Therefore +/ a adds the two 3 by 4 tables of a.

Since +/"2 applies to each of the rank 2 cells of a, it adds *their* items (that is, the rows) to produce a 2 by 4 result.

Similar remarks apply to the function mean. Since the semicolon boxes its left argument and catenates it with the right argument (first boxing it if it is simple) it can be used as illustrated to provide a compact display of several results.

- E.2 Use both positive and negative values for a and for the arguments of the dyads.
- E.3 They are symmetric about the diagonal, that is, t-: |:t.
- E.4 -/b -/ : . 1 x is equivalent to (1-2-3) - (4-5) - (6-7-8) which (because - is non-associative) does not equal -/x. Note that 1-2-3 is 1-(2-3). See Section IIE of the dictionary.
- E.5 These expressions illustrate the use of the power conjunction to provide repeated application of its argument (or of its inverse if the right argument of the conjunction is negative). The first two functions are uniform, but the others are not.
- E.6 Entry of a function alone (without an accompanying argument) displays the function itself, showing that the inverse of +: (double) is -: (halve), and that the inverse of ^ (exponential) is ^. (natural log). See §1:3 of Appendix E of the dictionary for alternative forms of display.
- E.7 The adverb b. yields certain basic properties of its argument, its inverse for the argument _1 and its ranks (in the order monadic, left, right) for the argument 0.
- F.1 See Section D of Chapter 5 of *Arithmetic* [2].

Perils of Subtraction: a New Language for an Old Algorithm

by Donald B. McIntyre

APL and the Classical Method for Computing Pi

The symbol π , π , was chosen in 1706 [12; 3] to represent the length of the *perimeter* of a circle with unit diameter. The classical way of obtaining its value was given by Archimedes about 240BC, making this one of the oldest known algorithms [1; 2; 13; 16; 17; 18]. An inscribed hexagon has six sides, each of length one half; consequently π is greater than 3, which is the perimeter of the hexagon. A circumscribed hexagon provides an upper bound.

Doubling the number of sides of the polygon gives a closer approximation, and successive doublings, with a larger and larger number of smaller and smaller sides, converges on the true value for the perimeter of the circle. The length of a new side, after doubling the number, is found by two applications of Pythagoras' theorem, which was well-known to Archimedes. If the length of the old chord is s , then the length of the new chord is given by:

$$\sqrt{\frac{1}{2}(1 - \sqrt{1 - s^2})}$$

The side of the 12-sided figure resulting from doubling is given in APL as follows:

```

Y← 0.5                               Side of the inscribed hexagon
(0.5 × 1 - (1 - Y*2)*0.5)*0.5
0.25889

▽ Z← CH Y                            Function defining the new chord length
[1] Z←(0.5×1-(1-Y*2)*0.5)*0.5
▽

CH CH 0.5                            Length of the new side after 2 doublings
0.130526

```

With 8 doublings the approximation to π is:

```

6 ×(2×2×2×2×2×2×2)× CH CH CH CH CH CH CH CH 0.5
3.14159

```

Using APL notation for exponentiation:

```

6 ×(2*8) × CH CH CH CH CH CH CH CH 0.5

```

We need an equivalent operation for repeated application of CH . Direct Definition (introduced to APL in 1976 [9; 14]) is concise, and leads naturally to Iverson's recent extension called J [10; 11; 15]. CH is written as follows. A colon separates the name of the function from the body, and reference to arguments is by the symbols α and ω :

```
CH:(0.5*1-(1- $\omega$ *2)*0.5)*0.5
```

The recursive function C applies CH to the right argument the number of times given by the left argument. The body, divided into three parts by two colons, should be read *execute the left-hand expression unless the proposition given in the middle expression is true, in which case execute the right-hand expression*:

```
C:( $\alpha$ -1) C CH  $\omega$  :  $\alpha$ =0 :  $\omega$ 
  2 C .5
0.130526
```

After eight doublings the estimate of pi is:

```
PI:6 * (2* $\omega$ ) *  $\omega$  C .5
PI 8
3.14159
```

From APL to J

To convert from APL to J, replace three APL symbols by ASCII equivalents:

$Y \leftarrow 0.5$	In APL
$(0.5 \times 1 - (1 - Y \times 2) \times 0.5) \times 0.5$	
$y = . 0.5$	In J
$(0.5 \times 1 - (1 - y \wedge 2) \wedge 0.5) \wedge 0.5$	
$\%: 0.5 \times 1 - \%: 1 - y \wedge 2$	Rewrite without parentheses:
	$\%:$ is <i>square root</i>
$\%:-: 1 - \%: 1 - *: y$	$-:$ is <i>half</i> and $-.$ is <i>1-</i>
$\%:-: -.\%:-: *: y$	

Now define a verb (function) to compute the length of a new chord:

```
ch=. 3 : '%:-: -.\%:-: *: y.'
```

This is an *explicit* definition, because $y.$ represents the (right) argument. ch is a verb because 3 is the sum of possible valences (0 for a noun, 1 for an adverb, and 2 for a conjunction).

```
ch 0.5
0.258819
```

With a left argument of 13 this automatically translates to *tacit* definition:

```
13 : '%:-:~.%:-.*: γ.'
[: %: [: -: [: -: [: %: [: -: .*:
```

The result is a *train* of 11 verbs, parsed from right to left as a sequence of *forks*. In this case a tacit definition can also be written using the *atop* conjunction (@):

```
ch=. %:@-:@-.@%:@-.*:
```

One Verb Under Another

Before taking the next step, consider a simpler example. First of all straight execution:

```
5      %: +/ *: 3 4      The square root of the sum of squares (Pythagoras).
```

Using names helps us generalize. The conjunction [returns its left argument (a verb).

```
f=. %: [ g=. +/ [ h=. *:
f@g@h 3 4      Conjunctions compose a new verb
5
[] -: f@h) γ=. i.4      These match; i.e. f is the inverse of h.
1
g&.h 3 4      g under h; i.e. apply f to the result of g@h
5
```

Another example. Find the shape of each boxed item (> opens a box and < closes it):

```
(<@&$@> -: $&.>) 1 2 3; 1.2 4      Match. Shape under open.
```

%: is the inverse of *: , so we rewrite ch as follows — the parentheses are not needed

```
ch=. %: @(-:@-.@%:@-.)@ *:
ch=. (-:@-.@%:@-.)&.*:
ch=. -:@(-.@%:@-.)&.*:
```

As -: is its own inverse, write:

```
ch=. -:@(%:&.-.)&.*:      Parentheses are now required.
ch 0.5
0.258819
```

The Power Conjunction with a Gerund: a New Way for Iteration

An elegant feature of J is that, just as we write $(2\wedge y)$, so we can write $(ch\wedge y)$; thus:

```
pi=. 3 : '6*(2^y.)*(ch^:y.) 0.5'
pi 8
3.14159
```

The tacit form is more subtle: one verb ($v1$) gives the number of times ch is to be applied, while another ($v2$) gives the value of ch 's initial argument. The two are tied together to form a *gerund*. In this case $v2$ always returns 0.5.

```
ch^:(v1`v2) y  ↔  ch^:(v1 y) (v2 y)    The definition
g=. ]^(0.5"_ )  The rank conjunction converts a noun into a verb of given rank
pi=. 6: * 28^ * ch^:g
pi"0 1.9        The rank conjunction applies pi to the rank-0 cells of 1.9
3 3.10583 3.13263 3.13935 3.14103 3.14145 3.14156 3.14158 3.14159
```

But Now for the Problem!

Unfortunately the results of further doublings now become worse and worse [15, pp.563-564]. After 28 doublings — the number of sides is 1,610,612,736 — the computed value of π is zero!

```
pi"0 [ 24 25 26 27 28
3.18198 3.3541 4.24264 6 0
```

To understand what goes so wrong, we must know how numbers are stored and manipulated.

How Numbers are Stored in an IBM System/360 and its Descendants

Floating-point numbers have three parts: a normalized fraction; an exponent (characteristic); and a sign. In the IBM System/360 and its descendants the leftmost byte contains the sign (counting from the left, this is bit 0) and the exponent (bits 1-7).

The fraction (3 bytes in short form and 7 in long) is expressed in *hexadecimal digits*, each a group of 4 bits (a *nibble*). The radix point is to the left of the fraction, which is normalized by shifting it until the first significant hexadecimal digit occupies bits 8-11. Because normalization is by *hexadecimal* digits, bits 8-10 may all be zero.

In a decimal fraction, the value assigned to a digit is determined by the place-value of the digit's position. Immediately to the right of a decimal point we have tenths; so in a hexadecimal fraction that is where we have sixteenths. The value corresponding to a digit represented by bits 8-11 is illustrated by the following examples:

bit	8	9	10	11		
	0	0	0	1	1/16	
	0	0	1	0	2/16	1/8
	0	1	0	0	4/16	1/4
	1	0	0	0	8/16	1/2
	1	1	1	1	15/16	

Each time the fraction is shifted one hexadecimal digit right or left during normalization, the exponent changes to record the shift. The exponent is a 7-bit binary number. It is the power, n , to which the base (16) must be raised so that the product of the fraction and 16^n equals the absolute value of the number represented. If the exponent is 0, the values of the fraction are multiplied by $16^0 = 1$ and are unchanged. If the exponent is 1, the values of the fraction are multiplied by $16^1 = 16$ and become 1 2 4 8 and 15. If the exponent is 2, the values of the fraction are multiplied by $16^2 = 256$ and become 16 32 64 128 and 240.

In order to provide for negative exponents, 64 (binary 100 0000) is added to all exponents. This is called *excess-64* representation.

411c ↔ 0100 0001 0001 1100 ↔ $16^1 (1/16^1 + 12/16^2)$ ↔ 1.75

c21d5a ↔ 1100 0010 0001 1101 0101 1010

c21d5a ↔ $-(16^2) (1/16^1 + 13/16^2 + 5/16^3 + 10/16^4)$ ↔ -29.3516

$-(16^2) * (1 \ 13 \ 5 \ 10 \ + \ . \% \ 16^1 \ 2 \ 3 \ 4)$ Executing in J
_29.3516

Translate from hexadecimal floating-point representation to decimal:

```
dfh=. '0123456789abcdef'&i.      Decimal from hexadecimal
bfh=. [: , 2 2 2 2"_ #: dfh      Binary from hexadecimal
fs360=. _1: ^ {.@bfh              Floating-point 360 sign
g=. #.@}.@bfh@{2&{.)              Floating-point 360 exponent
fe360=. 16"_ ^ g - 64"_
f=. 16"_ ^ >:@i.@#
ff360=. (+/ .% f)@dfh@{2&{.) f.   Floating-point 360 fraction (Hook)
fp360=. fs360 * fe360 * ff360
```

It is convenient to convert trailing blanks to zeros. *zfb* uses *item amend* to convert any blanks to zeros. First laminate a row of 0s to make a table of two rows. Select from row 1 unless there is a blank, in which case select from row 0. Parentheses

are not needed; they merely draw the reader's attention to the verb to which the adverb *amend* () applies.

```
zfb=. [: (~:&' '@{:}) '0'&,:      Zeros from blanks
dfh360=. [: fp360 [: zfb 16&{:    Decimal from 360 hexadecimal
```

Because the blanks are trailing, the *fit* conjunction (1.) can control the padding:

```
pad=. 16&{(.!.'0')      !. modifies {. by specifying the fill element
dfh360=. fp360@pad
dfh360 'c21d5a'
_29.3516
```

To deal with a list of hexadecimal strings, append a blank character to define the *fret* and use the *cut* conjunction (;.); for example:

```
<:._2 '428 424 c21d5a '


|     |     |        |
|-----|-----|--------|
| 428 | 424 | c21d5a |
|-----|-----|--------|


]:._2 '428 424 c21d5a '
428
424
c21d5a
pad;._2@(&' ' )'428 424 c21d5a'
4280000000000000
4240000000000000
c21d5a0000000000
```

Incorporate this in the verb *dfh360* (parentheses are required):

```
dfh360=. .,@(fp360"1@table@pad;._2@(&' '))
dfh360=. [: , [: fp360"1 pad;._2@(&' ' )
dfh360 '41c 414 411 c11 411c c21d5a'
12 4 1 _1 1.75 _29.3516
```

How Numbers are Stored in a PC

On the PC, J uses the *double extended* form of 64 bits: the sign is bit 0, the exponent bits 1-11, and the fraction bits 12-63 [7]. Unlike 360 architecture, both exponent and fraction are binary. Because normalized binary numbers always begin with a 1, there is no need to store it. As usual, the exponent keeps a record of shifts made by the fraction, but in this case the count is binary. 1023 is added to allow for negative exponents.

```
<:2^10      #:1023
1023        1 1 1 1 1 1 1 1 1 1
```

Evaluation of the 64-bit binary string is therefore:

$$(_1 \wedge \text{sign}) * (2^{(2\#.\text{exp}) - 1023}) * 1 + (2\#.\text{fr}) \% 2^{\wedge} \# \text{fr}$$

s, e, and f extract sign, exponent, and fraction respectively from the binary representation.

```
s=. _1: ^ { . [ . e=. [: ]. 12&{ . [ . f=. 12"_ ]. ]
exp=. [: 2&^ #. - 1023"_      Note that the dyad #. is 2&#.
fr=. [: >: #. % 2&^@#
dfbj=. (s * exp@e * fr@f) f.   Decimal from binary: J on PC
dfhj=. dfbj@bfh               Decimal from hexadecimal representation
dfhj '400921f9'               Compare ir 0.1 below
3.14159
```

Internal representation is obtained by the *foreign conjunction* !: . When used as 3!:3, the result is given in hexadecimal notation. Because J, like APL, stores information about the *personality* of data along with the values, we drop the prefix (length is 4 + rank):

```
ir=. 3!:3 } .~ 4: + #@$      Internal representation
ir 0.25                      ]y=. ir %1 2 4
00000000                    00000000
0000d03f                    0000f03f
                                00000000
                                00000000
                                0000e03f
ir 0.1                       0000e03f
182d4454                    00000000
fb210940                    0000d03f
```

Each atom is now represented by a pair of rows. Isolate these with the *infix* adverb (\) and then ravel them. Here are examples of how *infix* works:

```
_2 <\ y
┌──────────┬──────────┬──────────┐
│00000000│00000000│00000000│
│0000f03f│0000e03f│0000d03f│
└──────────┴──────────┴──────────┘

hx=. (_2: ,\ ] )@ir
bi=. <"_1      Box items
bi 8 2$ , hx 0.25
┌──┬──┬──┬──┬──┬──┬──┬──┐
│00│00│00│00│00│00│d0│3f│
```

```
_2 ,\ y
00000000000000f03f
000000000000e03f
000000000000d03f
```

Although this is how bytes are stored, it is convenient to reverse the order:

```
! . bi 8 2$ , hx 0.25
┌──┬──┬──┬──┬──┬──┬──┬──┐
│3f│d0│00│00│00│00│00│00│
```



```

23 3cf1c00000000000 0000000000000039412917374193050000000 3.15981
24 3cd1ffffffffff 000000000000009992007221626406000000 3.18198
25 3cb40000000000001 000000000000002775557561562891000000 3.3541
26 3ca00000000000001 000000000000001110223024625156400000 4.24264
27 3c900000000000000 000000000000000555111512312578200000 6
28 0000000000000000 000000000000000000000000000000000000 0

```

The apparent precision of numbers like 5.551115123125782e_1 is very misleading.

```
5.551115123125782e_17 ↔ dfhj '3c9'
```

The fraction has lost ALL significant figures, and the result comes from the exponent alone. With 360 architecture the result would be zero, but the PC adds an implied 1 to the fraction before multiplying by the decoded exponent. At the next iteration, however, the exponent too is zero.

STSC APL*PLUS PC System 5.0

n	*: next chord	Pi
23	3.8857805861880482e_15	3.13748
24	9.9920072216264070e_16	3.18198
25	2.2204460492503132e_16	3.00000
26	5.5511151231257832e_17	3.00000
27	0	0

```

3.8857805861880482e_15 ↔ (16^_11)*1 1 8 +/ .% 16^1 2 3
9.9920072216264070e_16 ↔ (16^_12)* 4 8 +/ .% 16 ^1 2
2.2204460492503132e_16 ↔ (16^_12)* 1* 16^_1
5.5511151231257832e_17 ↔ (16^_13)* 4*16^_1

```

VS APL on an IBM 4341 Computer

(360 Architecture — includes a guard digit):

n	*: next chord	Hex	Digits	Pi
23	3.8927194800919541e_15	351188	4	3.1403
24	9.7144514654701180e_16	344600	2	3.1375
25	2.3592239273284575e_16	341100	2	3.0923
26	5.5511151231257810e_17	334000	1	3.0000
27	1.3877787807814456e_17	331000	1	3.0000
28	0	000000	0	0

```

3.8927194800919541e_15 ↔ (16^_11)*1 1 8 8+/ .%16^>:1.4
9.7144514654701180e_16 ↔ (16^_12)*4 6 +/ .% 16^1 2
2.3592239273284575e_16 ↔ (16^_12)*1 1 +/ .% 16^1 2
5.5511151231257810e_17 ↔ (16^_13)* 4%16^1
1.3877787807814456e_17 ↔ (16^_13)* 1%16^1

```

Acknowledgements

I dedicate this study to Professor Winifred Asprey, Emeritus Professor of Mathematics, Vassar College, in whose honour the Computer Centre at Vassar is named. In 1967, when Vassar received the first IBM Model 30 installed at a customer site, Dr Asprey told me that this algorithm had been run (in Fortran) as a demonstration for the Trustees of the College. To everyone's embarrassment, the computer reported the value of π to be 3. I suggested running the program in double precision. This was duly done, only to find that the value was still 3 — to a greater number of decimal places.

I included some of this material in a paper at the 1982 I.P. Sharp Users Conference; in my Banquet Address at APL83; and elsewhere as an ACM Distinguished Lecturer. Because in 1991 several features used in the present paper had not yet been implemented in J, I omitted analysis of the algorithm's failure from my paper in the special issue of the *IBM Systems Journal* commemorating the 25th Anniversary of APL [15, pp.563-564].

Roger Hui and Ken Iverson have most generously given help and advice on countless occasions. I started this study in 1964 by reading Iverson's *A Programming Language* [8] and Falkoff, Iverson, and Sussenguth's *A Formal Description of System/360* [4]. I am particularly grateful to all these and to the late Dr J.W. Bergquist of IBM.

References

- [1] Beckmann, Petr. *A History of π (pi)*. The Golem Press (1977) 4th Ed. pp.184-189.
- [2] Dorn, W.S. and D.D. McCracken. *Numerical Methods with Fortran IV Case Studies*. John Wiley & Sons, New York (1972) pp.106-120.
- [3] Eves, Howard. *An Introduction to the History of Mathematics, with cultural connections by Jamie H. Eves*. Saunders College Publishing, Philadelphia (1990) 6th Edition. pp.117-118.
- [4] Falkoff, A.D., K.E. Iverson, and E.H. Sussenguth. *A Formal Description of System/360*. IBM Systems Journal, Vol. 3, Number 3 (1964) pp.198-263.
- [5] Forsythe, George. *Pitfalls in Computation*. American Mathematical Monthly, Vol.77, Number 9 (1970) pp.931-956.
- [6] Greenblatt, M.H. *The "Legal" Value of π , and Some Related Mathematical Anomalies*. American Scientist, Vol. 53 (December 1965) pp.427A-434A.

- [7] Hennessy, J.L. and D.A. Patterson. *The IEEE 754-1985 floating-point standard*. In: *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann Publishers, Inc. (1990) Appendix A
- [8] Iverson, Kenneth E. *A Programming Language*. John Wiley and Sons, New York (1962) 286pp.
- [9] Iverson, Kenneth E. *Elementary Analysis*. APL Press, Swarthmore, Pennsylvania (1976) 219pp. ISBN 0-917326-01-6. Preliminary edition *Elementary Functions*. IBM Corporation (1974).
- [10] Iverson, Kenneth E. *A Personal View of APL*. IBM Systems Journal, Vol. 30, Number 4 (1991) pp.582-593.
- [11] Iverson, Kenneth E. J. *Introduction and Dictionary*. Iverson Software, Inc., Toronto (1994) ISBN 1-895721-08-3.
- [12] Jones, William. *Synopsis Palmariorum Matheseos, or a New Introduction to the Mathematics*. London (1706) pp. 263.
- [13] Maddox, John. *False Calculation of Pi by Experiment*. Nature, Vol.370, No.6488 (4 Aug 1994) pp.323
- [14] McIntyre, Donald B. *Experience with Direct Definition One-Liners in Writing APL Applications*. An APL Users Meeting, Proceedings. I.P. Sharp Associates Ltd., Toronto (1978), pp.281-297.
- [15] McIntyre, Donald B. *Language as an Intellectual Tool: From Hieroglyphics to APL*. IBM Systems Journal, Vol. 30, Number 4 (1991) pp.554-581.
- [16] Schepler, H.C. *The Chronology of Pi*. Mathematics Magazine (1950), pp.165-170, 216-228, 279-283.
- [17] Smith, David. *An Historical Piece of Pi*. In: *Interface: Calculus and the Computer*. Saunders College Publishing, Philadelphia (1984) 2nd Edition. pp.63-75 and *Instructor's Manual* pp.23-25.
- [18] Wrench, J.W. *The Evolution of Extended Decimal Approximations to Pi*. The Mathematics Teacher, Vol. 53 (1960) pp. 644-650.

Primes, Spirals and Coffee Tables

by Keith Smillie
smillie@cs.alberta.ca

Algorithms in J for generating prime numbers are discussed and then used to construct square and rectangular spirals of primes which may be used in the design of a tiled top for a coffee table.

Primes

Prime numbers may be found by using a sieving method which is attributed to Eratosthenes, the Greek mathematician, astronomer and geographer of the third century B.C. who estimated the circumference of the earth and the distance from the earth to the moon and the sun. To find all primes less than or equal to 1000, say, write down in order the positive integers from 2 to 1000, and then cross out all multiples of 2, then all remaining multiples of 3, all remaining multiples of 5, etc. until no more integers may be crossed out. (It is not necessary to use divisors greater than 31 since the square root of 1000 is 31.6.) Those integers remaining are the required primes. Two simple methods which exploit features of APL and J for implementing this sieving process are due to Kenneth Iverson. The first method is based on the observation that the only two divisors of any prime number are itself and unity. The sieve may generated by first finding the residue table with a list of positive integers as each argument and then creating a list of 0s and 1s with the 1s corresponding to those columns of the table indicating exactly two divisors. This sieve may be expressed in J as

```
ResidueSieve=. 2:=[: +/ 0:=|/~@pn
```

where

```
pn=. >:@1.
```

gives the positive integers, and, for example, Residue Sieve 10 is 0 1 1 0 1 0 1 0 0 0.

A list of prime numbers is given by

```
Pr1=. ResidueSieve # pn
```

The second sieve is

```
ProductSieve=. ~.@{] e. [: , */~@])@{).@pn
```

since the primes do not occur in a multiplication table for the integers. A corresponding verb for prime numbers is

```
Pr2=. ProductSieve # ).@pn
```

Approximate times using a 33-megahertz 486 to generate all primes less than or equal to 750 were 9.2 seconds using Pr1 and 429 seconds using Pr2.

The residue sieve may be made more efficient by using as arguments in the residue table a list from which all even integers greater than 2 have been eliminated. Such a list is given by

```
PN=. ((2&= +. odd) # ] )@pn
```

where

```
odd=. 1: = 2&|
```

and, for example, PN 10 is 1 2 3 5 7 9. Therefore, the sieve is:

```
Sieve=. 2:=[: +/ 0:=|/~@PN
```

and the primes verb is

```
Primes=. Sieve # PN
```

The expression Primes 750 was executed in approximately 1.4 seconds.

Spirals

Some interesting problems, originating with the American mathematician Stanislaw Ulam and described by Martin Gardner in *Scientific American*, are suggested when the positive integers are arranged in a spiral and the primes marked. It was found that the primes seemed to occur in series of straight lines. Ulam and his colleagues at Los Alamos studied these patterns extensively using spirals of up to approximately the first 65000 positive integers. In this section, however, we shall be concerned only with the description of a simple recursive algorithm for the construction of square and rectangular spirals of integers.

A spiral of positive integers may be considered to be a list of integers equally spaced along a length of string that is to be wound onto a stick of zero width and of a length determined by the difference between the length and width of the completed spiral. The left side of the figure given at the end of the paper shows

the step-by-step construction of a spiral of the first fifteen positive integers with five rows and three columns. The verb

```
QtrTurn=. [ , "2 |. @ |: @]
```

is used to repeatedly rotate the partially completed table of integers counter-clockwise through ninety degrees and append another row of integers on the top. This method of constructing a spiral may be simply expressed by the recursive verb

```
Wind=. ['((rows@[{.]) QtrTurn [) Wind rows@[{.])@.(0:<#@)]
```

where

```
rows=. 1: >. {.@($@)]
```

gives the number of rows in its table argument. Spirals of arbitrary dimension are given by

```
Spiral=. (1. @ ((- / @ ] ) , 0:)) Wind pn@(* /)@]
```

and for convenience square spirals by

```
SquareSpiral=. (1.@(0:,0:)) Wind pn@*:@]
```

The spiral of primes may be displayed by

```
SpiralOfPrimes=. (Spiral@] e. Primes@(* / @ ]))
```

which gives a logical table in which the primes are represented by 1s. The figure shows the use of the last three verbs.

Coffee Tables

A spiral of primes gives a pleasing design for a tiled top of a coffee table with tiles corresponding to prime integers in one colour, tiles corresponding to composite integers in another colour, and the tile corresponding to the integer 1 in a third colour. Since the tiles would be laid down row by row or column by column beginning at one corner, it is convenient to have some guide to the coordinates of the tiles and the corresponding integers. The verb

```
TopIntegers=. <"_2 @(>@AllCoords , "_2 _2 ,:"0 @ Spiral)
```

where

```
AllCoords=. <"1 @ ([ #: |.@i.@])
```

gives the coordinates, displays the coordinates together with the corresponding integers as a table of triples. The verb

```
TopPrimes=. <"_2 @(>@AllCoords ,"_2 _2 ,:"0 @SpiralOfPrimes)
```

gives a similar display with the primes and composite integers shown as 1s and 0s, respectively. The figure gives examples of the use of these two verbs.

Keith Smillie
Department of Computing Science,
University of Alberta, Edmonton,
Alberta T6G 2H1, Canada

References

Gardner, M., 1971. *Martin Gardner's 6th Book Of Mathematical Diversions from "Scientific American"*. The University of Chicago Press, Chicago.

Iverson, K. E., 1972. *APL in Exposition*. IBM Philadelphia Scientific Center Technical Report No. 320-3010. IBM Corporation, Philadelphia, Penn.

Iverson, K. E., 1993. *J Introduction & Dictionary*. Iverson Software Inc., Toronto, Ontario.

```
]s=. 1. 2 0
```

```
    ]s=. 1 2 QtrTurn s  
1 2
```

```
    ]s=. 3 QtrTurn s  
3  
2  
1
```

```
    ]s=. 4 5 6 QtrTurn s  
4 5 6  
3 2 1
```

```
    ]s=. 7 8 QtrTurn s  
7 8  
6 1  
5 2  
4 3
```

```
    ]s=. 9 10 11 12 QtrTurn s  
9 10 11 12  
8 1 2 3  
7 6 5 4
```

```
    ]s=. 13 14 15 QtrTurn s  
13 14 15  
12 3 4  
11 2 5  
10 1 6  
9 8 7
```

```
s e. Primes 15  
1 0 0  
0 1 0  
1 1 1  
0 0 0  
0 0 1
```

Spiral 5 3

```

13 14 15
12  3  4
11  2  5
10  1  6
 9  8  7

```

SquareSpiral 5

```

21 22 23 24 25
20  7  8  9 10
19  6  1  2 11
18  5  4  3 12
17 16 15 14 13

```

(SpiralOfPrimes &.>) 5 3;5 5

```

+-----+
| 1 0 0 | 0 0 1 0 0 |
| 0 1 0 | 0 1 0 0 0 |
| 1 1 1 | 1 0 0 1 1 |
| 0 0 0 | 0 1 0 1 0 |
| 0 0 1 | 1 0 0 0 1 |
+-----+

```

TopIntegers 5 3

```

+-----+
| 4 0 13 | 4 1 14 | 4 2 15 |
+-----+
| 3 0 12 | 3 1  3 | 3 2  4 |
+-----+
| 2 0 11 | 2 1  2 | 2 2  5 |
+-----+
| 1 0 10 | 1 1  1 | 1 2  6 |
+-----+
| 0 0  9 | 0 1  8 | 0 2  7 |
+-----+

```

TopPrimes 5 3

```

+-----+
| 4 0 1 | 4 1 0 | 4 2 0 |
+-----+
| 3 0 0 | 3 1 1 | 3 2 0 |
+-----+
| 2 0 1 | 2 1 1 | 2 2 1 |
+-----+
| 1 0 0 | 1 1 0 | 1 2 0 |
+-----+
| 0 0 0 | 0 1 0 | 0 2 1 |
+-----+

```

Prime Spirals

Stereograms in J

by E.A. Clough

Introduction

This note was inspired by a recent article by C.A. Reiter in *Vector* [1], by a 'Low Level' note by M. Liardet in *Personal Computer World* [2], and by my desire to learn how to program in J. I assume familiarity with Reiter's article, which describes the structure of bitmaps and gives J functions for handling them. My approach to the production of 'stereograms' — pictures which may be seen in stereo — is a little different from that outlined by Liardet in that I form them from the information in the separate images seen by the left and right eyes.

Programs have been developed in Windows J v7.0. I am grateful to Norman Thomson for advice on tidying up the code and comments on presentation.

Geometry

A viewer sees images of two-dimensional objects which lie behind a planar screen made of pixels. The objects occupy planes parallel to the screen and the viewer's eyes lie on a line which is parallel to the pixel rows. Considerations of linearity then confirm what common sense suggests, viz. that, if there is an unobstructed view of an object, the screen image seen by the right eye is displaced to the right of that seen by the left eye but is otherwise geometrically identical: in other words, the left-right displacement is the same for every point on the object.

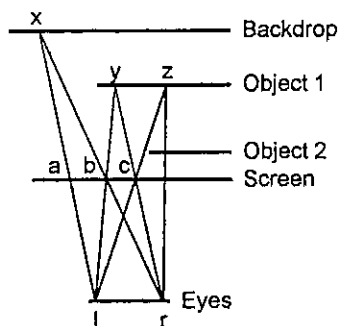


Figure 1: a Plan View

Figure 1 shows a plan view of an arrangement, with a backdrop behind the objects. From similar triangles, (xab, xlr) or (ybc, ylr) :

$$\text{left-right image displacement (} ab \text{ or } bc) = d / (1+q)$$

where d = distance between the eyes

q = ratio of the distances of eyes and the particular object from the screen.

Thus the displacement is a function of the distance or depth of the object behind the screen and it can be used as a key parameter in the development of a program.

Scaling

Because of the degrees of freedom in the system, scaling is not critical. I normally work with arrays of 128×96 pixels, to keep stereogram production runs down to about 5 minutes on my 486DX40 computer (8Mb RAM), but I print at 500% magnification to produce a picture about 6.7 inches wide. Since my eyes are about two and a half inches apart, the distance between them (d above), at this scaling, is:

$$128 * 2.5 / 6.7 = 48 \text{ pixels}$$

If the backdrop is at the same distance from the screen as the eyes, its q -value is unity and its displacement is 24 pixels. Images of objects may then have displacements of 1 pixel upwards. (My code does not permit an object to occupy a place on the screen, i.e. a displacement of zero.)

In practice I have obtained reasonable results with a backdrop displacement of 18 pixels but I have not experimented to determine whether there is an optimum for stereogram production.

Creation of a Stereo Image

In the formation of a stereo image, pixels only interact with others in the same row, because the rows are parallel to the line of the eyes.

In Figure 1, point ' x ' would be visible to the operator's left and right eyes so the two pixels ' a ' and ' b ', which the eyes see when looking in that direction, must be made the same colour. Also, if the left eye looks in the direction of pixel ' b ' it would be able to see the point ' y ', on object 1 and, since the right eye would also be able to see point ' y ', the pixel ' c ' must be made the same colour too. If, however, this process is continued via pixel ' c ', the left eye would be able to see point ' z ' but the right eye would not, because object 2 would be in the way,

terminating the sequence. The process may now be restarted at pixel ($a+1$) using a different colour (provided ' $a+1$ ' has not already been coloured).

Another type of terminating event, not shown in Figure 1, occurs if the path of the beam from the backdrop or object to the right eye lies beyond the edge of the screen.

Stereograms are produced by the function 'stereo' in Appendix 1, using a filename argument of the form 'xxxNN', where 'xxx' is the generic filename and 'NN' is the (two-digit) displacement integer for the backdrop. It applies the above logic to two arrays representing left- and a right- eye pictures. Execution of the program converts the right eye array into a stereo array indexing pixel colours which the program uses from a palette of 100 colours put in a random order. The palette array and the stereo array are then stored as a bitmap labelled 'xxxNN' which may be edited if necessary and magnified for printing.

The array retains its stereo characteristics even if re-saved as a monochrome bitmap; sometimes it actually seems to be easier to see the stereo image in this mode.

Creation of the Left- and Right-Eye Images

An image of the objects at one distance from the screen may be created in Windows Paintbrush. A black image on a white background will have an associated array, consisting of the numbers 0 and 255. Using J, another array may be amended so that those cells corresponding to the image cells hold the objects' displacement value. The images of objects at other distances may then be added to build up a composite image.

The function make in Appendix 1 produces left- and right- eye pictures from 'Cyclops' images of the objects that make up the scene, that is, from images which would be seen by a single central eye. The Cyclops images are held under filenames 'xxxnn', where 'xxx' is the generic name and 'nn' is the displacement value for the object. Function make is first called dyadically, with 'xxx' as left argument and with a vector right argument, the vector consisting of the array dimensions followed by the backdrop displacement value (or its negative if the arrays have been part-created in an earlier session). The same function is then called monadically with object displacement values as argument, starting with the object furthest from the screen, to produce left- and right-eye bitmaps labelled 'lxxxnn' and 'rxxxnn'. Figure 2 was produced from the following run:

```
'c:\jwin\scripts\' gets 'stereo'  
'geom'make 96,128,18
```

```
make"0 u=.15,12,9  
6!:2'stereo',''geom18''  
263.7
```

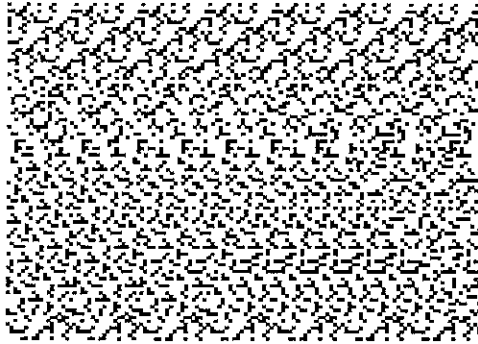


Figure 2 - Geometry (Triangle and Half-Circle)

Two Notes on Scenes

There is inevitably some loss of sharpness at the boundaries of stereogram images and this places a limit on the degree of complexity that can be achieved. Figure 3 was produced, using 256×192 arrays, by filling letters from the name of a well-known journal in Gill Sans MT Shadow font at 72 point. The letters are somewhat ragged. Those with faster machines than mine may be able to do better using larger arrays.

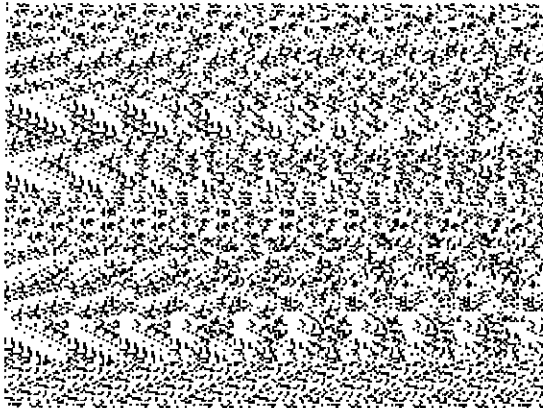


Figure 3 - The Name of a Well-known Journal

Because I work with images rather than objects, there is no automatic adjustment for the effects of perspective on image size and location. (Objects nearer to the screen will project larger images.) Great accuracy at the drawing stage will not normally be important but it may be more so if the 2-D objects are meant to serve as slices through 3-D objects. Figure 4 is an attempt at such a scene. I have no doubt that a simple J function could be designed to be applied before make, operating perhaps on outline rather than filled drawings of objects to produce the Cyclops images.

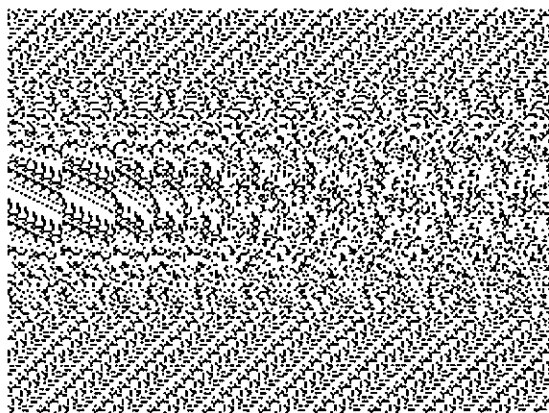


Figure 4 - Geometric Scene (Humps and Hollows)

E.A. Clough
Conifers
Leighton Beck Road
Slackhead, Beetham
Milnthorpe
Cumbria LA7 7AZ

References

- [1] C.A. Reiter, *Fractals RYIJ*, Vector Vol.11 No.2 Oct 1994.
- [2] M. Liardet, *Seeing Is Believing*, Personal Computer World Vol.17 No.12 Dec 1994.
- [3] Thimbleby and Neesham, *How to Play Tricks with Dots*, New Scientist, 9th October 1993, page 26

Appendix 1: The Script 'stereo.js'

NB. Statement 2 of 'readbmp8' and number 3 of 'writebmp8'
 NB. in Appendix 1 of reference 1 are each two statements.
 NB. After making these and other minor corrections, the
 NB. functions were filed as bmp8util.js

```
0! : 3 < 'c:\jwin\scripts\bmp8util.js'
```

```
make=: 0 : 0
z2=:<.y.%2 [ y=.list y. [ nam=._2}.name
('p';'b')=:readbmp8 'c:\jwin\'',nam,y,'.bmp' NB. read 'Cyclops'
b=.b=0 NB. changes image-cells to 1 and background to 0
('l','y') makes (-dim1){."1 b,"1 (dim0,(y.-z2))$0 NB. Shift left
('r','y') makes dim1{."1 b,"1~ (dim0,z2)$0 NB. Shift right
:
d01=:dim0*dim1 [dim0=:0{y. [ dim1=:1{y.
name=:x.,(list=:")z [ z1=.1+z=.{zz=.2{y.
$.=:3: ` 6: @.* zz<0 NB. don't initialise if zz<0
$.=.4+i.3 [ a=.(dim0,dim1) $ z [ q=.hue"0]2r3*(i.%<:)}z1
(q;a) writebmp8 'c:\jwin\l',name,'.bmp' NB. left background
(q;a) writebmp8 'c:\jwin\r',name,'.bmp' NB. right background
q=.' '
)

makes=: 0 : 0
''
:
xname=.(0{0{words),name [ words=.:x.
('q';'a')=:readbmp8 'c:\jwin\'',xname,'.bmp'
d=.,~.(,y.)*i.d01 NB. d is vector of image-cell linear indices
a=.(".2{words) d)a NB. adds image to left/right picture
(q;a)writebmp8 'c:\jwin\'',xname,'.bmp'
)

stereo=: 0 : 0
col=:c0 [c1=:1+c0=:."_2{y. [nn=:100 NB.'c0 is a 2-digit integer
('pp';'al')=:readbmp8 'c:\jwin\l',y.,'.bmp'
('pp';'ar')=:readbmp8 'c:\jwin\r',y.,'.bmp'
pp=: ((c1,3)$0),(nn?nn){hue"0]2r3*(i.%<:)}nn
ix=:i.x [X1=:X-1 [X=:1{$al
yloop"0 i.(0{$al) NB. loops through pixel-row dim
(pp;ar)writebmp8 'c:\jwin\'',y.,'.bmp'
:
''
)
```

```

yloop=: 0 : 0
yal=:y.{al [ Xy=:X*y.
y.& loop"0 iX
:
''
)

loop=: 0 : 0
:
$.=5:'1: @.* ci>(x=.y.){yar=.x.{ar
$.=2 [ col=:ci+nn!(col-c0) [ xs='''
$.=3:'4: @.* X1<x=.x+(z=.x{yal) [ xs=.xs,x
$.=4:'2: @.* z<:x{yar
$.=5 [ ar=:col (xs+Xy)}ar
)

t=.'readbmp8 writebmp8 hue make stereo'
SCRIPTNAMES=.t

```

NB. loops through pixels within rows

NB. resetting \$. in the last line avoids 'WS FULL'

NB. Is colour set?

NB. change colour

NB. beyond screen?

NB. rt eye blocked?

NB. change pixel colours

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Puzzle Corner: The Age of the Vicar (solution)	Alan Sykes	118
Technical Correspondence		119
Namespaces and $\square SE$	Kimmo Kekäläinen	119
More on APL 2.5	I Phelphritt (nee Leigh)	120
Understanding your File Timestamps	Ray Cannon	121
<i>The Common Mean and APL</i>	Roger Hui	123
IBM pre-announces the Pentium ... in 1977!	Bob Hoekstra	124
Cut and Paste from Winword to APL	Jan Karman (by phone)	124
Chaos — "computer error" not to blame	Nicholas Small	125
Answers to J Problems from Vector 11.3	Norman Thomson	127
ASCII Representation of APL Characters	Kenneth E. Iverson	131
At Work and Play with J: Cribbage 15s	Gene McDonnell	135
Functional Programming in J for Operations Research	Richard Brown	139

Puzzle Corner: The Age of the Vicar

The Solution from Alan Sykes

The Problem (restated)

A vicar says to his curate:

"I have three parishioners whose ages multiply together to equal 2450 and whose ages sum to twice your age — what are the ages of the parishioners?"

The curate thinks for a while and then tells the vicar that he does not have enough information. "Quite right" replies the vicar "but if I tell you an extra piece of information you will have enough." The extra piece of information is that the vicar is older than any of his parishioners.

Your task is to find the age of the vicar.

The Solution

Since 2450 equals $2 \times 5 \times 5 \times 7 \times 7$ you can work out a list of possible ages of the parishioners, which in increasing order of ages is:

2	25	49
2	35	35
5	10	49
5	14	35
7	7	50
7	10	35
7	14	25

The curate (naturally) knows his own age. These sets of numbers add up to 76 72 64 54 64 52 and 46 respectively, and so the curate's age is 32 and the possible sets of ages are 5 10 49 or 7 7 50, as 64 is the only number that is repeated.

The curate is told that the knowledge that the vicar is older than any of his parishioners is sufficient information for him to decide what the ages of the parishioners are. This can only be the case if the vicar is 50 — if he were 51 or more, you would not be able to choose between the two possible sets of ages!

Hence the curate is 32, the parishioners are 5 10 and 49 and the vicar is 50!

TECHNICAL CORRESPONDENCE

Namespaces and $\square SE$

From: kimmoke@tie.metsa.fi

3rd March 1995

Concerning the Dyalog session-file, I seem to be adopting a minimalist approach! I don't use any named `session.dse`; what I need I will load into a namespace under $\square SE$ dynamically from a set of little utility workspaces, that are saved and maintained in a quite traditional manner. The functions in them don't know a thing about been run in a namespace. (Most of them can be used as well under version 6.3.)

My APL loads in at the start a little workspace consisting only of a $\square 1x$ that moves the most generally used fns & idiomlike stuff (those *ROWFINDs*) to $\square SE$ to be always available. Among those is a two-line function *WS2NS* by which I'm able to load dynamically whatever tool-ws to a namespace under $\square SE$ or $\#$. This gives me the functionality, keeps my application workspace clean and allows me to get rid of the tool whenever space is needed, or if there is no use for it anymore.

There is nothing very brilliant or clever in this, but to me it seems to work perfectly, and brings me the advantages that I consider most relevant in the namespace-concept.

More on APL 2.5

From: I Phelphritt (nee Leigh)

1st April 1995

I was thrilled to see your review of IBM's APL2.5 in April's Vector 6-4, and felt moved to send you a further couple of examples. One consequence of the direct definition of operators which may not be immediately obvious, is the ease with which you can define the inverse operator for monadic functions:

INV: $\omega\beta$

INV produces a derived function which reverses the effect of its operand:

$A \leftrightarrow F \text{ INV } F \text{ } A$

for any array *A* and any monadic function *F*. The following examples show the power of this operator.

```

      ÷4
0.25
      ÷INV 0.25
4
      +\INV 1 3 6
1 2 3

```

In the case where the inverse is not simple, the system provides a 'least squares' solution:

```

      +/INV 6
6
      ?INV 4
7

```

and of course, *INV* may take a system function as its operand, so:

```

      □EX INV 1
FOO

```

... re-instates a previously expunged function, and:

```

2
      □DL INV 2

```

... produces the same effect as $\square DL \text{ } ^{-}2$ in Dyalog APL.

We are only now beginning to appreciate the power of this new implementation, but I'm sure that such examples will soon swell our idiom libraries.

Understanding your File Timestamps

From: Ray Cannon

5th March 1995

I have come across a "feature" of Dyalog APL/W which I thought other Vector readers might be interested in. This feature may not be restricted to Dyalog APL/W.

As each component in a component file is written or updated, its timestamp is also written/updated. The 3rd element of the result of the system function `⌈FRDCI` returns this information as "the time of the last update in 60ths of a second since 1st January 1970".

When I recently tried this out, the result returned (after conversion back to time of day) appeared to be 5 hours later than the time reported by the `⌈TS` function at the time of the update.

It transpires that unless you tell it otherwise (see later for how) Dyalog APL/W assumes that your clock is set to EST (Eastern standard time).

What `⌈FRDCI` returns is elapsed time, with an adjustment for the difference between GMT and local time. This allows files to be transferred between sites in different time zones, and "component saved times" to be compared on an equal basis.

OK then, how do we tell Dyalog APL/W which time zone we are in? Answer: set the DOS environment string TZ to specify the local time zone. (Note, PCs connected to a network should already have TZ set.) In the UK, add the line `TZ=GMT0BST` into your AUTOEXEC.BAT file.

The standard C library of time functions, relating to Local Time uses this variable whether in UNIX, DOS or Windows, so this may also apply to other APLs written in C. These time functions look for TZ and if they don't find it, set the time zone information to EST5EDT.

The following extract has been copied out of the Borland C++ on-line help file.

tzset

<TIME.H>

UNIX time compatibility; sets daylight, timezone, and tzname

Declaration: void tzset(void);

Remarks:

tzset sets the global variables daylight, timezone, and tzname based on the environment variable TZ.

`ftime` and `localtime` use these variables to correct Greenwich mean time (GMT) to the local time zone.

This is the format of the TZ environment string:

```
TZ = zzz[+/-]d[d][lll]
```

Component	What It Represents
-----------	--------------------

=====	=====
-------	-------

zzz	Three-character string representing the name of the current time zone. All three characters are required.
-----	---

[+/-]d[d]	Required field containing an optionally signed number with 1 or more digits. This number is the local time zone's difference from GMT in hours. Positive numbers adjust westward from GMT. Negative numbers adjust eastward from GMT. This number is used in the calculation of timezone.
-----------	---

lll	Optional three-character field that represents the local time zone's daylight saving time. If this field is present, daylight is set to non-zero. If this field is absent, daylight is set to 0.
-----	--

If the TZ environment string isn't present or isn't in the specified form, `tzset` presumes a default TZ = "EST5EDT" for the purposes of assigning values to `daylight`, `timezone`, and `tzname`.

Return Value: None

Portability:

DOS	UNIX	Windows
Yes	Yes	Yes

See Also:

`asctime` `ctime` `ftime` `gmtime` `localtime` `stime` `time`

So, if on your PC, the component time information does not tie in with your time stamp, set your time zone.

From: Roger Hui

March 1995

I read with interest Joseph De Kerf's article "*The Common Mean and APL*" in the January 1995 Education Vector. The various computations described therein have terse restatements in J:

```
a =. # %~ +/
g =. # %: */
h =. a&.%
```

The arithmetic mean is the number of items divided into the sum; the geometric mean is the n-th root of the product (n is the number of items); and the harmonic mean is the arithmetic mean under reciprocal, i.e. $h \ v \ * \ \%a\%v$. All three verbs work on arrays of numbers.

a 1 99	a 1 10 100
50	37
g 1 99	g 1 10 100
9.949874371	10
h 1 99	h 1 10 100
1.98	2.702702703

The common mean on a 2-element vector is an iteration where at each step, the elements are replaced by the geometric and arithmetic means. That is, at each step, v is replaced by $(g,a)v$, and the common mean is the first element of the limit $(g,a)^\wedge:_$

```
c =. {. @ ((g,a)^\wedge:_)
```

```
c 1 99
```

```
25.99812139
```

Since the power conjunction $^\wedge$: accepts an array of powers, successive iteration values are produced readily:

```
(g,a)^\wedge:(1.6) 1 99
```

```
1 99
```

```
9.949874371 50
```

```
22.30456721 29.97493719
```

```
25.85687532 26.1397522
```

```
25.99792902 25.99831376
```

```
25.99812139 25.99812139
```

Finally, the "ubiquitous constant U", the common mean of 1 and the reciprocal of the square root of 2, is:

```
c 1,%%:2
```

```
0.84721308479
```

IBM pre-announces the Pentium ... in 1977!

From: Bob Hoekstra

recd 8th Feb 1995

In "*The APL Handbook of Techniques*" on page 52, it says

B = 22233333334444445555555

C = 12345.67890123456789

So $(B * C) / B =$

12345.678901234567889994910

There is only one way to get such inaccuracy, and how did IBM know about it 15 years ago?

Cut and Paste from Winword to APL

Phone call from: Jan Karman

9th March 1995

The cut and paste problem (Vector 11.3 page 142) cannot be reproduced in Word for Windows version 6.0a. Perhaps Microsoft have fixed it! Can anyone else confirm this? It would be good to standardise on Winword for code transfer if the problem has gone away.

Chaos — “computer error” not to blame.

From: Nicholas Small

March 1995

As Gérard Langlet states at the beginning of his article in Vector 11.2 (p.82), chaotic behaviour occurs when a small variation in initial conditions has far-reaching effects on the development of a dynamical system. The essence of the chaotic system is that there exists no solution which is stable in the normal sense — the smallest displacement grows without bound. Now, the definition of “small” in this context is open to argument, but if one allows that it would be a few orders of magnitude larger than one part in $1E15$ (the typical precision of 64-bit floating point arithmetic), then any errors of computation (provided they are not cumulative — see below) are insignificant. In other words, if a system is not chaotic, rounding error is not of any consequence.

Accuracy in Arithmetic

In performing arithmetical computations, two common sources of loss of accuracy are: taking the difference of two values that are nearly equal; and, rounding repeatedly in the same direction. The most severe form of the latter is what might be termed an iterative random walk, the recipe for which is: after each new random step repeat all previous steps. As soon as two consecutive random steps are in approximately the same direction, the process moves in that direction at an exponentially increasing rate.

The case of repeatedly squaring a rotation matrix is an example of this. As soon as the determinant differs significantly from unity, the discrepancy doubles at each subsequent step. Typically, after about 45 squarings the value of the determinant is about 1.01 or 0.99; by 70, either *LIMIT ERROR* or zero is reached.

A random walk moving much more slowly, but no less surely, away from the starting point is obtained in the case of a biased random walk, in which the random component has a tendency to a particular direction. Consider the series that is the determinant of consecutive powers of a rotation matrix; there is randomness arising from the inaccuracy of each matrix multiplication, but if the determinant of the base matrix is not unity (though it typically is) then there is bias too. For most starting values, the error in the determinant never exceeds $1E-15$; in the worst case that I found, the error had grown to only $3E-12$ after 50000 terms of the series.

For many practical purposes, an error of one part in $1E15$ — equivalent to less than one hundredth of an inch in measuring the distance from the earth to the

sun — is of no consequence. However, there are occasions — for example if the number is to be the argument to a function such as *sine*, or in combinatorics — when the exact values of, or the differences between, very large numbers are required; in such circumstances, 64-bit floating point arithmetic is inappropriate.

The Logistic Equation

Contrast the discrepancies described in the previous section with the behaviour of the logistic equation examples in the following table, where two different starting values have been chosen with each of two values for the scale parameter. Light type shows where the figures for the second starting value differ from those for the first.

$$x[n+1] = 3.3x[n](1-x[n])$$

n	x[n]	x[n]
0	0.4	0.45
1	0.792	0.81675
2	0.543628	0.493909
3	0.818718	0.824877
4	0.489781	0.476700
5	0.824655	0.823208
6	0.477176	0.480269
7	0.823280	0.823715
8	0.480115	0.479187
9	0.823695	0.823570
10	0.479230	0.479496
11	0.823576	0.823612
12	0.479484	0.479406
13	0.823611	0.823600
14	0.479410	0.479432
15	0.823601	0.823604

$$x[n+1] = 4x[n](1-x[n])$$

n	x[n]	x[n]
0	0.4	0.40001
1	0.96	0.960007
2	0.153600	0.153570
3	0.520028	0.519946
4	0.998395	0.998408
5	0.006407	0.006355
6	0.025466	0.025261
7	0.099272	0.098492
8	0.357670	0.355167
9	0.918969	0.916094
10	0.297859	0.307462
11	0.836557	0.851716
12	0.546916	0.505181
13	0.991195	0.999892
14	0.034908	0.000429
15	0.134761	0.001717

With parameter value 3.3, a relatively large difference in starting values has almost disappeared after 15 iterations, whereas with parameter value 4, a relatively small difference simply grows.

The “Non-Syllogistic Mathematical Proof”

I leave it as an exercise for the reader, firstly, to show how to extend the technique used to prove that any given mathematical function is linear; secondly, to consider the effect of a small perturbation in $x[n] = 2 \cdot N$ upon subsequent terms in the doubling sequence, and the impact of this upon the derived series $\sin(x[n])$.

Answers to J Problems from Vector 11.3

From: Norman Thomson

February 25, 1995

Eugene McDonnell raises a number of issues in "*At Work and Play with J*" (Vol.11 No.3). The first is implicit in the casual remark that to exercise the verbs in the later sections you "may wish to get your system upgraded to J8". This fails to recognise that J8 and beyond are commercial versions and thus outwith the "shareware barrier". I am personally happy that version 7 has stabilised as an excellent piece of software with such immense possibilities for exploration at work and play that commercial style add-ons seem almost an irrelevance. Perhaps, however, Eugene might consider marking sections of his material with, say, *** to indicate "For Rich Readers Only". Poor readers such as myself can then get on with more interesting matters such as the problems which Eugene set. He asks for solutions by e-mail; being a very poor reader I have chosen to reply by v-mail (v = Vector) which has the compensation that my reply may reach a slightly wider audience.

Problem 1 asks for permutations of order n which, when repeatedly applied, generate, in the sense described below, all $2!n$ combinations from $1..n$ without any repetitions. First, the word "permutation" has always created difficulties because it is used to denote both a permutation vector, that is an arrangement of the items of $1..n$, and also an operation, of which J commendably makes it easy to make the distinction by defining a verb $p=. \&\{$. In what follows I shall use the word "permutation" in the sense of the operation and "permutation vector" to mean data. The permutation which is represented by a permutation vector has to be understood by constructing its cycle representation, conveniently obtained by $C.$, each item of which is a vector such as $4\ 0\ 2\ 1$, which is to interpreted as a sub-operation which moves item 0 to position 4, item 2 to position 0, item 1 to position 2, and item 4 to position 1. Eugene therefore seeks permutation vectors which when cycled through the $1..n-1$ powers of the permutation which they represent fulfil the pairs criterion — I shall call such vectors qualifying vectors.

If $n=6$, one qualifying vector is $5\ 2\ 0\ 3\ 1\ 4$ because when as a permutation it is applied to itself to the powers 0,1,2,3,4 it generates the permutation vectors

```
0 1 2 3 4 5
5 2 0 3 1 4
4 0 5 3 2 1
1 5 4 3 0 2
2 4 1 3 5 0
```

which read in sequence by rows as unordered pairs of items produce each of the fifteen pairs (0,1) (0,2), ... (4,5) once and once only.

For $n=6$, the $(n-1)$ -cycle requirement can be met by fixing just one of the items in 1.6 (which happens to be 3 in the permutation vector above) and making a derangement of the remaining $(n-1)$. (A derangement is a permutation vector in which no item occupies its original place). The fixed item can be selected in n ways. Suppose that in the general case it is chosen to be the last item $n-1$, so that the problem becomes one of selecting a derangement of $1..n-1$. If $(n-1)$ is prime, this is the only way in which Eugene's cycling property can be achieved. Not all derangements, however, satisfy the pairs property, for example the permutation 1 2 3 4 0 5 produces only six of the fifteen possible pairs.

The pairs property is derived by slicing the matrix of successive permutations into pairs of columns. This is difficult to relate in a mathematical way to other properties of permutations, but fortunately the J verb A. allows permutation vectors to be assigned numbers uniquely, so that qualifying permutation vectors can be found (all of them in the case that $(n-1)$ is prime) by the following procedure:

Use A. to progress through selected permutation vectors of $n-1$, selecting only those which satisfy the derangement property. For these derangements, generate the full $(n-1)$ cycle, append $(n-1)$, and count the number of distinct rows in the $2!n-1$ by 2 matrix of pairs. If this is $2!n-1$ then a single qualifying vector has been found.

The permutation vectors obtained by cycling, although they represent different permutations, should not be retained since they will eventually reappear as the full set of derangements is searched.

The J code to achieve this is relatively mundane, but it is worth quoting on account of its brevity (I have chosen to refrain from tacit programming snobbery!):

```

cycle=.(y.&{}^:(i.<:#y.)i. # y.' : ''
NB. returns y. raised by the permutation y. to the powers 0..n-1
b=.(2!#y.)=#~./:~"1((2!#y.),2)$,cycle y.' : ''
NB. returns 1 if all possible pairs arise in the matrix of cycles
d=.'-./y.=i.#y.' : ''
NB. returns 1 if perm vector is a derangement, otherwise 0

s1=.'i=.!y. [ $.=.x.#1 [ r=.(0,y.)$0'
s2=.'r=.r,((d*.(b@,&y.)v),y.)$v=.(i=.i-1) A. i.y.'
NB. s1 and s2 are the two lines of a dyadic verb f which searches
NB. the last x. permutation vectors of order y. and retains
NB. only those derangements which lead to qualifying vectors when
NB. y. is appended.
f=.' ' : (s1;s2)

```

thus (6 f 3),.3 gives two of Eugene's set of eight qualifying vectors of length 4. The remaining six arise by keeping 0, 1, and 2 in succession in their home locations. The reason for looping downwards from !y. is that A. orders permutation vectors in lexical order, which means that the first !i.y.-1 have their leading numbers in their "home" positions, and so cannot be derangements.

Turning to Eugene's problem 2, that is how many qualifying vectors are there of order n, unfortunately the number of derangements of n increases rapidly with n, the first few values being

```

n:2345678910
no. of der's:129442651854148331334961334961

```

An "elegant" approach might be to obtain all derangements of order y. by

```

der=.'(d"1 t)#t=.(i.!y.)A.i.y.' : ''

```

and then select those satisfying the pairs criterion by

```

ans=.'((b"1 t.,y.)#t=.der y.),.y.' : ''

```

Unfortunately for those of us who, like myself, work and play at or below the poverty line, this produces "ws full" for n=8 — hence the looping approach given above. Even this rapidly runs up against a wall of patience and/or "ws full" limitations as n increases.

For n=4, there are two fundamental permutation vectors corresponding to each of the two derangements of order 3. This is then multiplied by four to account for the fact that each of the items of i.4 can be placed in its "home" position. Division by !4 shows that one-third of all possible permutation vectors qualify.

In the case of $n=6$, there are 44 derangements of order 5 of which eight satisfy the pairs condition. Multiply by $n=6$ to give 48 possible permutation vectors, a qualifying rate of $48/720 = \text{one in fifteen}$.

For $n=8$ and above I have resorted to sampling. By looping through the first 500 derangements of order 7 using 500 $\neq 7$ I obtained 24 suitable ones leading to 192 qualifying vectors. Given that the search covered about one-eighth of the possible derangements, this figure scales up to an estimated qualifying rate of about one in 25.

For $n=10$, and again looping 500 times, I obtained 6 derangements and hence 600 qualifying vectors, suggesting a rate of about one in 100. For $n=12$, 500 times round the loop produced only one suitable derangement and thus 12 permutation vectors suggesting a rate of about one in 6000. As n increases there are more and more qualifying vectors but they become progressively less dense.

If, for a largish n , the objective is to obtain just one qualifying vector — any one — then an appropriate policy might be to repeatedly select a random left argument to $A_{\cdot}$ from the range $1..n$ until the first qualifying vector is generated.

ASCII Representation of APL Characters

From: Kenneth E. Iverson

March 1995

Introduction

Because of the problems in printing non-standard characters and of transmitting them on computer networks, there is a continuing concern with problems posed by the APL character set.

The situation at the time of designing the character set is described in the following passage from [1]:

Spelling

In natural languages the many words used are commonly represented (or *spelled*) in an *alphabet* of a small number of characters. In programming languages the words or *primitives* of the languages (such as *sin* and $=:$) are commonly represented by an expanded alphabet that includes a number of graphic characters such as $+$ and $=$.

When we came to implement APL, the alphabet then widely available on computers was extremely limited, and we decided to exploit a feature of our company's newly-developed Selectric* typewriter, whose changeable typing element allowed us to design our own alphabet of 88 characters. By limiting the English alphabet to one case (majuscules), and by using the backspace key to produce *composite* characters, we were able to design a spelling scheme that used only one-character words for primitives.

Because the characters of the underlying alphabet were distinct from the composite characters, both could be used to represent primitives. However, the restriction to one-character words had a cramping effect, leading to the divergence of different systems as each chose to use different new composites, and finally to the adoption of a "quad" prefix that led to a flood of multi-character words for miscellaneous facilities.

This is analogous to the situation in Chinese, where new single-character words are seldom adopted, but where a *radical* may be appended to a word to denote a new meaning, introducing, in effect, a multi-character spelling of words.

Continuing the passage from [1]:

Moreover, the spelling scheme was quite mnemonic in an international sense, relying on the appearance of the symbols rather than on names of functions in any national language. Thus the phrase $K \uparrow X$ takes K elements from X , and $\uparrow$ denotes *drop*.

The endurance of the character set through the many changes in the computing environment in the past thirty years is probably due primarily to the attraction of the mnemonic properties remarked above. Again continuing from [1]:

Because the use of the APL alphabet was relatively limited, it was not included in the standard ASCII alphabet now widely adopted. As a consequence, it was not available on most printers, and the printing and publication of APL material became onerous. Nevertheless, in spite of some experiments with "reserved words" in the manner of other programming languages, the original APL alphabet has remained the standard for APL systems.

The set of graphics in ASCII is much richer than the meagre set available when the APL alphabet was designed, and it can be used in spelling schemes for APL primitives that still avoid the adoption of reserved words. Such a scheme using variable-length words was presented in Reference 6 [Reference 2 in the present paper], and received limited use for communicating APL programs using standard printers, but was never adopted in any commercial implementation. A much simpler scheme using words of one or two letters was adopted in J, in a manner that largely retains, and sometimes enhances, the mnemonic character of the APL words.

Names of Variables and of Primitives

The discussion thus far has concerned only names for *primitives*, that is, names whose referents cannot be changed by the programmer. A programmer also needs a class of so-called *variable* names whose referents may be changed at will.

By using only letters from a native alphabet for variable names, and only names that include special graphic symbols for primitives, APL provides a clear distinction between primitive and variable names, and avoids the blurring of the classes produced by declaring certain variable names to be *reserved* for primitives.

This distinction carries through to national languages other than English, provided only that all characters of the national alphabet (including accented characters) be included in the underlying system and be permitted in variable names. Unfortunately, the use of some letters from the Greek alphabet for APL primitives prohibits their use in variable names.

Representations for APL Primitives

The universal adoption of the ASCII character set on computer systems makes it possible to choose a clearly-defined class of names for APL primitives that do not

conflict with national names for variables: denote each primitive by an ASCII graphic or by one or more ASCII graphics appended to a (possibly empty) list of national letters.

For example, the ASCII double-quote might be used as follows:

i"	for APL iota
av"	for quadAV (the underlying alphabet)
/"	for slash-bar
r"	for rho (shape)

It would also be possible to introduce a second termination symbol, or to use two occurrences of the same symbol. For example:

/% or /""	for grade-up
\% or \""	for grade-down

Although the accent grave is a National Use symbol in ASCII, it might well be used in some languages as a less obtrusive terminal character. For example:

/`	for slash-bar
/``	for grade-up
\`	for backslash-bar
\``	for grade-down

This scheme will be further illustrated for the special characters in the list on pages 327-9 of Gilman and Rose [3]. It should be noted that * is not used alone in ASCII because * is used (for power) in APL:

$\leq$	<`	$\geq$	>`	
$\neq$	=`			
$\vee$	+`	$\forall$	+``	
$\wedge$	*`	$\forall$	*``	
$\div$	%	$\times$	*`	[?! Ed]
$\in$	e`	ρ	r`	
$\uparrow$	take`	$\downarrow$	drop`	
ι	i`			
$\circ$	o`	$\circ$	o`	
ϕ	ro`	$\ominus$	ri`	
$\otimes$	tr`	$\otimes$	lg`	
$\lceil$	max`	$\lfloor$	min`	
Φ	/``	Ψ	\``	
$\perp$	#`	$\top$	#``	
$\underline{}$	exe`	$\mathbb{F}$	fmt`	
$\boxplus$	mi`			
$\diagup$	/`	$\diagdown$	\`	
$\leftarrow$	is`	$\rightarrow$	goto`	
∇	del`	∇	del``	
Δ	delta`			
$\square$	quad`	$\square$	qq`	
$\mathfrak{A}$	`	$\diamond$	``	
$S\Delta$	stop`	$T\Delta$	trace`	

References

- [1] K.E. Iverson, "A Personal View of APL", *IBM Systems Journal*, Vol. 30 no. 4, 1991
- [2] K.E. Iverson, "A Dictionary of APL", *APL Quote-Quad* 18 no. 1, 202-211, ACM, New York, 1990.
- [3] L. Gilman and A.J. Rose, *APL, An Interactive Approach*, Second Edition, Wiley, 1974.

At Work and Play with J Cribbage 15s

by Eugene McDonnell

Sir John Suckling (1609-1642) lived to just the age of 33, as you can see. He is supposed to have taken poison to avoid a life of penury, during the Civil War, in which he was a cavalier, not a roundhead. He was also a poet with an entrancing lyrical gift. He wrote:

*Her lips were red, and one was thin
Compared to what was next her chin
(Some bee had stung it newly);*

More to the point, John Aubrey, in his *Brief Lives*, tells us that Suckling also invented the game of cribbage, developing it from an earlier game, called noddy. I know nothing of noddy, but I was taught cribbage while I was in the army (1944-6) by a fellow soldier named Goman, who came from Duluth, Minnesota (named for a French explorer named Du Luth). According to Goman, Duluth was the cribbage capital of the world. The reason for this was that Duluth is at the very western end of the westernmost of the Great Lakes, Lake Superior, and in the winter, when lake traffic has stopped, there is nothing for people to do but go to their neighbourhood pub and play cribbage while drinking beer. They also had the world's championship cribbage match, to which all of the best cribbage players in the world came. Most of them didn't have to come far, since many were from Duluth. This may have changed over the last 50 years.

Just recently, when our nightly double solitaire game palled, my wife and I decided to play cribbage instead. I got out the cribbage board I had made, close to fifty years ago, and, with the aid of some wooden matches to use as pegs, we began to play. My wife hadn't played the game as much as I had, and it was clear that she was missing some opportunities to score because she didn't see some of the card combinations that added to 15. Such scores are a key part of the game.

To help her out, I decided to tabulate all the combinations of cards of length two through five that added to 15. This was easy enough for length two and three, but it became a little tedious and uncertain for greater lengths. I decided to take a few minutes longer and get it right by doing it with J.

Because I had been spending most of my time recently on studies involving Jane Austen, I hadn't been doing much programming, and I was a bit rusty. My first

attempt used a technique which generated all of the representations of 2, 3, 4, and 5-digit decimal numbers, removed those which had more than four of any digit, sorted the rows that were left (so that 4 1 became 1 4, for example), removed duplicates, and removed rows that didn't add to 15. This strategy foundered because I ran out of space.

I took a few more minutes to think of a more space-efficient strategy, and decided to use a program called 'part' that had been communicated to me several years ago by Roger Hui, which allowed one to generate all the length k partitions of a positive integer n. For example,

```
2 part 7
1 6
2 5
3 4
3 part 6
1 1 4
1 2 3
2 2 2
```

Having this made it almost too easy.

First I had to generate the partitions of 15 having length 2, 3, 4, and 5.

Next, since in cribbage the card values are from 1 (for ace) to 10 (for 10, jack, queen, or king), I had to remove rows having elements greater than 10.

Next, for the 5-partitions, I had to remove the last row, since this would consist of five threes (how did I know this?), and thus not be valid (there are only four threes in a deck of cards).

Last, I wanted to box the results so they could be joined together.

To get rid of rows containing values beyond 10 I wrote:

```
i=.*./"1@(10&>:) # ]
```

This takes as argument a table of numbers and produces a Boolean list by taking the and (.*.) over (/) the rows ("1) of the conforming table having a 1 for values for which 10 (10:) was greater than or equal (>:), and using this to copy (#) only those rows from the argument (1).

To get rid of the last row if the rows had length 5, I wrote:

```
m=.: ^: (5: = #@{.)
```

This curtails (:) the table if (^:) 5 (5:) is equal to (=) the length (#) of its first (.) row.

Boxing is primitive, so the entire result could then be obtained by writing:

```
]p=.2 3 4 5 <@m@i@part"(0) 15
```

5	10	1	4	10	1	1	3	10	1	1	1	2	10
6	9	1	5	9	1	1	4	9	1	1	1	3	9
7	8	1	6	8	1	1	5	8	1	1	1	4	8
		1	7	7	1	1	6	7	1	1	1	5	7
		2	3	10	1	2	2	10	1	1	1	6	6
		2	4	9	1	2	3	9	1	1	2	2	9
		2	5	8	1	2	4	8	1	1	2	3	8
		2	6	7	1	2	5	7	1	1	2	4	7
		3	3	9	1	2	6	6	1	1	2	5	6
		3	4	8	1	3	3	8	1	1	3	3	7
		3	5	7	1	3	4	7	1	1	3	4	6
		3	6	6	1	3	5	6	1	1	3	5	5
		4	4	7	1	4	4	6	1	1	4	4	5
		4	5	6	1	4	5	5	1	2	2	2	8
		5	5	5	2	2	2	9	1	2	2	3	7
					2	2	3	8	1	2	2	4	6
					2	2	4	7	1	2	2	5	5
					2	2	5	6	1	2	3	3	6
					2	3	3	7	1	2	3	4	5
					2	3	4	6	1	2	4	4	4
					2	3	5	5	1	3	3	3	5
					2	4	4	5	1	3	3	4	4
					3	3	3	6	2	2	2	2	7
					3	3	4	5	2	2	2	3	6
					3	4	4	4	2	2	2	4	5
									2	2	3	3	5
									2	2	3	4	4
									2	3	3	3	4

To make this a bit more useful, I copied it to my text editor (MacWrite Pro) and changed 10 to T and 1 to A, adjusted widths a bit, and added a few header and footer lines, giving:

Ways of counting fifteen with 2 3 4 5 cards in cribbage

5.	T	A	4	T	A	A	3	T	A	A	2	T
6	9	A	5	9	A	A	4	9	A	A	3	9
7	8	A	6	8	A	A	5	8	A	A	4	8
	A	7	T	A	A	6	7	A	A	5	7	
	2	3	T	A	2	2	T	A	A	6	6	
	2	4	9	A	2	3	9	A	A	2	2	9
	2	5	8	A	2	4	8	A	A	2	3	8
	2	6	7	A	2	5	7	A	A	2	4	7
	3	3	9	A	2	6	6	A	A	2	5	6
	3	4	8	A	3	3	8	A	A	3	3	7
	3	5	7	A	3	4	7	A	A	3	4	6
	3	6	6	A	3	5	6	A	A	3	5	5
	4	4	7	A	4	4	6	A	A	4	4	5
	4	5	6	A	4	5	5	A	A	2	2	8
	5	5		2	2	2	9	A	A	2	2	7
				2	2	3	8	A	A	2	2	6
				2	2	4	7	A	A	2	5	5
				2	2	5	6	A	A	2	3	6
				2	3	3	7	A	A	2	3	4
				2	3	4	6	A	A	2	4	4
				2	3	5	5	A	A	3	3	5
				2	4	4	5	A	A	3	4	4
				3	3	3	6	2	2	2	2	7
				3	3	4	5	2	2	2	3	6
				3	4	4	4	2	2	2	4	5
								2	2	3	3	5
								2	2	3	4	4
								2	3	3	3	4

T = 10, J, Q, or K

The key to this is Roger Hui's partition function, 'part':

```
part =. basis`recur@.test
basis =. (0&@] , [])$ (1&@[ 1&>.@* ])
recur =. (mask form ] )@(part&<:)
mask =. start <:/ {"1 <. -.@(-/)@(_2&{.)"1
start =. +/@{. >:@i.@<.@%&&>: {:@$
form =. prx@[ decr@.. ind@[ { ]
pfx =. +/"1 #>:@i.@#
decr =. (>:@(-/)@(_1 0&{) _1} ])"1
ind =. , #*/@$ $ 1.@{:@$
test =. 1&<@[ *.<
```

All in all, I spent a happy half-hour at play with J, and my wife now beats me pretty regularly at cribbage.

Functional Programming in J for Operations Research

by Richard L.W. Brown rlwbrown@mathstat.yorku.ca
reprinted by permission from 'Gimme Arrays' Nov 1994

What is Functional Programming?

Functional programming is a style of computer programming which, in its purest form, involves only functions. (For a short survey article on functional programming see the August 1994 issue of *Byte*.) If x represents the input data of the program, then the goal is to design a function f such that $y = f(x)$ is the desired output. Thus a program is a function. The function f is built by combining various primitive functions that are provided by the functional programming language.

This approach to programming is quite different from the conventional approach (exemplified by assembly language, FORTRAN, Pascal, C, etc.) which may be called *imperative* programming because a program is a list of commands. In imperative programming, data is stored in memory cells with symbolic addresses and a list of commands is executed in order, usually with some branching or looping. This conventional approach is really a sugaring of machine language for a von Neumann type of computer architecture with stored program, stored data, and step-at-a-time execution of commands. The functional approach is machine independent: functional programs do not have to be rewritten for a different architecture, say, for a parallel machine.

What is J?

J is a programming language developed from *A Programming Language* (APL) by Ken Iverson (father of APL) and Roger Hui. Unlike APL, which used a special character set, J uses only the ASCII characters. Primitive functions are denoted by single characters like $+$ or by following such characters with a period or colon as in $+.$ or $+:$. Like APL, J has primitive functions for operating on arrays without using indexes or looping. For example, if A and B are two matrices of the same shape then $A+B$ computes the element-wise sum of A and B , and $>. /A$ computes the list of greatest elements of the columns of A .

Why use J for Functional Programming?

J has a vast collection of primitive functions to call upon and these extend easily to arrays. More importantly, J has a variety of ways of combining functions that go beyond the usual composition of functions. One of the most important combinations is called the *fork*. If f and g are monadic functions and h is dyadic then $f h g$ is a monadic function and its meaning is as follows: $f h g(x)$ is $f(x) h g(x)$. This construction is familiar in mathematics where, for example, the sum of two functions $f + g$ is a function whose value on an argument x is defined to be $f(x) + g(x)$. A standard example of a fork from the J tutorials illustrates the idea. Suppose that x is a list of numbers and we want $y = a(x)$ to be the arithmetic mean. We write

```
sum   =:  +/
```

(read as 'plus over', see Appendix D for a dictionary of all the J primitive functions used in this article) and then the fork

```
a   =:  sum % #
```

(which reads as 'sum divided by number') is the desired arithmetic mean function.

The assignment statements used in the sentence above are assignments of names to functions (or programs or verbs as they are called in J). No use has been made of assignments of names to data (nouns in J). Moreover, the assignments made above are permanent names. Contrast this with the traditional approach. The code for arithmetic mean in a conventional language is a sequence of commands with assignment statements of data to named memory cells (variables). The sequence of statements has control structures that produce loops. Some variables refer to memory cells whose contents change each time you go around a loop. Thus variables are transient things that are used in the program but have no permanent meaning. (In functional programming, an assignment of the form $x := x + 1$ makes no sense! In imperative programming such statements are commonplace!)

In J, a *dyadic fork* is also defined. This definition is more general than normal mathematical notation or the kind of compositions defined in other functional programming languages. The fork $+*-$ is defined as follows: $x(+*-)y$ means $(x+y)*(x-y)$. We will use the dyadic fork extensively below. A sequence of three verbs is a fork. A sequence of two verbs in J is called a *hook*. The first verb must be dyadic and the second monadic. For example, $x(+%)y$ means $x + 1/y$ and $(+%)y$ means $y + 1/y$. Thus we can read the hook $+%$ as 'plus reciprocal'.

What is Operations Research?

Most problems in operations research involve optimizing an objective subject to constraints. Typical examples are maximizing profit subject to resource constraints or minimizing shipping costs subject to the constraints of a transportation system. One of the most common types of problems has a linear objective function and linear constraints and such problems define a subject called *linear programming*. (This is a quite different use of the word 'programming'!) Here is a very small example:

$$\begin{aligned} &\text{maximize } z = 2x + 3y \\ &\text{subject to } 4x + y \leq 12 \text{ and } 2x + 5y \leq 15 \\ &\text{with } x \geq 0 \text{ and } y \geq 0. \end{aligned}$$

The *simplex* method was developed by G. Dantzig in the 1950s and has been the standard method for solving linear programming problems since then. Only recently has the simplex method been challenged by other methods and only in large problems with thousands of variables and constraints. To apply the simplex method, we convert the inequalities to equalities using slack variables: $4x + y + s = 12$ and $2x + 5y + t = 15$ with $s \geq 0$ and $t \geq 0$. We write our problem as a system of linear equations:

$$\begin{aligned} 1z - 2x - 3y + 0s + 0t &= 0 \\ 0z + 4x + 1y + 1s + 0t &= 12 \\ 0z + 2x + 5y + 0s + 1t &= 15 \end{aligned}$$

and study various feasible (i.e. non-negative) solutions of this system. Notice that the system is 'solved' for the variables z , s , and t in terms of x and y . We can set $x = 0$ and $y = 0$ and we get three equations in three unknowns with obvious solution $z = 0$, $s = 12$, $t = 15$. This solution is feasible because all variables are non-negative; but this solution does not have a very good value of z . In the simplex method, we move from solution to solution in such a way that z increases with each move until no further increase is possible.

If we divide an equation by a nonzero constant or if we add a multiple of one equation to another equation, then we get a new system of equations that is equivalent to the old system. On the computer, we do these operations on the matrix of coefficients. We divide a row by a constant and add multiples of a row to other rows.

Below we show the original coefficient matrix and beside it we show the result of dividing the bottom row by 5 and then adding multiples of the new bottom row to the rows above to create zeros in the rest of the y-column.

z	x	y	s	t		z	x	y	s	t		
1	-2	-3	0	0	0	1	-0.8	0	0	0.6	9	
0	4	1	1	0	12	-->	0	3.6	0	1	0.2	9
0	2	5	0	1	15		0	0.4	1	0	0.2	3

The new matrix represents a system of equations

$$\begin{aligned} 1z - 0.8x + 0y + 0s + 0.6t &= 9 \\ 0z + 3.6x + 0y + 1s - 0.2t &= 9 \\ 0z + 0.4x + 1y + 0s + 0.2t &= 3 \end{aligned}$$

which is solved for z , y , and s . If we set $x = 0$ and $t = 0$ we get the feasible solution $z = 9$, $s = 9$, and $y = 3$. The row operations above are called a 'pivot' for geometric reasons (the feasible set has been pivoted around so that a new point of it is at the origin). It turns out that if we do another pivot that converts the 3.6 to 1 and clears the rest of the x-column then we get a new feasible solution with $x = 2.5$, $y = 2$, $z = 11$. Moreover, this turns out to be the optimal solution.

The simplex method can be divided into two parts: (1) finding the location of the pivot (including testing to see if the current solution is already optimal and to see if the objective function is unbounded) and (2) the pivot operation itself.

Pivoting in J

Let M denote the original matrix of coefficients above. Let R denote the pivot row, $R =: 0 \ 2 \ 5 \ 0 \ 1 \ 15$, let P denote the pivot, $P =: 5$ and let C denote the pivot column, $C =: _3 \ 1 \ 5$. Then the row operations making up the pivot can be summarized in one J expression:

$$M - (C-B) * (R \% P)$$

where $B =: 0 \ 0 \ 1$. This formula says that the pivoted matrix is the original matrix M minus various multiples (the multiples are in the expression $C-B$) of the row R divided by the pivot P . The B -term is necessary because the third row is treated differently from the other rows; without the B -term, the pivot row would become a row of zeros. Given this formula, it is clear how to write J code in non-functional style for doing the pivot. (See Appendix C for the simplex method written in J using a traditional approach.) But we will show how to use a purely functional approach based on the use of dyadic forks.

We will define a function `pat` ('pivot at') that would be used as follows to do the above pivot:

```
M pat 3 3
```

meaning 'M pivot at row 3 and column 3'. Note that we are indexing rows from 1 to 3 and columns from 1 to 6 (not 0 to 2 and 0 to 5 as is normally done in J). This means that our functions must convert back and forth between mathematical indexing (with origin 1) and J indexing (with origin 0).

According to Appendix D, the dyadic verb `⌈` gives its left argument. Therefore the definition of `pat` is the dyadic fork

```
pat =: ⌈ - ext
```

where `ext` is the dyadic function such that `M ext 3 3` computes $(C-B) * (R \% P)$. If we now use `mc` and `mr` ('modified column' and 'modified row') for the dyadic functions computing $(C-B)$ and $(R \% P)$ then the definition of `ext` is the fork

```
ext =: mc */ mr
```

and it remains to define `mc` and `mr`. Assume we have defined dyadic functions `xc` and `xr` and `xe` ('extract column', 'extract row', 'extract element') that work as follows: `M xr 2 3` extracts the second row of `M` (ignoring the column index) `M xc 2 3` extracts the third column of `M` (ignoring the row index) and `M xe 2 3` extracts the element in the second row and third column. Also suppose `bv` is a dyadic function such that `M bv 2 3` is `0 1 0` (a basis vector with a 1 in the second position, ignoring the column index 3). Then `mc` and `mr` can be defined as forks

```
mc =: xc - bv
mr =: xr % xe
```

Now we define `xc`, `xr` and `xe`. Recall that we are referring to the first row as row 1 not as row 0. Each verb is a fork with central verb 'from' (which is `⌈` in J, see Appendix D). The right verb is `⌈` which means 'left argument'. The left verb is a composition (using `@`) of several verbs. The monadic verb `<:` is 'decrement' and is used to change from mathematical indexing to J indexing. Refer to Appendix D for other verbs.

```
xr =: (<:@{.@]) { ⌈
xc =: (<:@{:@]) {"1 ⌈
xe =: (<@<:@]) { ⌈
```

Actually we could make shorter definitions using hook instead of fork and using the commute adverb ~ to swap the arguments of { .

```
xr =: {~    <:@{.
xc =: {~"1  <:@{:
xe =: {~    <@<:
```

Finally, we define bv. We use a fork with central verb equals. The left verb takes the left argument, finds the number of items (rows) m, and then creates an index set 1; 2...m. The right verb takes the right argument, takes the first entry (which is the pivot row index) and decrements it. Under equals, the result is a vector of 0s and a single 1 where the index equals the pivot row index.

```
bv =: (i.@#@[) = (<:@{.[@])
```

This completes the definition of pat. It can be tested as shown in the examples below.

The Simplex Method in J

It remains to specify where to pivot and when to stop pivoting. The pivot column is selected as that column (excluding the last column) whose first row has the most negative entry. In the example above, starting with the solution $x = 0, y = 0, z = 0, s = 12, t = 15$ one sees that $z = 2x + 3y$ can be improved by increasing either x or y ; but an increase in y is better. So the y -column is selected as the pivot column. In the matrix, this choice corresponds to selecting the most negative element in the top row. If none of these entries is negative then no improvement in z is possible so no further pivot is required and the optimal solution has been found. Once the pivot column has been selected, we need to know how large to make the variable (y in the example) that is being increased. Each constraint (that is, each row of the matrix below the first row) sets a limit on the size of the variable being increased. The limit can be computed as a quotient with numerator from the right border and denominator from the pivot column. For example, the first constraint gives the limit $y \leq 12/1$ and the second constraint gives the limit $y \leq 15/5$. All such constraints must be satisfied. If the smallest is satisfied then all will be satisfied; hence the pivot row is selected to minimize the quotients. However, there is a catch: if the denominator is zero or negative then the constraint should be ignored.

In teaching the simplex method, one finds that students have no problem doing the simple calculations that find the pivot column and the pivot row as outlined in the previous paragraph. But the pivot operation involves tedious arithmetic. Hence it is convenient for students, who are learning about the simplex method,

to find the pivot manually and then use the function `pat` defined above to do the pivot.

It is, perhaps, surprising that, although finding the pivot is a trivial part of the simplex method when done by hand, the computer code for finding the pivot is rather tricky! First we need a function to find the location of the minimum element in a list. Least is 'lesser over' and is therefore `<./`. The hook below can be read as 'index of least'

```
m1 =: i. <./
```

Now we can write the function `pc` that finds the pivot column. We could take the first row, drop the last element, and find the minimum location. Then we could increment this index so the first row has index 1 (not 0) as mentioned above. This can be accomplished as a composition (using `@`) of `m1` with primitive functions:

```
pc =: >: @ m1 @):@ {.
```

However, this is not good enough! Recall that we also want to detect an optimal matrix so we can tell the simplex method when to stop. Therefore, we adopt a somewhat different approach. We take the first row, drop the last element, append a leading zero, and apply `m1`. If the row has a negative element then `m1` will find the location of the most negative element and, because of the leading zero, that index will already be incremented into the range 1 to `n`. If there is no negative element in the top row, then `m1` will find the appended leading zero at location 0 and `pc` will return a zero result. This can be used to detect optimality with the function `opt`.

```
pc =: m1 @ {0&,) @ }:@ {.  
opt =: 0&= @ ]
```

To find the pivot row we need a new version of division. We want ordinary division, else, if the denominator is not positive, infinity. When such 'quotients' are searched by `m1` for the location of the minimum, those whose denominators are not positive will be ignored. In the code below, the function `inf` is a constant function with value infinity. The function `dnp` ('denominator not positive') is 1 if the denominator is not positive and 0 otherwise. The new division operation is `dp` which stands for 'divide positive'.

```
else =: '  
if =: @.  
inf =: _:@ [  
dnp =: 0&>: @ ]  
dp =: % else inf if dnp"0
```

We can now find the pivot row with the dyadic function `pr` whose left argument is the matrix and whose right argument is the index of the pivot column.

```
pr =: >: @m1 @ ((xc 0:) dp xc)
```

(The dirty trick in the above line is this: the hook `(xc 0:)` appears to extract column zero but actually extracts the last column to be used as the list of numerators! This is because `xc` includes a decrement function that reduces the 0 to -1. In J the index -1 means last.) Note that the function `pr` works on the entire columns including the first row. The first row is not a constraint and therefore should be ignored. But the first row will always give an infinite 'quotient' under `dp` and so will be ignored by `m1` as desired.

What happens if all 'quotients' are infinite? In that case there is no limit on the size of the variable being increased and the objective function is unbounded. This situation can be detected by the condition that `pr` selects the first row and produces the result 1. The function `unb` detects an unbounded problem.

```
unb =: 1&= @ {. @ ]
```

We create a function `cpat` ('conditional pivot at') which is 'pivot at else infinity if the problem is unbounded'.

```
cpat =: pat else inf if unb
```

Now we create a function `pin` ('pivot in') such that `M pin 2` means 'M pivot in column 2'. The pivot row is found automatically by `pr`.

```
pin =: [ cpat pr,]
```

This train of five dyadic verbs is parsed by J as `[ cpat (pr,)]` which is a fork in which the third verb is itself a fork.

We can now define either a recursive or iterative version of the simplex method. The recursive version is simple:

```
simplex =: (simplex @ pin else [ if opt) pc
```

For the iterative version, we define 'conditional pivot in' and 'pivot'

```
cpin =: pin else [ if opt
pivot =: cpin pc
```

Finally, the `smplx` function iterates the pivot function 'to infinity'! This means the matrix is pivoted until the same matrix occurs twice in succession (and hence no

further change will occur). The result will be either the optimal matrix or infinity if the problem is unbounded.

```
smplx =: pivot^:_
```

Note: As one might expect, there are many ways to write the simplex method in J using a functional programming style. For a different approach, see Appendix B.

Examples

To solve the small problem posed above, we define the matrix

```
M=:1 _2 _3 0 0 0 0 4 1 1 0 12 0 2 5 0 1 15
M=:3 6$M
M
1  _2  _3  0  0  0
0  4  1  1  0  12
0  2  5  0  1  15
```

Then we apply either of the functions `smplx` or `simplex`

```
smplx M
1 0 0 0.222222 0.555556 11
0 1 0 0.277778 -0.055556 2.5
0 0 1 -0.111111 0.222222 2
```

and get the optimal matrix. As a system of equations, this is

$$\begin{aligned} 1z + 0x + 0y + 0.222s + 0.555t &= 11 \\ 0z + 1x + 0y + 0.277s - 0.055t &= 2.5 \\ 0z + 0x + 1y - 0.111s + 0.222t &= 2 \end{aligned}$$

it tells us that the optimal solution is $z = 11$, $x = 2.5$, $y = 2$. (And $s = 0$ and $t = 0$.)

One advantage of the functional approach is that we have developed a collection of useful functions along the way. These can be tested separately or used to show students the steps of the simplex method. For example

```
pc M          NB. Finds pivot column.
3
M pr 3        NB. Finds pivot row in third column.
3
M1=:M pat 3 3 NB. Pivots at 3, 3.
M1
1 -0.8 0 0 0.6 9
0 3.6 0 1 -0.2 9
0 0.4 1 0 0.2 3
```

```

      pc M1
2      M1 pr 2
2      M1 pat 2 2
1 0 0 0.222222 0.555556 11
0 1 0 0.277778 -0.055556 2.5
0 0 1 -0.111111 0.222222 2

```

Appendices

Appendix A: Functional Program Summary

The functions defined in this article are: `pat ext mr mc xr xc xe bv ml pc opt if else inf dnp dp pr unb cpat pin simplex cpin pivot smplx`.

They can be entered in any order except that `else` and `if` must be entered before they are referenced by another function. (That is because J parses an unknown word as a verb; but `else` and `if` are conjunctions. To be parsed correctly, they must be identified as conjunctions before they are parsed.)

Note that it may be desirable to use J's *fix* adverb to 'compile' some of these functions into compositions of primitives. For example we can write

```
pat =: pat f.
```

and then we find that instead of the original definition, `pat` is now defined as

```
[ - (({~"1<:@{:} - (1.@#@[=<:@{.@])) */ (({~<:@{.) % ({~<@<:))
```

The new 'compiled' version runs a bit faster than the original. Moreover, we could now erase some of the functions that were used to define `pat` if we have no need for these functions and don't want them cluttering up the symbol table.

```

erase =: (41:55)@;:
erase 'ext mr mc xr xc xe bv'
1 1 1 1 1 1

```

(The output of `1s` shows that all six items were erased.)

For readability it may be desirable to present the functions in a different format. In what follows, we make use of the conjunction `[]`. (which simply produces the verb that is its left argument) to place several definitions on the same line. We also eliminate the `ext` function by defining `pat =: [] - mc*/mr`.

```

NB.-----
NB.  General  Utility  Functions
NB.-----
    WHERE=:AND=: [.
        else=: '
        if=:@.
NB.-----
NB.  The  Simplex  Method
NB.-----
    simplex=: (simplex@pin else [ if opt) pc WHERE pin=: [cpat pr,] AND;
    opt=: 0&=@ AND pc=: ml@(0&.)@: @(.
    cpat=: pat else inf if unb WHERE pat=: [-mc*/mr AND;
    inf=: _; @ [ AND unb=: 1&=@{. @.
    mr=: xr%xe WHERE xr=: {-<:@{. AND xe=: {-<@<;
    mc=: xc-by WHERE xc=: {"1<:@{; AND bv=: {1.@#@[] = (<:@{. @.
    pr=: >:@ml@: ((xc 0:) dp xc) WHERE dp=: % else inf if dnp"0 AND;
    ml=: i.<./ AND dnp=: 0&>:@.
NB.-----
NB.  Example
NB.-----
    [M=: 3 6$1 _2 _3 0 0 0 0 4 1 1 0 12 0 2 5 0 1 15
1  _2 _3 0 0 0
0  4  1  1  0 12
0  2  5  0  1 15
    simplex M
1  0  0  0.222222  0.555556  11
0  1  0  0.277778  _0.055556  2.5
0  0  1  _0.111111  0.222222  2

```

Appendix B: A 'Higher Level' Functional Version

Here is a different approach to the simplex method. First we assume the pivot is in the first row and first column. That allows us to define a monadic function `piv` such that `piv M` will do the pivot operation on `M`. Moreover, we will give a new way of doing this pivot. We handle the first row first (by dividing it by its first element) and then do the lower rows by subtracting a `*` product.

```

fr  =: {.%{.@{.
lrs =: }. - (({"1@.)*)/({.
piv =: ({., lrs)@ (fr, .)

```

The dyadic function `row` is used as in `M row 4` which takes the first 4 rows of `M` and reverses their order, thereby bringing the 4th row to the top. (Of course `M row 4` applied again will restore `M` to its original form.)

```
row =: ((|. @ {.), .)~

```

Similarly, `col` does the same for columns.

```
col =: row"1
```

Now we define a conjunction `at` so that `M piv at row 4` will bring row 4 to the top, do the pivot and then put row 4 back where it belongs.

```
at =: @:'J.'J\
```

(At the end of this appendix is an explanation of `at`.)

We define a function `pr` that finds the pivot row of a matrix assuming that the pivot column is the first column. For that, we use the functions `if`, `else`, `dnp`, `dp`, `m1` defined in the main article above.

```
pr =: >:@m1@:(((:"1)dp( {. "1))
```

We also use the same functions `inf`, `unb`, `opt`, and `pc`. Then we put it all together as follows:

```
pin      =: (piv at row else inf if unb pr) at col
simplex =: (simplex@pin else [ if opt) pc
```

This code is more sophisticated than that of the main article because it uses a defined conjunction `at` which is a higher level function (its arguments are functions).

To describe the conjunction `at`, it is best to introduce the explicit method of defining functions. In J one may define functions in two ways: tacitly or using explicit variables as parameters. We have used the tacit method in this article. For example, consider the dyadic function `x f y` which computes `10 + x + y`. We could define this tacitly by `f =: (10&+ )@+` or by the fork `f =: 10"_ + + .` Or we could define it explicitly using J's parameters `x.` and `y.` by

```
f =: 3 : '10+x.+y.'
```

(Here the '3' tells J to make a verb. For a conjunction, use '2', for an adverb use '1' and for a noun use '0'.) In this case, the function `f` will be stored by J in the explicit form given and when called, as in `5 f 6`, the values 5 and 6 will be substituted for `x.` and `y.` respectively. It is also possible to ask J to convert an explicit form to a tacit form. To do this, we use '13' instead of '3'. Thus

```
f =: 13 : '10+x.+y.'
```

causes J to produce the tacit verb `f =: 10"_ + + .`

In our case, we want to create a conjunction *at* that applies its right argument (some verb, in our case *row* or *col*) first, then applies its left argument (some other verb, in our case *piv*) and then applies its right argument again. (Note: This is similar to, but not exactly the same as, the J primitive conjunction *&.*) Thus we can define *at* explicitly by writing

```
at =: 2 : '(x@:y.)y.]'
```

When given an instance like *M piv at row 4* the conjunction *at* forms a fork (*(piv@:row)row*) which combines with the arguments *M* and *4* to give (*(piv(M row 4))row 4*) which is just what we want.

If, instead of the above, we ask J to create a tacit version by writing

```
at =: 12 : '(x@:y.)y.]'
```

then J produces the tacit definition *at =: @:'.')]'* given above. However, to program this tacit conjunction directly (or even to untangle its curious grammar!) one must go into much more detail on J than is appropriate in this short article.

Appendix C: J Program in Imperative Style

Here is the simplex method written in the traditional non-functional way — except that we define four functions at the start! This illustrates the fact that one may combine the functional programming style and the imperative programming style as one sees fit to create an efficient and readable program.

```
where =: and =: [  
all    =: */.  
ml     =: i.<./
```

The program is a sequence of assignment statements involving variables (nouns). We use the *=.* assignment instead of *=:* because we want the nouns to be local to the function. That is, nouns like *M* and *obj* are special nouns that exist only inside the function *simplex* and do not conflict with other nouns of the same name that may exist outside *simplex*.

```
simplex =: 3 : 0  
obj =. }.M and indx=.i.m where m =. #M and M =. y.  
if. all obj >: 0 do. M else.  
  den =. }.C where C =. pc{"1 M and pc =. ml obj  
  if. all den <: 0 do. _ else.  
    quo=.(pos#num)%(pos#den) where pos=.den>0 and num=.}.{"1 M  
    P=.pr{C and R=.pr{M where pr=.1+(ml quo){pos#{indx  
    simplex M-(C-BV)*(R%P) where BV=.pr=indx end. end.  
)
```

Appendix D: Primitive J Functions Used

Here are the J primitive functions that were used above. For arithmetic we used the four basic operations of addition, subtraction, multiplication, and division.

$2+3$	$2-3$	$2*3$	$2\%3$
5	<u>1</u>	6	0.666666666667

Note that the negative sign, which is part of a negative number, is the underbar. An underbar by itself is the 'number' positive infinity and negative infinity is . Also note that division is % not / .

$1+$	$1\%0$	$1\%0$
-	-	--

Primitive verbs in J are denoted by single characters like + or by such a character followed by a period or colon like +. or +: . Most primitive function symbols in J have separate monadic and dyadic meanings. For example, the monadic meaning of % is reciprocal. The monadic meaning of <. is 'floor' and the dyadic meaning is 'lesser' (i.e. 'min').

$\%2$	$< . 2.56$	$3 < . 2$	$2 < . 3$
0.5	2	2	2

The obvious analogue statements are true about > . .

The monadic meaning of <: is 'decrement' and the dyadic meaning is 'less than or equals'.

$<: 2.56$	$2 <: 3$	$3 <: 2$
1.56	1	0

There are monadic and dyadic constant functions and monadic and dyadic identity functions:

$0: 2$	$2 0: 3$	$] 3$	$2] 3$
0	0	3	3

(But note that $2[3$ is 2 because $[]$ is the identity function of its left argument, ignoring the right.) Constant functions $0:$ to $9:$ are defined; to make more general constant functions, the conjunction " may be used. Thus $10"0$ is a constant function that has value 10 on 0-cells. Applied to a matrix, it gives a matrix of 10s. Or $10"1$ has value 10 on arguments of rank 1. Applied to a matrix it gives a 10 for each row of the matrix. And $10"_$ applied to a matrix gives the number 10.

We used both the monadic and the dyadic versions of the adverb /. The monadic version applies its left argument (a dyadic verb) between the items of its right argument.

```

      2+3+4      +/2 3 4      */2 3 4      <./7 3 9 6
9                                24                                3

```

The dyadic version gives a 'multiplication table' for the verb to the left

```

      2 3 4 */ 7 8 9
14 16 18
21 24 27
28 32 36

```

The & symbol is a conjunction. One common use of it is to combine a dyadic verb with an argument to make a monadic verb.

```

f =: 7&+      g =: %&4      f 2 3 4      g 2 3 4
9 10 11      0.5 0.75 1

```

Composition of functions is represented by @ .

```

f g 2 3 4      (f@g)2 3 4      7+(5+6)      5(f@+)6
7.5 7.75 8      7.5 7.75 8      18      18

```

For working with arrays (vectors, matrices and higher arrays) the following operations are available.

```

      0 2 _1{8 4 3 7      {.8 3 4 7      {:8 3 4 7
8 3 7      8      7
      }.8 3 4 7      }:8 3 4 7      2{.8 3 4 7
3 4 7      8 3 4      8 3
      2}.8 3 4 7      i. 5      8 3 4 7 i. 4
4 7      0 1 2 3 4      2      M =: 2 2$8 3 4 7
      $8 3 4 7      #8 3 4 7      #M
4      4      2
      M      $M
8 3      2 2      2
4 7

```

The last two functions show that M has the shape of a two-by-two matrix and that M has two items (each item is a row).

The conjunction " is used to specify how a verb will act on an array. A matrix is a 2-cell, each of its rows is a 1-cell, and its elements are 0-cells.

9,M	9(,"2)M	9(,"1)M	
9(,"0)M			
9 9	9 9	9 8 3	9 8
8 3	8 3	9 4 7	9 3
4 7	4 7		
			9 4
			9 7

The power conjunction ^: is used to iterate a function.

(f^:3) 3	7+7+7+3	(g^:3) 16	(g^:_) 16
24	24	0.25	0

If an iteration gives the same result as the previous case (to within a small tolerance) then the iteration stops. Therefore limits of iterations can be programmed by simply using the infinite power.

Gerunds are arrays of verbs that can be handled like nouns and then used as verbs. For example

```
ger =: f'g
```

is a gerund with components *f* and *g*. On the one hand we can treat it like a noun by selecting its components or by shaping it into a matrix, etc. On the other hand we can evoke it as a verb in a variety of ways. One way is through the use of @. which is called 'agenda'. The expression *f'g@.v x* applies the verb *v* to the argument *x* to get a result *v(x)* which should be an integer, in this case either 0 or 1. The index *v(x)* is used to select *f* or *g* to act on *x*. Thus the final answer will be *f(x)* if *v(x)* is 0 or *g(x)* if *v(x)* is 1. In effect, we get the result *f(x)* else *g(x)* if *v(x)=1*. Thus we make the code *h=:f'g@.v* more readable by defining

```
else =: '
if =: @.
h =: f else g if v
```

Richard Brown
 Dept of Mathematics and Statistics
 York University
 4700 Keele Street, North York
 Ontario, CANADA M3J 1P3

J Release 2.05 for Windows

review by Dave Ziemann - concluded

Dynamic Data Exchange (DDE)

The depth of the window driver scripting language is impressive; I counted over 180 window driver commands in the User Manual appendix, covering not only window programming calls, but also graphics, Windows program execution and DDE and OLE (not OLE 2 yet). I could not find a way of making an arbitrary Windows API call.

The User Manual describes how to do DDE cold and hot links, and there are sample scripts to try, including hot and cold J to J links and cold Word for Windows to J links. The documentation did not make it clear that the WordBasic macros are for Word for Windows 6, and do not work for version 2.

I decided to try a DDE example. What I wanted was something that would be relevant to the preparation of this review. I got fed up alt-Tabbing out of my Word session into J, where I would type in an expression, execute it, select the expression and its result, copy them, alt-Tab back to Word, paste the text, and then finally re-select it all and set the appropriate ISIJ font and size.

What if I could get J and Word to do it for me? The first step is to write a WordBasic macro to take the currently selected text, push it into J via DDE, get the result back from J, and insert it after the selected text, as a new paragraph with the font set to 13 point ISIJ. WordBasic macros are very boring, but the salient DDE lines are:

```
cn = DDEInitiate("jserver", "mytopic")
DDEPoke cn, "myitem", Selection$()
ans$ = DDERequest$(cn, "myitem")
```

The code assumes that the J DDE server will establish a link with name "jserver", and that the DDE topic and items names are "mytopic" and "myitem". The variable cn holds the DDE channel number. In a cold link the client has to explicitly request the data from the server, which is shown in the last line above

The next step is to establish the DDE cold link between J and Microsoft Word for Windows. I was able to modify an example J script to do this. The script

establishes the DDE server name and then waits for the DDE client to do something:

```
wd 'ddename jserver;'
...
wd 'wait;'
```

*type	11 ddepoke
*parent	sysddepoke
*topic	mytopic
*item	myitem
*data	i.2 8

This result is returned when the client macro executes. It contains the expression to be executed alongside the *data identifier. The script then formats the result, turns it into a character vector and returns it to the client as follows:

```
wd 'ddereqd ',charvec,',';
```

The character vector appears as the ans\$ variable in the WordBasic macro, which then continues with code to insert, select and set the J font for the result text. The final step was to add a "J" option to the bottom of my Word for Windows "Tools" menu, which runs the stored J macro. The experiment seemed to work quite well, although the cold link technique is timing sensitive, so that problems can arise if J isn't ready with the result when WordBasic asks for it. I only got one GPF during the whole business, and I'm not at all sure J was to blame.

J Runtime

J Professional includes a runtime facility. Installing the separate runtime disk creates a new directory containing the runtime J execution program jrt.exe. I then used J Windows Help to learn how to build my runtime. I wanted to try and runtime the J server application I had just built. Conveniently (but strangely) you use the window driver to convert the application script file into a special kind of encoded runtime script file, as follows:

```
wd 'makejr c:\jrt2\jsr.js jsr.jr;'
```

Then all you do then is execute the runtime program jrt.exe with the jr file name as its parameter. And it worked first time when I tried it. Now I can just double-click a Program Manager icon, and my J server starts up, ready to accept DDE

requests from my Word session. The jr file was just slightly larger than the source script file, about 4kb. The jrt.exe and j387.dll files are about 360kb apiece.

For a really professional look you can package your runtime application into a tailored Windows Setup disk which contains everything you need to install the application on another machine.

The J Product Set and Pricing

J release 2 is available in three forms; the Professional, Personal and Freeware editions. The earlier shareware version is no longer distributed or supported. A Professional licence is required for any work-related use of J, and includes royalty-free runtime, which is not included in the Personal edition. The Freeware edition of J is based on the J2.0 release, and is therefore slower and also lacks DDE, ODBC and DLL support, most VBX support, the GUI form editor, the component filing system, and language enhancements since J2.0. In other respects J Freeware is a full J/Windows system, and is ideal for learning and demonstrating J.

ISI have clearly been concentrating on J for Windows, although the core product is now available on DOS, OS/2, Linux, Mac, RS/6000 and Sparc platforms. At the time of writing (March) the application development environment is not available for these platforms, although new releases which include this feature are expected for the OS/2 and Mac systems later this year.

J Professional is \$495 (US Dollars). J Personal (and any system for which the development environment is not available) costs \$100. These systems are also available unbundled, i.e. the disks cost \$40 and the manuals are \$35 apiece. With the full text of both manuals now available as Windows Help, the paper copies are no longer a necessity.

Teachers can install a single Personal system used as part of a course on all computers in the lab, although students needing their own copies will have to buy their own Personal system. The Professional system is available at a 60% education discount.

J is distributed by Strand Software Systems, and authorised dealers. Many J resources are available via anonymous FTP from [watserv1.waterloo.edu](ftp://watserv1.waterloo.edu). J Freeware can be found in the directory `languages/j`.

Thanks are due to Chris Burke and Eric Iverson for answering my questions during the preparation of this review, and to Peter Biddlecombe for helping me with WordBasic.

Sustaining Members' News: Dyadic Systems Limited

The latest addition to Dyalog APL/W is a new system function `⌈CS` (Change Space), a significant enhancement to the namespace concept. Dyalog APL namespaces allow you to organise your workspace in a similar manner to the way you may organise your disk into different sub-directories. However, this is not all that namespaces do and perhaps more importantly, namespaces provide *separate execution contexts* in which your APL functions run.

For example, if you make a namespace called *MODEL* in which you install a function called *CALC*, when you run *MODEL.CALC* the function sees only the other functions and variables within the *MODEL* namespace and avoids potential name conflicts with other workspace objects. This is exactly right if *MODEL* represents a self-contained sub-system, but what if *MODEL* represents a particular scenario and you have several other namespaces; *PLAN*, *ACTUALS*, *FORECAST*, and so forth, containing similar, but independent scenarios, that you want to analyse. One thing you do not want to do is to maintain copies of *CALC* and any sub-functions it calls throughout your namespaces, except where the functions differ. This is where `⌈CS` comes in.

`⌈CS` provides a mechanism for a function to switch its execution context. This means that your *CALC* function can switch into the *PLAN* namespace if you want to analyse the *PLAN* scenario, or into the *ACTUALS* namespace if you want to use actual data. Functional differences between the way certain calculations are performed can be handled by having different sub-functions within the different namespaces. Functions that apply to all scenarios can be exported to the target namespace (the one that `⌈CS` switches into) by specifying their names in its optional left argument.

This technique lets you implement a class of objects (in this case a planning system) as a namespace (*MODEL*) which contains the base data variables (properties) and functions (methods) that define the behaviour of the class itself. Instances of the class are represented by other namespaces *PLAN*, *ACTUALS*, *FORECAST*, which contain just the *differences* (different data and perhaps some different calculation routines) between each scenario and the class itself. The main calculation routines exist only in the *MODEL* namespace. When you execute *MODEL.CALC*, you supply the name of the instance namespace in which it is to run. *CALC* then uses `⌈CS` to switch into the desired instance namespace, taking with it any class data and sub-functions that it needs.

This technique is not the only way to solve problems of this sort, but it adds yet another powerful tool to your armoury and is a further step towards the goal of object-oriented APL.

Index to Advertisers

Beautiful Systems Inc	2
Dyadic Systems Ltd	33
MicroAPL	48
Soliton	6
Strand Software	4
APL Booklist (Renaissance Data Systems)	32
Vector Back Numbers	47

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Compuserve: 100331,644.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Anthony Camacho,
11 Auburn Road, Redland,
BRISTOL, BS6 6LS
Tel: 0117-973 0036
Email: acamacho@cix.compulink.co.uk

Authors wishing to use Windows Write or Word for Windows should contact Vector Production for a copy of the Vector APL TrueType font and Vector APL typebox.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO6 4JJ.

Tel: 01439-788385
Compuserve: 100331,644.

British APL Association: Membership Form

Membership is open to anyone interested in APL. The membership year normally runs from 1st May to 30th April, but new members may join from 1st August, November or February if preferred. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____

Address: _____

Postcode / Country: _____

Telephone Number: _____

Email Address: _____

Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐

UK private membership £12 ☐

Overseas private membership £14 ☐

Airmail supplement (not needed for Europe) £4 ☐

UK Corporate membership £100 ☐

Corporate membership overseas £135 ☐

Sustaining membership £430 ☐

Non-voting UK member (student/OAP/unemployed only) £6 ☐

PAYMENT — in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Data Protection Act
 The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON E3 5HU, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1994/95 Committee

Chairman:	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Secretary:	Sylvia Camacho 0117-973 0036	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Treasurer:	Nicholas Small 0181-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Anthony Camacho 0117-973 0036 acamacho@cix.compulink.co.uk	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Activities:	Duncan Pearson 01653-618900 100265.1564@compuserve.com	Keeper's Cottage, Firby, YORK YO6 4LH
Education:	Dr Ian Clark 01388-527190 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Technical:	Jonathan Barman 01488-648575 100116.1030@compuserve.com	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Projects:	George MacLeod 01442-878065 100412.1305@compuserve.com	Greymantle Associates Ltd., Bartrum House, Ravens Lane, BERKHAMSTED, Herts HP4 2DY
Publicity:	David Eastwood 0171-922 8866 microapl@applelink.apple.com	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jon Sandles 01904-411635	22a Arthur Street, Lawrence Street, York YO1 3EL
Administration:	Rowena Small 0181-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Anthony Camacho	0117-973 0036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan & Bridget Barman (01488-648575), Ray Cannon (01252-874697), Richard and Adam Weber (01302-539761), Sylvia Camacho, Duncan Pearson, John Searle (0181-858 6811), David Ziemann (0171-267 8032), Jon Sandles	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
5 The Mallings, Castlegate,
MALTON, North Yorks YO17 0DP
Tel: 01853-696760
Fax: 01653-697719
Email: 100265.1564@compuserve.com

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: Sales@dyadic.com

Insight Systems Aps
Nordre Strandvej 119A
DK-3150 Hellebæk
Denmark
Tel: +45 42 10 70 22
Fax: +45 42 10 75 74
Email: insight@inet.uni-c.dk

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 0171-922 8866
Fax: 0171-928 1006
Email: microapl@microapl.demon.co.uk

Dutch APL Association
Postbus 1341
3430 BH Nieuwegein
Netherlands
Tel: 03474-2337

Compass R&D Ltd
10 Frederick Sanger Road
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel: 01483-302249
Fax: 01483-302279

BMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0171-353 8900
Fax: 0171-353 3325
Email: 100020.2632@compuserve.com

Manugistics
2115 East Jefferson St
Rockville
MARYLAND 20852 USA
Tel: +1 (301) 984-5412
Fax: +1 (301) 984-5094
E-mail: apisales@manu.com (US)
Email: intl@manu.com (international)

Soliton Associates Ltd
Groet Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: jh@soliton.com